

无人机投放烟幕干扰弹模型与优化策略研究

摘要

本文针对无人机投放烟幕弹的多重决策问题,建立了一套针对不同场景的决策优化模型。结合运动学、几何学与智能启发算法,给出相应的最优决策方案,实现对目标有效遮蔽时长的最大化。

对于问题一,要求根据已知参数计算烟幕弹的有效遮蔽时长。本文基于物理规律与向量推导,精确刻画了导弹、烟幕弹及烟幕云团在平抛与匀速下沉阶段的运动轨迹,建立了静态场景下有效遮蔽时长的计算模型。接着根据立体几何原理,本文引入**高斯曲率**,严格证明了目标遮挡的有效判据为上、下底面边缘点集是否全被遮挡。本文据此建立了描述**极限遮蔽状态的非线性方程组**,求解得到在给定的策略下,单枚烟幕弹对导弹M1的总有效遮蔽时长为**1.400555**秒。最后对模型进行**灵敏度分析**和**鲁棒性分析**,说明模型精度较高且稳定性好。

对于问题二,本文建立以无人机航向角、速度、烟幕弹投放与起爆时刻为决策变量的**单目标优化模型**,创新性地引入导弹是否**穿云**对遮蔽情况的影响,并结合**垂距公式**与**向量投影系数**,构建了完备的导弹遮蔽判定准则。考虑到遮蔽时间连续,本文采用**由粗到细**的二分搜索法确定遮蔽首末时刻并计算遮蔽时长,并通过**梯度化适应度粒子群算法**求解模型。结果显示,最优策略为航向角**6.67°**、速度**137.53 m/s**、投放点为**0.066 s**、起爆点为**0.772 s**,总有效遮蔽时间为**4.587695 s**。

对于问题三,本文在问题二模型基础上,补充对**多云团遮蔽情况**的详细分析,并将目标点采样范围修正为圆柱侧面与上底面。考虑到此时遮蔽时长不再连续,本文用**测度**量化遮蔽时长。通过**向量化矩阵运算**与**变步长二分搜索**对已有粒子群算法进行改进,可求得总有效遮蔽时长为**6.4031 s**,并进行可视化分析。结果详见 result1.xlsx。

对于问题四,本文在问题三基础上引入无人机分配变量,采用全局搜索能力更强的混合进化算法求解。结果表明,该场景下的遮蔽时长进一步提升,得到总有效遮蔽时长为**11.134009 秒**,结果详见 result2.xlsx。

对于问题五,本文将遮挡判定条件重构为三导弹视线均被遮挡,并构建了**联合遮蔽优化模型**,以**遮蔽时间交集测度最大化**为目标函数,融合三导弹遮蔽轨迹并行判别、时空约束与组合规划算法,在高维复杂空间中实现最优解的搜索,最终获得总遮蔽时长为**20.31 秒**,对应策略详见 result3.xlsx。

关键字: 改进粒子群优化算法 射影几何 高斯曲率 向量化矩阵运算 变步长二分搜索

一、问题重述

1.1 问题背景

本文探究在现代军事防御场景中,利用无人机投放烟幕干扰弹以保护高价值目标免受精确制导导弹攻击的优化策略问题。防御系统包括一个真目标和一个为其提供掩护的假目标,坐标系以假目标为原点建立。来袭的空地导弹以 300 m/s 的速度直线飞向假目标,当警戒雷达探测到多枚来袭导弹时,控制中心会立即向待命的无人机指派拦截任务,通过投放烟幕弹形成一个短暂的、可移动的遮蔽云团,来阻断导弹的瞄准视线。

无人机接收任务后,可瞬时调整航向,并以 $70\text{-}140\text{ m/s}$ 范围内的某一恒定速度及高度作匀速直线飞行。无人机投放的烟幕弹在重力作用下下落,并在预定时间引爆,形成一个半径为 10 m 、可持续提供 20 s 有效遮蔽的球状烟幕云团,该云团会以 3 m/s 的速度匀速下沉。为实现最优防御效果,需要建立数学模型,优化无人机的飞行路径、速度,以及烟幕弹的投放与引爆时机,以达到最大化对真实目标的总有效遮蔽时间的总目标。

1.2 问题提出

为解决烟幕干扰弹的投放策略优化问题,实现更有效的烟幕干扰效果,需建立数学模型解决以下问题:

问题 1: 无人机 FY1 以 120 m/s 的速度飞向假目标,在受领任务 1.5 s 后投放一枚烟幕弹,该弹于 3.6 s 后引爆。求解此策略下烟幕弹对导弹 M1 的有效遮蔽时长。

问题 2: 允许无人机 FY1 自主选择飞行方向和速度,并决定其所投放的一枚烟幕弹的投放点与引爆点。目标是设计一个最优策略,使对导弹 M1 的有效遮蔽时间尽可能长。

问题 3: 在问题一、二的基础上,可投放烟幕弹的数量增加到 3 枚,要求给出一套完整的投放策略,包括每枚弹的投放与引爆细节,以实现导弹 M1 的干扰。

问题 4: 要求利用 FY1, FY2, FY3 三架无人机,各投放一枚烟幕弹,协同对导弹 M1 进行干扰。重新设计相应的协同投放策略。

问题 5: 5 架无人机全部可用,每架最多可投放 3 枚烟幕弹,以应对 M1, M2, M3 三枚导弹攻击。要求给出一套最优的烟幕弹投放策略,以实现有效遮蔽时间的最大化。

二、问题分析

对于问题一,要求在无人机速度及方向、烟幕干扰弹投放点和起爆点给定的预设条件下,求算烟幕干扰弹对真目标的最长遮蔽时间。

利用物理运动学知识,可以求得导弹、无人机、烟幕弹和烟幕云团的轨迹方程,同时根据数学几何知识可以得出点与直线的距离,进而求得圆柱体被遮蔽的判定条件,随后求解方程组可以得到导弹开始遮蔽和结束遮蔽的时刻,两时刻相减即为总遮蔽时长。

对于问题二,不再限制无人机速度及方向等问题一中给定的参数,求算最长有效遮蔽时间。同样地,可以在问题一模型的基础上添加约束条件,同时对问题一的遮蔽判定条件加以完善,如考虑出云团时的情况,进而转化为一个简单单目标优化问题求解。

对于问题三,可用无人机数量不变,可投放的烟幕干扰弹数量增加到 3 个,要求给出最优投放策略。本问中决策变量增加为 8 个,将单目标优化问题变为协同规划。与前两问不同,本问中需要考虑多个云团协同遮蔽的情况,关键在于处理三枚烟幕弹在三维坐标系下的时空关系,并满足投放间隔约束,模型的目标函数相应地变为最大化多个遮蔽时间集合的并集测度,同样对模型进行优化求解。

对于问题四，可用无人机数量增加到 3 架，但每架无人机可投放的烟幕干扰弹只有一枚，协同对一个来袭导弹进行干扰，要求给出最优投放策略。本题可以对问题三模型进行改进，添加无人机编号变量 j ，决策变量仍为 8 个，对约束条件进行必要补充，再结合问题三给出的优化模型进行求解。

对于问题五，将场景升级为 5 架可投放 3 枚烟幕干扰弹的无人机共同防御 3 枚来袭导弹。相比前四问，导弹的数量不再只有一个，因此有效遮蔽必须是对三枚导弹的同时遮蔽，利用问题四模型分别求算出三枚导弹各自的有效遮蔽时间，再通过取交集可以求算出真正有效遮蔽时间。

三、模型假设

为保证模型的准确性与合理性，本文结合实际情况，排除部分干扰因素，做出以下假设：

- 假设 1 无风场因素干扰，烟幕干扰弹爆炸后形成的烟幕云团水平速度为零；
- 假设 2 不考虑烟幕云团在消散前坠地的影响；
- 假设 3 为提升结果准确性，本文 g 取 $9.80665m/s^2$ ；
- 假设 4 不考虑起爆时，导弹恰好在云团内部；
- 假设 5 当导弹发现真目标时会立刻改变方向销毁真目标。

四、符号说明

符号	含义	单位
$P_{j,i}$	第 j 架无人机投掷的第 i 枚烟幕弹坐标	/
$M_K(t)$	t 时刻编号为 k 导弹的坐标	m
C_S	烟幕云团中心坐标	m
x	烟幕干扰弹的 X 轴坐标	m
y	烟幕干扰弹的 Y 轴坐标	m
z	烟幕干扰弹的 Z 轴坐标	m
t	距雷达发现导弹时刻的时间	s
v_D	烟幕干扰弹的初速度	m/s
v_S	烟幕云团匀速下降的速度	m/s
$\vec{e}$	单位方向向量	/
γ	轮廓曲率	/
$d(t, P)$	烟幕云团中心到导弹视线的垂距	m
Ω	有效遮蔽时间集合	s
α	无人机航向角	$^\circ$
σ_t	t 时刻向量投影	/
ε	视线段归一化投影系数	/

五、问题一的模型的建立和求解

本题要求在给定无人机飞行方向及飞行速度、以及烟幕弹的投放时机和起爆时机的前提下，求解烟幕干扰弹对导弹 M1 的有效遮蔽时长。不考虑风场等物理因素的干扰，本题可以视作一个静态场景。

5.1 静态场景下有效遮蔽时长的计算模型建立

为清晰计算烟幕干扰弹对 M1 的有效遮蔽时长，我们分别建立无人机 FY1、导弹 M1 和烟幕云团的运动轨迹模型，接着对有效遮蔽的情况与时长进行分析，模型建立过程如下：

5.1.1 建立笛卡尔三维坐标系

为对问题中的场景进行精确的数学描述，我们以假目标的中心为坐标原点 $O(0,0,0)$ ，水平面定义为 xy 平面，垂直于水平面向上的方向为 z 轴正方向，建立了一个三维右手笛卡尔坐标系 ($O-xyz$)，如图5-1所示。

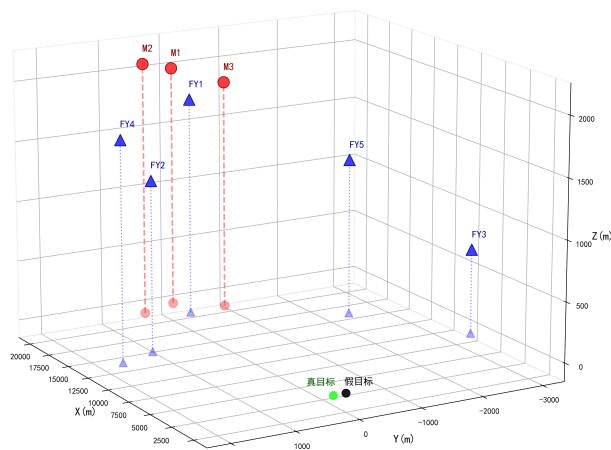


图 5-1 初始时刻各物体的位置

5.1.2 烟幕云团的轨迹方程

设本问中烟幕干扰弹在运动过程中的坐标为 $P_1(x_1(t), y_1(t), z_1(t))$ ，简记为 $P_1(t)$ ，由题目得其初始位置为 $P_1(0) = (17800, 0, 1800)$ 。要求得其运动轨迹方程，主要分为三个阶段求解，第一阶段烟幕干扰弹跟随无人机 FY1 做朝向假目标方向的等高度匀速直线运动；第二阶段烟幕干扰弹被投放，由于拥有水平方向的初速度，在重力作用下做平抛运动；第三阶段烟幕干扰弹在空中完成爆炸，瞬时形成烟幕云团，由题意知该烟幕云团以 3m/s 的速度匀速下沉，水平方向速度变为零，直到云团消散。

(一) 匀速直线运动阶段：

在此阶段无人机载着烟幕干扰弹以 120m/s 的初速度等高度匀速飞行，方向为朝着假目标 (原点) 的位置，烟幕干扰弹的三维坐标的计算公式为：

$$\begin{cases} x(t) = 17800 - v_{D1}t \\ y(t) = 0 \\ z(t) = 1800 \end{cases} \quad (5-1)$$

此阶段烟幕干扰弹在三维笛卡尔坐标系下的坐标为：

$$P_1(x(t), y(t), z(t)) = (17800 - v_{D1}t, 0, 1800) \quad (5-2)$$

其中，时间需满足 $0 \leq t \leq t'$ ，第一问中投放烟幕干扰弹时间 $t' = 1.5s$ ，无人机速度为 $v_{D1} = 120m/s$ 。

(二) 平抛阶段：

在此阶段无人机将烟幕干扰弹完成投放，由于 Y 轴坐标始终为零，我们对其在 X-Z 平面上进行运动学分析，我们对无人机做受力分析，在问题背景下忽略空气阻力，仅受

自身重力，在 z 轴方向上做自由落体运动。其余方向不受力。因此无人机将在空中做水平初速度为 120m/s 的平抛运动，如下图所示：

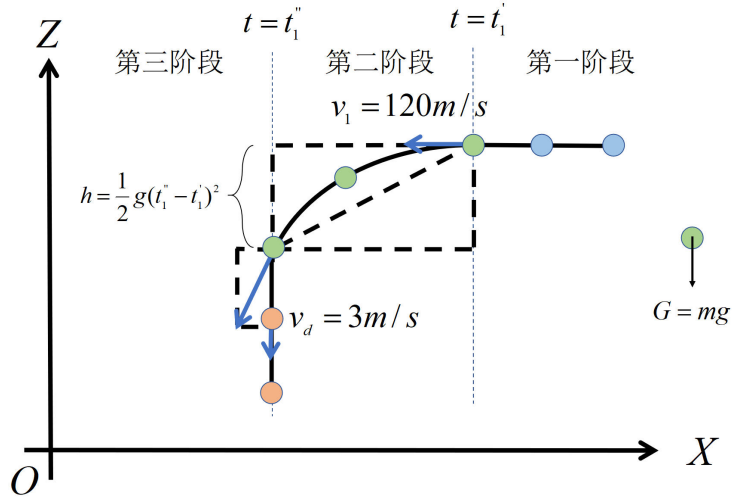


图 5-2 问题一烟幕云团轨迹示意图

如图，我们可以将平抛曲线运动分解为水平和竖直方向上的直线运动，水平方向的速度为 120m/s ，则 X 轴坐标为：

$$x(t) = 17800 - v_{D1}t \quad (5-3)$$

垂直方向上受到重力作用，做自由落体运动，加速度为 g ，依据假设，加速度 g 取为 9.80665 。根据能量守恒定律计算， Z 轴坐标为：

$$z(t) = 1800 - \frac{1}{2}g(t - t')^2 \quad (5-4)$$

故此阶段烟幕干扰弹在三维笛卡尔坐标系下的坐标为：

$$P_1(x(t), y(t), z(t)) = \left(17800 - v_{D1}t, 0, 1800 - \frac{1}{2}g(t - t')^2 \right) \quad (5-5)$$

其中，时间满足 $t' \leq t \leq t''$ ，在本小问中，烟幕干扰弹起爆时间 $t_1' = 1.5 + 3.6 = 5.1\text{s}$ 。

(三) 匀速下落阶段：

在此阶段烟幕干扰弹完成爆炸，形成烟幕云团，我们的问题转为求解烟幕云团的运动轨迹。我们仍对其在 X - Z 平面上进行分析，根据题意，烟幕云团以 3m/s 的速度匀速下沉，水平速度变为 0 。

则烟幕云团的三维坐标的计算公式分别为：

$$\begin{cases} x(t) = 17800 - v_{D1}t'' \\ y(t) = 0 \\ z(t) = 1800 - \frac{1}{2}g(t'' - t')^2 - 3(t - t'') \end{cases} \quad (5-6)$$

整理得：

$$P_1(x(t), y(t), z(t)) = \left(17800 - v_{D1}t'', 0, 1800 - \frac{1}{2}g(t'' - t')^2 - 3(t - t'') \right) \quad (5-7)$$

题中指出云团在起爆 20 秒后消散，故时间满足 $t'_1 \leq t \leq t'' + 20$ 。
烟幕云团的轨迹方程可总结为：

$$\begin{cases} P_1(t) = (17800 - v_{D1}t, 0, 1800) & 0 \leq t \leq t' \\ P_1(t) = \left(17800 - v_{D1}t, 0, 1800 - \frac{1}{2}g(t - t'')^2 \right) & t' \leq t \leq t'' \\ P_1(t) = \left(17800 - v_{D1}t'', 0, 1800 - \frac{1}{2}g(t'' - t')^2 - 3(t - t'') \right) & t'' \leq t \leq t'' + 20 \end{cases} \quad (5-8)$$

5.1.3 导弹 M1 的轨迹方程

设导弹 M_1 的坐标为 $M_1(t)$ ，其初始位置为 $M_1(0) = (20000, 0, 2000)$ 。由题意得，导弹 M1 以 $300m/s$ 的初速度朝向假目标（原点）飞行，运动轨迹可抽象为简单的匀速直线运动。令 $\vec{e}_1$ 为原点 O 指向导弹 M_1 的单位方向向量：

$$\vec{e}_1 = \frac{(20000, 0, 2000) - (0, 0, 0)}{|OM_1|} = \frac{(20000, 0, 2000)}{|OM_1|} \quad (5-9)$$

则用向量表示导弹 M_1 位置为：

$$M_1(t) = (20000, 0, 2000) - 300\vec{e}_1 t \quad (5-10)$$

5.1.4 有效遮蔽的情况分析

为计算遮挡时长，需确定圆柱体目标在球形遮挡物下的临界遮蔽时间。由于目标具有体积，故不能简单看作质点，而需详细刻画其可见部位。根据视觉几何原理，可见轮廓为表面法向量与视线方向垂直的边界点，这些点形成图像边缘，是我们识别的关键。因此在极限遮蔽状态（视线与球体相切）下，圆柱体的最后可见轮廓可能出现在以下三处：（1）上底面或下底面的边缘弧线；（2）既包括底面边缘又包括侧边；（3）仅侧边直线部分。

三种遮蔽情形示意图如下：

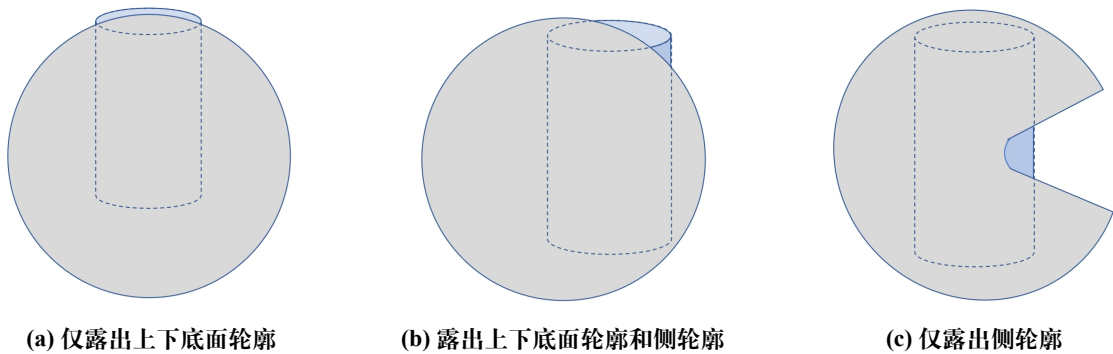


图 5-3 圆柱体被遮挡时的典型可见轮廓情形示意图

根据实际推断，极限遮蔽下仅可能出现情形（a），另外两种情形均不成立，下文将给出详细证明：

考虑到遮挡物与目标轮廓的凸凹性，我们引入高斯曲率进行几何刻画，将三维空间中的观察点、圆柱体与球体在像平面上作中心投影，记球的投影外轮廓为圆弧集合

$\Gamma = \partial\pi(S)$ ，由 Γ 围成的闭盘为 D 。设圆柱在像平面上的轮廓含有一条来自侧面的线段 L ，端点为 A, B 。

平面曲线的有向轮廓曲率公式^[1]为：

$$\kappa(t) = \frac{|\gamma'(t) \times \gamma''(t)|}{|\gamma'(t)|^3} \quad (5-11)$$

$$\gamma(t) = (x(t), y(t)) \quad (5-12)$$

由投影几何可知，遮挡物在像平面上的可见外轮廓为圆，其任一点曲率为正。圆柱侧边在像是直线段 L ，对 L 上任一点曲率为 0。

基于此，对 (b)(c) 两种极限遮蔽情况分析如下：

① 仅露出侧轮廓：

不妨设上下表面均被球遮挡，但侧边直线段 L 的内段 I 在像平面上可见，端点 A, B 与上下表面均落在遮挡区域 D 。在靠近 A, B 的接口处，像上可见边界两端分别来自球的投影轮廓和侧边直线，但由于球的投影边界为处处 $\kappa > 0$ 的光滑凸曲线，若其在开区间内与直线段相接，则在接口处曲率由大于零的数突跳为 0，会违背几何性质，故此情形不可能发生。

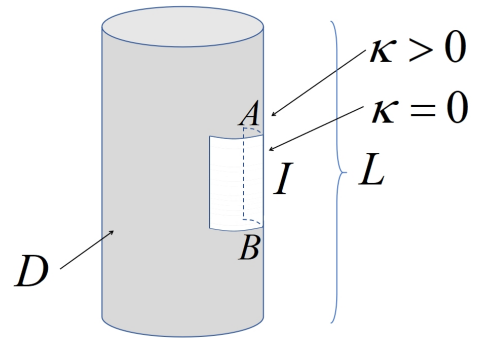


图 5-4 仅露出侧轮廓分析图

② 露出上下底面轮廓和侧轮廓：

假设上底面边缘弧段 C 与侧边直线段 L 在像上同时可见，而下底面及侧边其余部分被遮挡。设 P 为 C 与 L 的接口点，则可见边界在 P 附近由上底面的凸弧 C 与直线段 L 相接，接口处曲率同样会发生跃迁，违背了几何性质。

又由遮挡区域 D 是一个凸集，其中任意两点连线段仍在 D 内，又因为 L 作为可见边界，则 L 的端点皆在 D ，可知整段 $L \subset D$ ，从而 L 不可能在像上出现在可见域 D^c ，与假设相违背。因此，在极限遮蔽下，目标的最后可见部分不可能同时包含上底面边缘与侧边直线段。

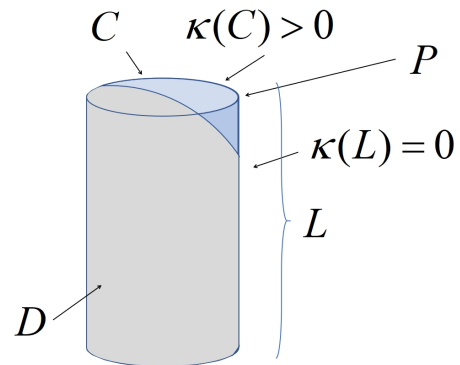


图 5-5 露出上下底面轮廓和侧轮廓

至此，我们证明了在极限遮蔽条件下，视线与遮挡体相切的点，必然是圆柱体上、下底面边缘上的点，并将点集记为 $\mathcal{S}_{\text{top}} \cup \mathcal{S}_{\text{bottom}}$ 。

5.1.5 有效遮蔽时长的判定

为了准确计算有效遮蔽时长，我们需要找到一个准确的判定依据，用于判断在某一段时间内，烟幕干扰弹产生的烟幕云团是否能有效遮蔽目标，从而阻断来袭导弹与真目标之间的视线。在任意时刻 t ，如果该连线与烟雾云团的几何球体相交，则认为此刻遮蔽有效，有效遮蔽时长即为所有这些有效时刻的总和。下文我们阐释有效遮蔽的判定方法：

连接导弹 $P_M(t)$ 与目标上任意一点 P 的视线为一条直线。根据向量叉乘的几何意义，烟雾云团中心 $C_S(t)$ 到该视线直线的距离 $d(t, P)$ 为：

$$d(t, P) = \frac{|\overrightarrow{P_M(t)P} \times \overrightarrow{P_M(t)C_S(t)}|}{|\overrightarrow{P_M(t)P}|} \quad (5-13)$$

其中，导弹位置为 $P_M(t) = (x_M(t), y_M(t), z_M(t))$ ，烟雾云团中心 $C_S(t) = (x_S(t), y_S(t), z_S(t))$ 。

将上式展开为坐标形式得：

$$d(t, P) = \frac{|(P - P_M(t)) \times (C_S(t) - P_M(t))|}{|P - P_M(t)|} \quad (5-14)$$

在时刻 t ，如果上述距离小于或等于烟雾云团半径 $R_S = 10$ ，则视线被遮蔽，遮蔽判定条件为：

$$d(t, P) \leq 10 \quad (5-15)$$

要使整个圆柱体目标被遮蔽，必须保证其上、下表面边缘上的所有点都满足遮蔽条件，这需要满足在所有点中计算出的距离 $d(t, P)$ 的最大值不大于 10。因此，完整的目标被遮蔽的判定为：

$$\max_{P \in S_{top} \cup S_{bottom}} \left(\frac{|(P - P_M(t)) \times (C_S(t) - P_M(t))|}{|P - P_M(t)|} \right) \leq 10, t \in [t_1'', t_1'' + 20] \quad (5-16)$$

若该不等式成立，则判定在时刻 t 目标被完全有效遮蔽。需要特别注意的是，导弹穿出云团瞬间，导弹与圆柱体上点的连线与云团中心的距离也是小于 10 的，此时并不是相切的临界点，但此刻失去了烟雾云团的遮挡。在问题一的特殊位置下，出云位置，云团中心都位于 XOZ 平面上，并不会出现上述情况，后续模型需要考虑上述判定条件的适用范围。

本问场景中还需要考虑遮蔽情况的连续性，即烟幕云团对导弹的遮蔽情况是按照“未开始遮蔽—开始遮蔽—导弹穿云—导弹穿出云团遮蔽结束”的顺序进行的，不会出现两段遮蔽时长。因此，对于问题一遮蔽时间计算，我们可以采取计算遮蔽开始、结束时间节点，进而计算时长。

为便于计算有效遮蔽时长，我们将满足条件 (5-15) 的时间集合记为

$$\Omega = \left\{ t \in [t'', t'' + 20] \mid \max_{P \in S_{top} \cup S_{bottom}} d(t, P) \leq 10 \right\}. \quad (5-17)$$

由于在问题一中，遮蔽过程只会出现一次连续的时间区间，因此遮蔽时段可以表示为一个闭区间：

$$\Omega = [T_0, T_1], \quad (5-18)$$

设遮蔽开始时刻为 T_0 ，遮蔽结束时刻为 T_1 ，数值上等于集合 Ω 的下界和上界，则总遮蔽时长 $T_{总}$ 为：

$$T_{总} = T_1 - T_0 \quad (5-19)$$

5.1.6 静态有效遮蔽时长模型汇总

综上，本文最终建立的静态场景有效遮蔽时长计算模型为：

$$\left\{ \begin{array}{l} \text{云团轨迹: } \begin{cases} P_1(t) = (17800 - v_{D1}t, 0, 1800) & 0 \leq t \leq t'_1 \\ P_1(t) = (17800 - v_{D1}t, 0, 1800 - \frac{1}{2}g(t-t'_1)^2) & t'_1 \leq t \leq t''_1 \\ P_1(t) = (17800 - v_{D1}t''_1, 0, 1800 - \frac{1}{2}g(t''_1 - t'_1)^2 - 3(t-t''_1)) & t''_1 \leq t \leq t''_1 + 20 \end{cases} \\ \text{导弹轨迹: } \begin{cases} \vec{e}_1 = \frac{(20000, 0, 2000)}{|OM_1|} \\ M_1(t) = (20000, 0, 2000) - 300\vec{e}_1t \end{cases} \\ \text{有效遮蔽判定: } \begin{cases} d(t, P) = \frac{|\vec{P_M(t)} \times \vec{P_M(t)} C_S(t)|}{|\vec{P_M(t)} \vec{P}|} \\ \max_{P \in \mathcal{S}_{top} \cup \mathcal{S}_{bottom}} (d(t, P)) \leq 10, \quad t \in [t''_1, t''_1 + 20] \end{cases} \\ \text{遮蔽时间集合与时长: } \begin{cases} \Omega = \left\{ t \in [t''_1, t''_1 + 20] \mid \max_{P \in \mathcal{S}_{top} \cup \mathcal{S}_{bottom}} d(t, P) \leq 10 \right\} \\ \Omega = [T_0, T_1] \\ T_{\text{总}} = T_1 - T_0 \end{cases} \end{array} \right. \quad (5-20)$$

5.2 问题一的模型求解

5.2.1 采样精度的灵敏度分析

基于5.1.4中的分析，判定烟幕有效遮蔽的复杂几何模型即可被简化为从真目标上下圆周采样后，处理直线与球体相交的问题。理论上，圆柱上下圆周存在无数个采样点。但是考虑到模型求解的时间复杂度，本文采用离散化处理，从上下圆周均匀采样若干个点进行判定。

理论上，采样点数量越多，对目标轮廓的描述越精确，计算结果也越逼近真实值。然而过多的采样点会显著增加计算负担，但若采样点过于稀疏，则可能出现“漏检”现象：即当目标的一小部分轮廓已经不再被遮蔽时，模型因未能捕捉到这一变化而错误地判定遮蔽仍在继续，从而导致计算出的有效遮蔽时长被高估。

为了在计算精度和效率之间取得最佳平衡，我们对采样精度进行了灵敏度分析，结果如图 7-1 所示。从图中可以清晰地观察到，当采样点数从 10 增加到约 50 时，计算出的遮蔽时长出现了急剧下降，这验证了我们关于稀疏采样会导致结果高估的判断。当采样点数超过 100 后，遮蔽时长的计算结果迅速收敛并稳定在约 1.4006 秒。继续增加采样点数量对结果的提升微乎其微，但会徒增计算时间。

综合考量，我们选择在上下圆周各均匀采样 **100** 个点的方式进行后续的判定分析。考虑到问题一情景的特殊性，此后的采样精度必须大于本问精度。

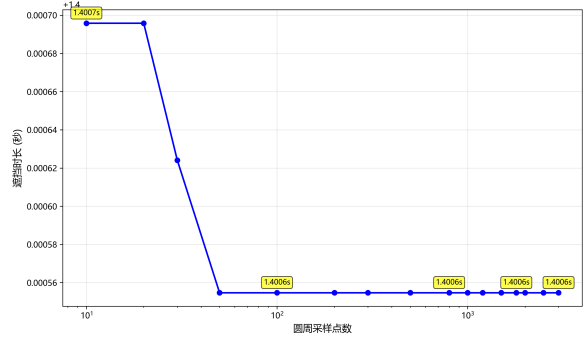


图 5-6 遮挡时长随采样精度变化趋势

5.2.2 有效遮蔽时长计算

至此，本问求解所需的参数已确定。由于遮蔽状态判定函数的高度非线性，我们无法直接求得 T_{start} 和 T_{end} 的解析解。因此，我们设计了“粗搜-精搜”的两阶段数值求解策略，其核心流程如图 5-7 所示。

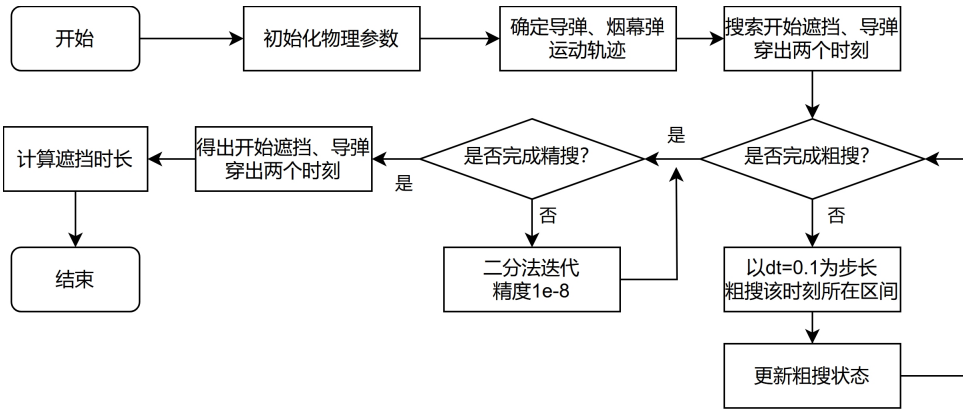


图 5-7 问题一求解流程图

算法的具体步骤阐述如下：

- Step1:** 将导弹、烟幕弹的空间位置描述为随时间 t 变化的向量函数 $\vec{P}_M(t)$ 与 $\vec{P}_S(t)$;
- Step2:** 定义两种状态判定函数：遮蔽判定为在时刻 t ，对所有预设的采样点 $\vec{P}_T$ ，均满足前文建立的有效遮蔽条件式 (5-20)；导弹位于烟幕内判定为 $\|M_1(t) - C_S(t)\| \leq R_s$ 。
- Step3:** 对烟幕弹的有效时间窗口 $T_{start} \sim T_{end}$ ，以 $dt = 0.1$ 为步长进行粗搜，得到遮蔽开始时间区间和导弹穿出烟幕时间区间；
- Step4:** 对两个区间分别进行精度为 $10^{-8}s$ 的二分法精搜，找到两种状态转换的精确时刻，进而得出有效遮蔽时长。

5.3 问题一的模型结果分析

根据5.2.2中介绍的算法，我们通过数值计算得到了问题一中各个关键事件的精确发生时刻，其结果汇总于表 5-1。

表 5-1 关键遮蔽事件的时间节点与持续时长分析

事件	开始时刻 (s)	结束时刻 (s)	持续时长 (s)
目标完全遮蔽	8.047 622 14	9.448 177 19	1.400 555 05
导弹穿越烟幕	9.389 184 27	9.448 177 19	0.058 992 92

最终，我们得出问题一的结论：在题目给定的条件下，烟幕干扰弹对导弹 M1 的有效遮蔽时长为 **1.40055505** 秒。

5.3.1 鲁棒性分析

为检验问题一模型在关键假设被放宽时的可靠性，我们分别对烟幕云团的初始水平速度（无人机速度的 10%-20%）和烟幕云团半径（在 10m 基础上浮动 $\pm 10\%-20\%$ ）进行了 20 次蒙特卡洛模拟。扰动结果如图5-8、5-9所示。

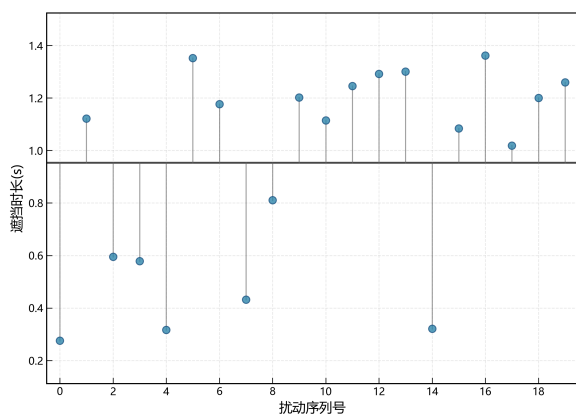


图 5-8 烟幕云团水平速度扰动结果

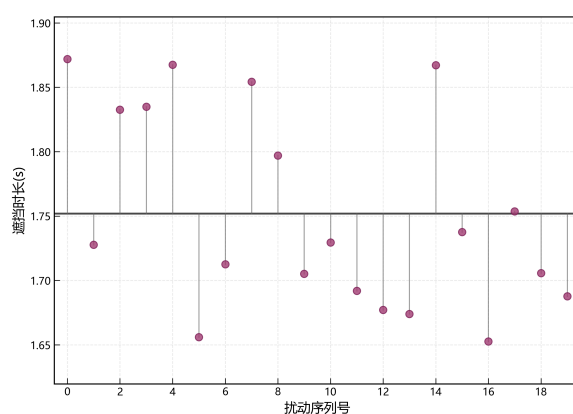


图 5-9 烟幕云团半径扰动结果

分析扰动结果分布，我们得出以下两个结论：

1. 速度扰动影响显著：如图5-8所示，遮蔽时长对烟幕的初始水平速度高度敏感，最大变化率为 $\pm 71.07\%$ 。这是因为烟幕的时空位置直接影响其与高速导弹的几何交汇，微小的位置变化都会被高速运动放大。

2. 半径扰动影响较小：如图5-9所示，遮蔽时长对烟幕的半径变化则表现出良好的稳定性，结果的标准差仅为 $0.0770s$ ，最大变化率控制在 $\pm 6.85\%$ 以内。这说明我们基准模型计算出的 1.40055505 秒，是一个可靠的参考值。

综上所述，可认为我们前文求解的结果是较为准确且鲁棒的。分析同时表明，模型对影响时空位置的动态参数（速度）比对影响几何尺寸的静态参数（半径）更为敏感。

六、问题二的模型的建立和求解

与问题一不同的是，问题二在不限制无人机的运动情况下求解最长遮蔽时间。我们可以对问题一中的模型进行改进，加入未知参数，进而转化为一个单目标优化模型，最后使用粒子群算法进行搜索最优参数。

6.1 单无人机单烟幕弹的动态遮蔽时长优化建模

6.1.1 决策变量约束

根据题意，本文决策变量为航向角、等高速度、投放与起爆时刻四个变量。令航向角为 α ，取值范围为 $[0^\circ, 360^\circ)$ ，根据附件角度确立方式为以 x 轴正方向为零、逆时针方向为正；等高速度 v_D ；题目要求确定干扰烟幕弹的投放点和起爆点坐标。为此，我们引入投放时刻 t' 与起爆时刻 t'' 作为决策变量。在后续求解中，可通过已知的轨迹关系推算出对应的空间坐标。

记单位航向向量为 $\vec{e}(\alpha)$ ，数学表达式为：

$$\vec{e}(\alpha) = (\cos \alpha, \sin \alpha, 0) \quad (6-1)$$

① **时序和范围约束：**各事件发生有严格的时间顺序，且只有在烟幕云团未消散前才会有遮蔽效果，即时间需要满足范围约束：

$$t' < t'' \quad (6-2)$$

$$t \in [t'', t'' + 20] \quad (6-3)$$

② **速度和航向角约束**：本题中规定无人机的速度必须控制在 70 到 140m/s 的范围内，因此有：

$$v_D \in [70, 140] \quad (6-4)$$

$$\alpha \in [0^\circ, 360^\circ] \quad (6-5)$$

6.1.2 状态方程

根据题意，导弹的运动轨迹方程仍同问题一，在此不再赘述。下面我们着重叙述问题二载弹无人机及烟幕云团的运动轨迹：

① $0 \leq t < t'$ ：此阶段无人机载弹飞行，烟幕干扰弹的运动轨迹同无人机：

$$P_1(t) = P_1(0) + v_D t \vec{e}(\alpha) = (17800 + v_D \cos \alpha t, v_D \sin \alpha t, 1800) \quad (6-6)$$

② $t' \leq t \leq t''$ ：此阶段为无人机投放烟幕干扰弹到烟幕干扰弹爆炸这段时间内，我们研究烟幕干扰弹的运动轨迹，数学表达式为^[2]：

$$\begin{aligned} P_1(t) &= R + v_D (t - t') \vec{e}(\alpha) + (0, 0, -\frac{1}{2}g(t - t')^2) \\ &= (17800 + v_D \cos \alpha t, v_D \sin \alpha t, 1800 - \frac{1}{2}g(t - t')^2) \end{aligned} \quad (6-7)$$

$$R = P_1(t') = P_1(0) + v_D t' \vec{e}(\alpha) \quad (6-8)$$

其中， R 是前一阶段结束时的坐标，也是平抛阶段的开始坐标，表示烟幕干扰弹投放点。

③ $t \in [t'', t'' + 20]$ ：此阶段烟幕干扰弹完成爆炸，直到烟幕云团消散。设 B 表示该阶段开始时的坐标为烟幕干扰弹起爆时刻的坐标，起爆时刻为 t'' ，其数学表达式为：

$$\begin{aligned} B = P_1(t'') &= R + v_D (t'' - t') \vec{e}(\alpha) + (0, 0, -\frac{1}{2}g(t'' - t')^2) \\ &= (17800 + v_D \cos \alpha t'', v_D \sin \alpha t'', 1800 - \frac{1}{2}g(t'' - t')^2) \end{aligned} \quad (6-9)$$

烟幕云团的运动轨迹如下，其中 $v_s = 3 \text{ m/s}$ ：

$$\begin{aligned} C_S(t) &= B + (0, 0, -v_s(t - t'')) \\ &= (17800 + v_D \cos \alpha t'', v_D \sin \alpha t'', 1800 - \frac{1}{2}g(t'' - t')^2 - v_s(t - t'')) \end{aligned} \quad (6-10)$$

真目标为半径 $r = 7 \text{ m}$ 、高 $h = 10 \text{ m}$ 的圆柱，底面圆心 $(0, 200, 0)$ 。问题一中我们已经证明极限遮蔽情况下最后可见点位于上下底面边缘，表示为：

$$\begin{cases} \partial S_{\text{bottom}} = \{(r \cos \theta, 200 + r \sin \theta, 0) : \theta \in [0, 2\pi)\} \\ \partial S_{\text{top}} = \{(r \cos \theta, 200 + r \sin \theta, h) : \theta \in [0, 2\pi)\} \end{cases} \quad (6-11)$$

6.1.3 云团遮挡情况分析

在问题一中，由于烟幕云团和导弹均在 X-Z 平面上运动，因此不用考虑导弹是否穿过云团以及导弹与云团的复杂位置关系，在本问中我们将一一探索。我们对一个任意的时刻 t_0 ，首先需要判断导弹在该时刻前是否曾进入过云团，进而判断 t_0 时刻导弹与云团的位置关系，再决定选用什么样的判别条件来判别遮蔽情况。我们绘制出了分析过程的流程图，如下：

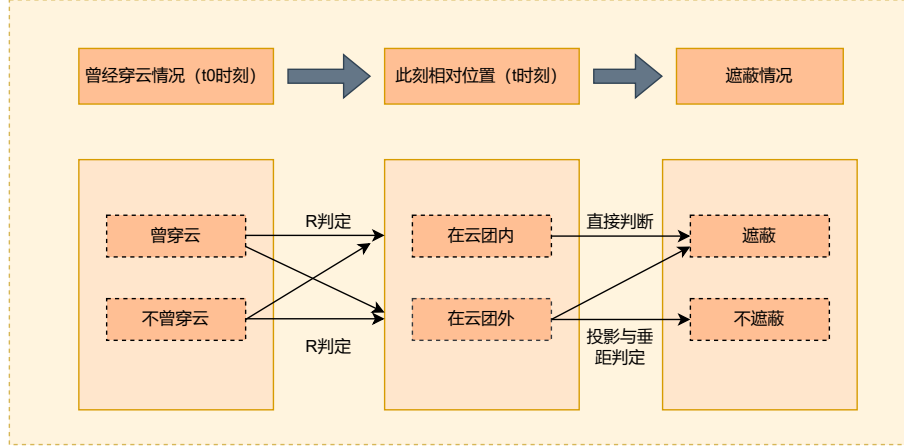


图 6-1 问题二遮蔽情况分析流程图

① 垂距判定法则：

垂距判定法则已经不能够满足一些特殊情况的判定，例如在导弹位于云团中心与目标点连线之间时，垂距可能大于 10 但此时不会被遮蔽。因此我们需要寻找更完备的判定法则，同时将垂距判定法作为辅助来判断遮蔽情况。垂距判定法的判定公式见问题一，在此不再赘述。

② 是否穿云判定：

为了更清晰描述云团对导弹的遮蔽情况，对某时刻 t ，我们需要判断导弹在该时刻前是否进入过云团。定义导弹与云团中心点的连线段长度为 $R(t) = |\overrightarrow{P_M(t)C_S(t)}|$ ，判断方式为 R 是否曾小于等于 10，为了清晰的表示，我们引入指示函数：

$$Y(t) = \begin{cases} 1 & \text{if } \exists t_0 < t, R(t_0) \leq 10 \\ 0 & \text{else} \end{cases} \quad (6-12)$$

如果曾经有过时刻 t_0 ，满足导弹与云团中心连线长度小于等于 10 的情况，证明曾经进入过云团或者与云团擦肩而过。如果曾经没有过 $R \leq 10$ 的情况，则导弹没有穿入过烟幕云团。

③ 投影判定法则：

对于某些特殊情况，单纯地依靠垂距判定可能会失效，这时需要引入更准确的方法-投影判定准则。如图，仍设 t 时刻烟幕云团中心为 $C_S(t)$ ，导弹所在点为 $P_M(t)$ ，圆柱体上下表面任意边缘点为 $P(t)$ ，导弹指向烟幕云团中心的向量为 $\overrightarrow{P_M C_S}$ ，导弹指向目标边缘点的向量为 $\overrightarrow{P_M P}$ ，根据几何原理，当烟幕云团中心点在三维空间中位于导弹与目标之间时，才有可能发生遮挡，转化为数学语言，我们利用向量 $\overrightarrow{P_M C_S}$ 在向量 $\overrightarrow{P_M P}$ 上的投影范围之间来判断。首先我们令向量投影为 σ_t ，定义为：

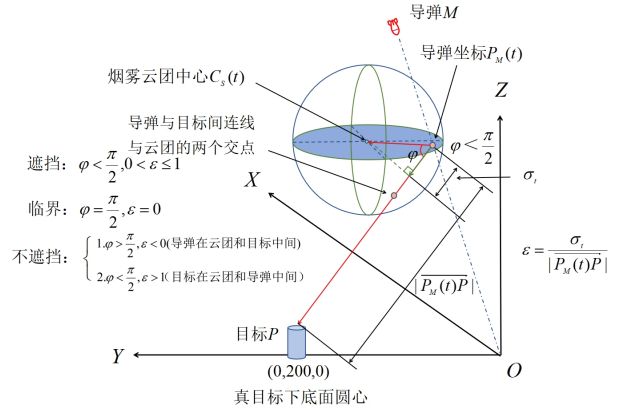


图 6-2 投影法判断示意图

$$\sigma_t = \frac{\overrightarrow{P_M C_S} \cdot \overrightarrow{P_M P}}{|\overrightarrow{P_M P}|} \quad (6-13)$$

接着，我们引入视线段归一化投影系数，定义为该向量投影 σ_t 与向量 $\overrightarrow{P_M P}$ 的模比值为：

$$\varepsilon(t) = \frac{\overrightarrow{P_M C_S} \cdot \overrightarrow{P_M P}}{|\overrightarrow{P_M P}|^2} \quad (6-14)$$

遮挡情况下该比值范围应限制在 $(0, 1]$ 范围内，如图所示，若 $\varepsilon \leq 0$ 或 $\varepsilon > 1$ 时，导弹与目标点位于云团同侧，此情况下真目标不会被遮挡；若 $\varepsilon = 0$ ，则两向量呈直角，此时两条线相切，处于临界点，此处规定临界点处于恰好看见状态；若 $\varepsilon \in [0, 1]$ ，则两向量呈锐角，此时导弹的遮挡情况需要结合垂距判定法来判断，若导弹到云团与目标点连线所在直线的垂距 $d_{(t,P)} < 10$ ，则导弹视线被遮挡，当导弹继续飞行到导弹与目标边界连线正好与烟幕云团相切时，导弹才不被遮挡；若垂距 $d_{(t,P)} \geq 10$ ，则导弹视线没有被遮挡。进而我们可以计算更加精确的有效遮蔽时长。

经过整理归类，我们绘制出以下包含十种情况的表格^[3]：

表 6-1 所有遮挡情况分类

情况	是否穿云	导弹与云心距离	投影系数	遮挡情况	情况说明
1	否	$R = 10$	$\varepsilon \in (0, 1]$	遮挡	导弹穿入云团或与其相切
2	否	$R = 10$	$\varepsilon \leq 0, \varepsilon > 1$	未遮挡	
3	否	$R > 10$	$\varepsilon \in (0, 1]$	遮挡 ($d \leq r$) 未遮挡 ($d > r$)	导弹未进入云团
4	否	$R > 10$	$\varepsilon \leq 0$	未遮挡	
5	否	$R < 10$	/	遮挡	受步长精度影响，恰进入云团
6	是	$R = 10$	$\varepsilon \leq 0, \varepsilon > 1$	未遮挡	导弹恰穿出云团
7	是	$R = 10$	$\varepsilon \in (0, 1]$	遮挡	
8	是	$R < 10$	/	遮挡	导弹在云团内
9	是	$R > 10$	$\varepsilon \in (0, 1]$	遮挡	导弹已穿出云团
10	是	$R > 10$	$\varepsilon \leq 0, \varepsilon > 1$	未遮挡	

为便于后续计算，我们依据表格中的判别规则建立二值指示函数，并表达成抽象函数 f 的形式：

$$Y'(t) = f(Y(t), R(t), \varepsilon(t), d(t)) \quad (6-15)$$

本文约定 1 为遮挡，0 为未遮挡，有：

$$Y'(t) = \begin{cases} 1 & \text{遮挡} \\ 0 & \text{未遮挡} \end{cases} \quad (6-16)$$

6.1.4 有效遮蔽时长计算

我们定义遮蔽时间区间集合，有：

$$\Omega = \{t | Y'(t) = 1\} \quad (6-17)$$

其中， $Y'(t)$ 表示在时刻 t 的遮蔽状态，当 $Y'(t) = 1$ 时，表示真目标被遮蔽。

通过上文中遮蔽情况连续性分析，与问题一类似，本小问遮蔽过程同样具有连续性。因而，我们可以计算遮蔽收尾时刻之差，得到更加精确的有效遮蔽时长。定义真目标开始被遮蔽的时刻为 T_0 ，遮蔽结束的时刻为 T_1 ，则有：

$$\Omega = [T_0, T_1] \quad (6-18)$$

故总遮蔽时长 $T_{\text{总}}$ 为：

$$T_{\text{总}} = T_1 - T_0 \quad (6-19)$$

6.1.5 目标函数

依题意，目标函数定义为最大化被遮蔽时长总和：

$$\max T_{\text{总}} \quad (6-20)$$

6.1.6 问题二模型总结

总遮蔽时长优化模型为：

$$\max T_{\text{总}}$$

$$\begin{aligned}
& \left. \begin{aligned}
& \text{决策变量: } \begin{cases} \alpha \in [0^\circ, 360^\circ) \\ v_D \in [70, 140] \\ 0 \leq t' \leq t'' \\ 0 \leq t \leq t'' + 20 \end{cases} \\
& \text{导弹轨迹: } \begin{cases} \vec{e}_1 = \frac{(20000, 0, 2000)}{|OM_1|} \\ M_1(t) = (20000, 0, 2000) - 300 \vec{e}_1 t \end{cases} \\
& \text{云团轨迹: } \begin{cases} P_1(t) = (17800 + v_D \cos \alpha t, v_D \sin \alpha t, 1800) & 0 \leq t < t' \\ P_1(t) = (17800 + v_D \cos \alpha t, v_D \sin \alpha t, 1800 - \frac{1}{2}g(t-t')^2) & t' \leq t \leq t'' \\ P_1(t) = (17800 + v_D \cos \alpha t'', v_D \sin \alpha t'', \\ \quad 1800 - \frac{1}{2}g(t''-t')^2 - v_s(t-t'')) & t'' \leq t \leq t'' + 20 \end{cases} \\
& \text{s.t. } \begin{cases} \partial S_{\text{bottom}} = \{(r \cos \theta, 200 + r \sin \theta, 0) : \theta \in [0, 2\pi)\} \\ \partial S_{\text{top}} = \{(r \cos \theta, 200 + r \sin \theta, h) : \theta \in [0, 2\pi)\} \\ d(t, P) = \frac{|\overrightarrow{P_M(t)\dot{P}} \times \overrightarrow{P_M(t)C_S(t)}|}{|\overrightarrow{P_M(t)\dot{P}}|} \\ \max_{P \in S_{\text{top}} \cup S_{\text{bottom}}} d(t, P) \leq 10, \quad t \in [t'', t'' + 20] \\ R(t) = |\overrightarrow{P_M(t)C_S(t)}| \\ Y(t) = \begin{cases} 1 & \text{if } \exists t_0 < t, R(t_0) \leq 10 \\ 0 & \text{else} \end{cases} \\ \varepsilon(t) = \frac{\overrightarrow{P_M C_S} \cdot \overrightarrow{P_M \dot{P}}}{|\overrightarrow{P_M \dot{P}}|^2} \\ Y'(t) = \begin{cases} 1 & \text{遮挡} \\ 0 & \text{未遮挡} \end{cases} = f(Y(t), R(t), \varepsilon(t), d(t)) \\ \text{有效遮蔽时长计算: } \begin{cases} \Omega = \{t | Y'(t) = 1\} \\ \Omega = [T_0, T_1] \\ T_{\text{总}} = T_1 - T_0 \end{cases}
\end{cases}
\end{aligned} \right\} \quad (6-21)
\end{aligned}$$

6.2 问题二的模型求解

基于题设，式6-21中模型仅对 t' 和 t'' 进行了时间序列上的先后约束，而并未设置可行的数值上界。为避免求解中在不可行域进行过多的无效搜索，本节基于题设背景和已知参数，对 t' 和 t'' 的数值上界进行合理地约束。

6.2.1 约束条件的修正

已知导弹 M1 从 (20000, 0, 2000) 以 300 m/s 飞向 (0, 0, 0), 则到达假目标所需时间为:

$$t_m = \frac{|(20000, 0, 2000)|}{300} \approx 66.7 \text{ s} \quad (6-22)$$

根据经验, 更优的投放策略通常满足烟幕云团在导弹击中前消散, 因此起爆时刻至少在导弹击中前 20s, 有以下约束:

$$t' \leq t'' \leq t_m - 20 = 46.7 \text{ s} \quad (6-23)$$

进一步, 烟幕干扰弹投放点至起爆点的垂直位移为 $\Delta z = \frac{1}{2}g(t'' - t')^2$ 。若时间差 $t'' - t'$ 过大, 将导致烟幕弹在地面起爆。我们取起爆最低高度为 0m, 可反解出起爆时间:

$$t'' - t' \leq \sqrt{\frac{2 \times 1800}{g}} \approx 19.2 \text{ s} \quad (6-24)$$

由此可知, 烟幕弹飞行时长最大不超过 19.2s, 水平速度范围 70 ~ 140, 因此烟幕弹最远只能飞到

$$L_{\max} = 140 \times 19.2 \approx 2688 \text{ m} \quad (6-25)$$

显然, 烟幕弹不可能真正飞到目标位置附近。因此若在地面起爆, 其对于高空导弹的遮蔽将完全失效。因此有:

$$t'' \leq t' + 19.2 \quad (6-26)$$

6.2.2 基于改进粒子群算法的模型求解

(一) 算法流程伪代码

该优化问题属于四维连续优化问题, 由于存在复杂的约束条件, 传统数值方法难以进行解析求解。鉴于此, 本文引入因此本文使用启发式算法——粒子群算法对其进行求解。具体算法流程如下所示:

算法 1: 基于改进粒子群算法的投放策略优化

输入: 最大迭代数 T_{max} , 群体规模 N , 决策变量边界 **Bounds**

输出: 最优投放策略向量 $\mathbf{G}_{best} = (\alpha^*, v_D^*, t'^*, t''^*)$

```
// 1. 初始化
1:  $\mathbf{Pop}^{(0)}, \mathbf{v}^{(0)} \leftarrow \text{InitializePopulation}(N, \mathbf{Bounds})$  // 随机初始化种群位置与速度
2:  $\mathbf{P}_{best} \leftarrow \mathbf{Pop}^{(0)}$  // 初始化个体历史最优位置
3:  $\mathbf{G}_{best} \leftarrow \arg \max_{\mathbf{x} \in \mathbf{Pop}^{(0)}} f(\mathbf{x})$  // 初始化全局历史最优位置 (最大遮蔽时长)
// 2. 迭代搜索
4: for  $t = 0 \rightarrow T_{max} - 1$  do // 开始迭代
5:   for  $i = 1 \rightarrow N$  do // 遍历每个粒子, 即一个投放策略方案  $\mathbf{x}_i = (\alpha_i, v_{D,i}, t'_i, t''_i)$ 
6:      $w(t) \leftarrow w_{max} - (w_{max} - w_{min}) \cdot (t/T_{max})$  // 采用线性递减惯性权重
7:      $r_1, r_2 \sim U(0, 1)$  // 生成 (0,1) 之间的随机数
8:      $\mathbf{v}_i^{(t+1)} \leftarrow w(t)\mathbf{v}_i^{(t)} + c_1 r_1 (\mathbf{P}_{best,i} - \mathbf{x}_i^{(t)}) + c_2 r_2 (\mathbf{G}_{best} - \mathbf{x}_i^{(t)})$ 
9:      $\mathbf{x}_i^{(t+1)} \leftarrow \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}$ 
10:     $\mathbf{x}_i^{(t+1)} \leftarrow \text{ConstraintHandle}(\mathbf{x}_i^{(t+1)})$  // 对不满足约束条件采用罚函数法
11:     $f_{new} \leftarrow \text{CalculateGradedFitness}(\mathbf{x}_i^{(t+1)})$  // 用梯度化适应度函数评估策略优劣
    // 若  $f_{new} > 0$ , 其值为实际遮蔽时长; 否则为引导性的“接近度”负分
12:    if  $f_{new} > f(\mathbf{P}_{best,i})$  then // 更新个体最优策略
13:       $\mathbf{P}_{best,i} \leftarrow \mathbf{x}_i^{(t+1)}$ 
14:      if  $f(\mathbf{P}_{best,i}) > f(\mathbf{G}_{best})$  then // 更新全局最优
15:         $\mathbf{G}_{best} \leftarrow \mathbf{P}_{best,i}$ 
16:      end if
17:    end if
18:  end for
19: end for
// 3. 返回结果
20: return  $\mathbf{G}_{best}$ 
```

(二) 算法改进策略

① 梯度化适应度函数设计

传统粒子群优化算法在处理具有稀疏可行域的复杂优化问题时, 常因目标函数缺乏梯度信息而陷入无效搜索。本文针对烟幕遮蔽时长优化这一特殊问题, 设计了分层梯度化适应度函数, 为粒子提供有效的搜索方向。

改进的梯度化适应度函数构造为:

$$f'(\mathbf{x}) = \begin{cases} T_{\text{总}}(\mathbf{x}) & \text{if } T_{\text{总}}(\mathbf{x}) > 0 \\ -1 + 0.95 \cdot S_{\text{prox}}(\mathbf{x}) & \text{if } T_{\text{总}}(\mathbf{x}) = 0 \end{cases}$$

其中接近度评价函数 $S_{\text{prox}}(\mathbf{x}) \in [0, 1]$ 定义为:

$$S_{\text{proximity}}(\mathbf{x}) = \max_{t \in \mathcal{T}} [w_1 S_d(t) + w_2 S_p(t) + w_3 S_t(t)]$$

式中: $\mathcal{T}$ 为关键时刻集合; $S_d(t)$ 为导弹-烟幕距离接近度; $S_p(t)$ 为导弹-目标连线与烟幕的几何关系评价; $S_t(t)$ 为时间窗口合理性评价; 权重系数 $w_1 = 0.4, w_2 = 0.4, w_3 = 0.2$; R 为烟幕有效半径。

② 约束处理与边界修复策略

考虑到烟幕遮蔽问题的物理约束特性，本文针对时间约束 $t' \leq t''$ 和时间差约束 $(t'' - t') \leq 19.2$ ，采用如下修复策略：

对于粒子 $\mathbf{x}_i = (\alpha_i, v_{D,i}, t'_i, t''_i)$ ，约束修复过程为：

Step1: 若 $t'_i > t''_i$ ，则交换 t'_i 与 t''_i ；

Step2: 若 $(t''_i - t'_i) > 19.2$ ，则调整 $t''_i = t'_i + 19.2$ ；

Step3: $t''_i = \min(\max(t''_i, t''_{\min}), t''_{\max})$ 。

该策略确保所有粒子始终位于可行域内，避免了罚函数法可能导致的搜索效率降低。

③ 超参数优化

本文对粒子群的超参数进行合理优化，以有效防止种群过早收敛至局部最优解。

惯性权重：采用线性递减策略以平衡全局探索与局部开发：

$$w(k) = w_{\max} - \frac{w_{\max} - w_{\min}}{k_{\max}} \cdot k$$

其中 $w_{\max} = 0.9$ ， $w_{\min} = 0.4$ ， k 为当前迭代次数， $k_{\max}$ 为最大迭代次数。

学习因子：针对烟幕遮蔽问题存在多个局部最优解的特点，采用提高认知因子、降低社会因子的策略：

$$c_1 = 2.8, \quad c_2 = 1.3$$

通过 $c_1 > c_2$ 的参数配置，鼓励种群在不同的参数空间区域并行探索，提高发现全局最优解的概率。

解的规模：经过多轮调试，本文设置粒子数为 200，迭代轮数为 100，以平衡搜索性能和计算成本。

6.3 结果的求解及合理性分析

综上所述，本文求解得到问题二模型的最优参数以及最大遮蔽时长，如下表所示：

表 6-2 最优参数与最大遮蔽时长

参数项	最优值	参数项	最优值
航向角 (α)	6.67°	无人机速度 (v_D)	137.53 m/s
投放时刻 (t')	0.066 s	起爆时刻 (t'')	0.772 s
投放点坐标 (m)	(17809.02, 1.06, 1800.00)	起爆点坐标 (m)	(17905.45, 12.34, 1797.56)
最大遮蔽时长	4.587695 s		

结果显示，无人机 FY1 的飞行方向为与 X 轴正方向逆时针成 6.67°，飞行速度为 **137.53m/s**，投放时刻为 0.066s，投放点坐标为 (17809.02, 1.06, 1800.00)，起爆时刻 0.772s，起爆点坐标为 (17905.45, 12.34, 1797.56)，总最大遮蔽时长为 **4.587695s**。

该结果具有一定合理性：为避免导弹接近目标而减小拦截时间，无人机在受领任务后极短时间内朝向导弹方向飞行，几乎同时投下烟幕干扰弹，且飞行速度非常接近题目允许的最大速度，能够为后续遮蔽真目标争取时间，符合客观事实，说明本模型具有客观实际性。

七、问题三的模型的建立和求解

问题三在问题二的基础上，将单枚干扰弹的优化问题扩展为3枚干扰弹的协同优化问题。这三枚烟幕弹不再是独立优化的，它们的投放时机、引爆时机、以及形成的烟幕云团在时空上的关系，共同决定了最终的总遮蔽效果。下面我们基于问题二的模型，改进模型使其适用于问题三场景。

7.1 单无人机多烟幕弹的动态遮蔽时长优化建模

7.1.1 多云团遮挡判定模型

与单个云团的判定情况不同，对于空间中同一时间可能存在多个云团的场景。由于实际的遮挡效果可能是三个云团共同实现的，如图7-1所示。因此，仅对单一云团进行遮挡判定是不完善的，问题二中建立的遮挡判定模型不再完全适用。此时我们将进行遮挡判定的对象由云团转变为目标圆柱体。

由图可知，目标点选取为圆柱上下底面边缘情况不再适用，为此我们建立了新的目标取点方式：圆柱体的上表面、侧面上的所有点，以及下表面的所有边缘点构成取点集合 D 。其原因是下底面与地面接触，不会被导弹探测到。

接着，我们制定了新的遮挡判定规则：对于 D 上的任意一点，需要对当前时刻存在的所有云团逐个进行问题二中的遮挡判定步骤。只要存在一个云团，例如无人机 j 释放的第 i 个烟幕干扰弹，能够实现对该点的遮挡，则该情况就被判定为成功遮挡。只有当圆柱表面上采样的所有点均满足遮挡判定时，才能确定该时刻下整个目标圆柱被判定为遮挡，采取判定公式如下：

$$Y_{j,i}^p(t) = \begin{cases} 1 & \text{if } t \text{ 时刻该点 } P \text{ 被无人机 } j \text{ 投出的云团 } i \text{ 遮蔽} \\ 0 & \text{else} \end{cases} \quad (7-1)$$

$$Y^p(t) = \begin{cases} 1 & \text{if } \exists(i, j), Y_{j,i}^p(t) = 1 \\ 0 & \text{else} \end{cases} \quad (7-2)$$

对目标圆柱体上任意一点 P ，我们分析该点在整个过程中的被遮挡情况：

首先需要判断导弹与云团的位置关系：当 $R(t) < 10$ 时，导弹在云团内部，此时 P 点一定被遮蔽；当 $R(t) \geq 10$ 时，导弹在云团边界上或云团外部，此时需要进一步判断导弹到云团中心与 P 点连线所在直线的距离以及投影系数的范围。同问题二，我们将所有6种情况绘制成表7-1进行展示：

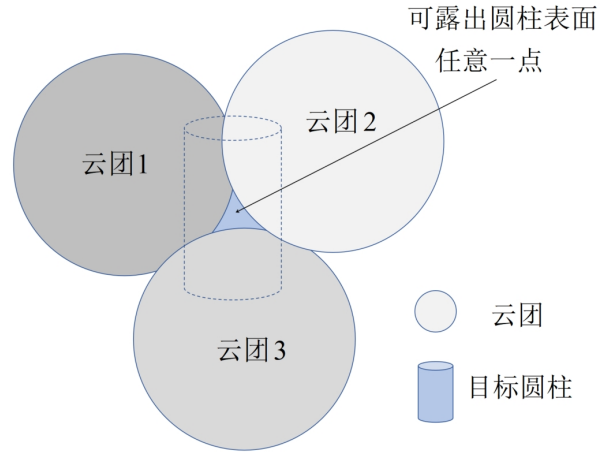


图 7-1 协同遮挡示意图

表 7-1 多云团遮挡情况分类

情况	导弹与云心距离	投影系数	垂距范围	遮挡情况
1	$R < 10$	/	/	遮挡
2	$R = 10$	$\varepsilon \leq 0, \varepsilon > 1$	/	不遮挡
3		$\varepsilon \in (0, 1]$		遮挡
4	$R > 10$	$\varepsilon \in (0, 1]$	$d \geq 10$	不遮挡
5			$d < 10$	遮挡
6		$\varepsilon \leq 0, \varepsilon > 1$	/	不遮挡

与问题二相似，我们结合表格判定准则，把式 (7-1) 表达成抽象函数形式：

$$Y_{j,i}^p(t) = \begin{cases} 1 & \text{遮挡} \\ 0 & \text{未遮挡} \end{cases} = g(R_i(t), \varepsilon_i(t), d_i(t, P)) \quad (7-3)$$

至此，我们可以求得目标圆柱体被完全遮蔽的总时长，即满足所有目标点 $p \in D$ 同时被遮蔽的时间 t 集合，数学表达式为：

$$\Omega = \{t | \forall p \in D, Y^p = 1\} \quad (7-4)$$

7.1.2 决策变量约束

问题三的烟幕干扰弹数量增加到 3 个，整个策略由以下 8 个决策变量共同决定：无人机飞行速度 v_D ，航向角 α ，三枚烟幕干扰弹各自投放与引爆时刻： t'_i, t''_i ($i = 1, 2, 3$)，且满足 $t''_i - t'_i > 0$ ，前两个决策变量约束条件不变。

$$t''_i - t'_i > 0, \quad i = 1, 2, 3 \quad (7-5)$$

题中指出每一架无人机投放烟幕弹的时间间隔至少为 1s，有：

$$t'_2 - t'_1 \geq 1 \quad (7-6)$$

$$t'_3 - t'_2 \geq 1 \quad (7-7)$$

7.1.3 状态方程

状态方程描述了场景中各个物体随时间 t 的位置，用坐标 $P(t)$ 表示，下文我们将在问题二基础上改进模型，求得导弹、烟幕弹及烟幕云团的新的轨迹方程。导弹轨迹方程不变，故此处不再赘述。仅叙述烟幕弹系统轨迹方程：

记无人机 FY1 的坐标为 $P_1(t)$ ，该无人机从初始位置 $P_1(0) = (17800, 0, 1800)$ 以速度 v_D 和航向 α 做匀速直线运动。其单位航向向量为 $\vec{e}(\alpha) = (\cos \alpha, \sin \alpha, 0)$ ，则无人机轨迹为：

$$P_1(t) = P_1(0) + v_D t \vec{e}(\alpha) = (17800 + \cos \alpha v_D t, \sin \alpha v_D t, 1800) \quad (7-8)$$

我们将载弹飞行阶段的无人机、平抛阶段的烟幕干扰弹和匀速下落阶段的烟幕云团看作一个烟幕弹系统，分阶段研究该系统轨迹路线。设第 i 枚烟幕干扰弹系统中心的位置

置为 $P_{S,i}(t) (i = 1, 2, 3)$, 其运动轨迹分三个阶段:

$$\begin{aligned}
 P_{S,i}(t) &= \begin{cases} P_1(t) & 0 \leq t \leq t'_i \\ P_1(t'_i) + v_D(t - t'_i)\vec{e}(\alpha) - (0, 0, \frac{1}{2}g(t - t'_i)^2) & t'_i < t \leq t''_i \\ C_{S,i}(t) & t''_i < t \leq t''_i + 20 \end{cases} \\
 &= \begin{cases} (17800 + \cos \alpha v_D t, \sin \alpha v_D t, 1800) & 0 \leq t \leq t'_i \\ (17800 + \cos \alpha v_D t, \sin \alpha v_D t, 1800 + \frac{1}{2}g(t - t'_i)^2) & t'_i < t \leq t''_i \\ (17800 + v_D \cos \alpha t''_i, v_D \sin \alpha t''_i, 1800 - \frac{1}{2}g(t''_i - t'_i)^2 - v_s(t - t''_i)) & t''_i < t \leq t''_i + 20 \end{cases} \quad (7-9)
 \end{aligned}$$

其中, $C_{S,i}(t)$ 是第 i 个烟幕云团中心的位置, 由引爆点位置减去下沉位移得到。

至此, 我们在问题二基础上得到了一个无人机可用, 可投放多枚烟幕弹情境下各主体的轨迹方程, 为后续优化提供依据。

7.1.4 目标函数

我们的目标是使总有效遮蔽时间最长。本问及后续小问中, 遮蔽时间不再满足连续性。因此总遮蔽时间不再表现为连续区间, 而是一个离散或不连续的时间集合。基于此, 则目标函数可表示为时间集合的测度最大化, 又因为上文得总有效遮蔽时间 $\Omega = \{t | \forall p \in D, Y^p = 1\}$, 可得目标函数^[4]:

$$\max T_{\text{总}} = m(\Omega) \quad (7-10)$$

7.1.5 问题三模型汇总

综上, 问题三模型在问题二基础上略作修改, 并作如下补充:

$$\begin{aligned}
 &\max T_{\text{总}} = m(\Omega) \\
 \text{s.t.} \quad &\left\{ \begin{array}{l} \text{补充约束条件: } \begin{cases} 0 \leq t'_i \leq t''_i, \quad i = 1, 2, 3, \\ t'_2 - t'_1 \geq 1, \quad t'_3 - t'_2 \geq 1 \end{cases} \\ \\ \text{多云团有效遮蔽: } \begin{cases} d_i(t, P) = \frac{|\overrightarrow{P_M(t)}\vec{P} \times \overrightarrow{P_M(t)}C_{S,i}(t)|}{|\overrightarrow{P_M(t)}\vec{P}|} \leq 10, \quad t \in [t''_i, t''_i + 20] \\ R_i(t) = \frac{|\overrightarrow{P_M(t)}C_{S,i}(t)|}{|\overrightarrow{P_M(t)}\vec{P}|} \\ \varepsilon_i(t) = \frac{|\overrightarrow{P_M(t)}C_{S,i}(t) \cdot \overrightarrow{P_M(t)}\vec{P}|}{|\overrightarrow{P_M(t)}\vec{P}|^2} \\ Y_{j,i}^p(t) = \begin{cases} 1 & \text{遮挡} \\ 0 & \text{未遮挡} \end{cases} = g(R_i(t), \varepsilon_i(t), d_i(t, P)) \\ Y^p(t) = \begin{cases} 1 & \text{if } \exists (i, j), Y_{j,i}^p(t) = 1, j = 1, i = 1, 2, 3 \\ 0 & \text{else} \end{cases} \end{cases} \\ \\ \text{遮蔽时间: } \begin{cases} \Omega = \{t | \forall p \in D, Y^p = 1\} \\ T_{\text{总}} = m(\Omega) \end{cases} \end{array} \right.
 \end{aligned}$$

7.2 问题三模型求解

7.2.1 算法设计

由于问题三存在多云遮挡情况，本文在问题二粒子群算法基础上进行了如下优化：
1. 采样方式：考虑到圆柱上除了底面外的任一点都可能暴露，因此本文对圆柱侧面进行 $\theta_{step} = 24$ 、 $z_{step} = 10$ 、对圆柱上圆面进行 $\theta_{step} = 24$ 、 $r_{step} = 4$ 的定步长采样，共计 $240 + 73 = 313$ 个采样点

2. 向量化运算：问题三搜索空间相比问题二更加复杂，为提高运算效率，本文采用向量化计算策略对核心函数进行优化。通过将原有的嵌套循环结构转换为矩阵运算，将时间复杂度从 $O(N)$ 降至 $O(1)$ 。

3. 变步长搜索：为进一步降低搜索的时间复杂度，本文设计了变步长计算策略，在保证计算结果精度的前提下，避免了计算测度的高昂时间成本。具体步骤如下：

Step1：设定时间步长 $\varepsilon = 0.05s$ ，将连续时间域 $[t_{start}, t_{end}]$ 离散化为检查点序列 $\{t_i = t_{start} + i \cdot \varepsilon\}_{i=0}^N$ ，其中 $N = \lfloor (t_{end} - t_{start})/\varepsilon \rfloor$ 。对每个检查点 t_i ，计算其遮蔽状态 $S(t_i) \in \{0, 1\}$ ，生成遮蔽状态掩码序列 $\mathbf{M} = [S(t_1), S(t_2), \dots, S(t_N)]$ 。

Step2：基于粗搜结果识别遮蔽状态转换点，设 $\{j_k\}$ 为状态跳变的索引集合，即 $S(t_{j_k}) \neq S(t_{j_k+1})$ 。对每个跳变区间 $[t_{j_k}, t_{j_k+1}]$ ，采用二分法求解精确边界。

7.2.2 求解结果

问题三基于上述算法求解得到如下结果：

表 7-2 无人机投放烟幕干扰弹策略

编号	无人机运动方向	速度 (m/s)	投放点坐标	起爆点坐标	有效干扰时长 (s)
1	8.32°	110.4204	(17800,0,1800)	(17800.13,0.02,1800)	2.5625
2	8.32°	110.4204	(17909.26,15.98,1800)	(17909.39,15.99,1800)	3.9234
3	8.32°	110.4204	(18076.66,40.45,1800)	(18211.95,60.23,1792.48)	0

7.3 问题三的求解结果分析

将上述结果可视化后得到：

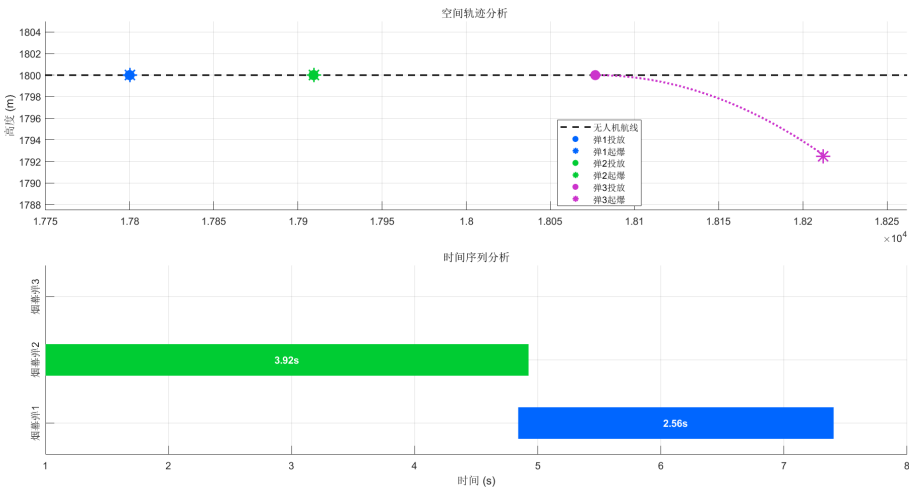


图 7-2 烟幕弹投放策略时空分析

求解结果显示, 烟幕干扰弹 1 的有效干扰时长为 2.5625s, 烟幕干扰弹 2 的有效干扰时长为 3.9234s, 两烟幕弹的投放与起爆几乎同时发生; 而烟幕干扰弹 3 的起爆与投放之间相隔很长时间, 有效干扰时长为 0。两时间段具有重叠, 总有效干扰时长为 6.4031s。

7.4 误差分析

本问中误差主要源于圆柱表面离散采样与数值积分步长: 我们选取的 313 个点虽覆盖上下底及侧面, 但仍可能漏检微小裸露弧段, 从而导致遮蔽时长被高估; 变步长二分搜索以 0.05 s 为初值, 对云团边界切换时刻存在 $\pm 0.01s$ 截断误差, 在 6.4 s 总时长中相对影响约 0.3%。

八、问题四的模型的建立和求解

问题四在问题二的基础上, 将 1 架无人机优化问题拓展为 3 架无人机的协同优化问题。我们在问题二、三模型基础上, 改进模型使其适用于问题四场景。

8.1 多无人机单烟幕弹的动态遮蔽时长优化建模

8.1.1 决策变量约束

问题四的无人机数量增加到 3 架, 我们将其编号记为 j ($j = 1, 2, 3$)。本问共有 12 个决策变量: 无人机 j 飞行速度 $v_{D,j}$, 航向角 α_j , 无人机 j 投下的烟幕干扰弹 i ($i = 1$) 各自投放与引爆时刻: $t'_{j,1}$, $t''_{j,1}$, 于是有约束条件:

$$t'_{j,1} \leq t''_{j,1}, \quad j = 1, 2, 3 \quad (8-1)$$

8.1.2 状态方程

导弹轨迹同前三问, 不再重复赘述; 对于云团轨迹, 加入无人机编号 j 后, 我们更新如下:

$$P_j(t) = \begin{cases} P_j(0) + v_D t \vec{e}(\alpha_j) & 0 \leq t \leq t'_{j,1} \\ P_j(t'_{j,1}) + v_D(t - t'_{j,1})\vec{e}(\alpha) - (0, 0, \frac{1}{2}g(t - t'_{j,1})^2) & t'_{j,1} < t \leq t''_{j,1} \\ P_j(t''_{j,1}) - (0, 0, v_s(t - t''_{j,1})) & t''_{j,1} < t \leq t''_{j,1} + 20 \end{cases}$$

$$= \begin{cases} (P_j(0)_x + v_{D,j} \cos \alpha_j t, P_j(0)_y + v_{D,j} \sin \alpha_j t, P_j(0)_z) \\ (P_j(0)_x + v_{D,j} \cos \alpha_j t, P_j(0)_y + v_{D,j} \sin \alpha_j t, P_j(0)_z - \frac{1}{2}g(t - t'_{j,1})^2) \\ (P_j(0)_x + v_{D,j} \cos \alpha_j t'', P_j(0)_y + v_{D,j} \sin \alpha_j t'', P_j(0)_z - \frac{1}{2}g(t''_{j,1} - t'_{j,1})^2 - v_s(t - t''_{j,1})) \end{cases} \quad (8-2)$$

8.1.3 有效遮蔽时间

有效遮蔽判定模型类同问题三, 此处不再赘述。对于有效遮蔽时间模型, 我们修正 i, j 下标范围, 如下:

$$Y^p(t) = \begin{cases} 1 & \text{if } \exists(i, j), Y_{j,i}^p(t) = 1 \quad i = 1, j = 1, 2, 3 \\ 0 & \text{else} \end{cases} \quad (8-3)$$

最终, 求遮蔽时间范围公式如下:

$$\begin{cases} \Omega = \{t | \forall p \in D, Y^p = 1\} \\ T_{\text{总}} = m(\Omega) \end{cases} \quad (8-4)$$

8.2 问题四的模型求解与结果分析

相较于前述问题, 问题四决策变量的维度增大至 12 维, 导致解空间稀疏。先前所采用的粒子群算法极易过早地收敛到某个局部最优解, 难以进行充分的全局探索。因此, 本文选用搜索能力更强的混合进化算法。通过引入多个子种群并行搜索、融合差分进化与遗传算法的寻优机制, 能够更好地进行全局探索。具体的算法流程设计如下:

算法 2: 基于混合进化算法的三架无人机协同投放策略优化

输入: 最大代数 G_{max} , 种群规模 N , 子种群数 K

输出: 最优协同策略 $\mathbf{G}_{best} = (\alpha_1^*, v_{D,1}^*, t_1'^*, t_1''^*, \alpha_2^*, v_{D,2}^*, t_2'^*, t_2''^*, \alpha_3^*, v_{D,3}^*, t_3'^*, t_3''^*)$

// 1. 针对 FY1、FY2、FY3 的协同烟幕投放策略初始化

1: 采用多种策略初始化 K 个子种群, 模拟不同无人机编队的投放方案组合

2: $\mathbf{G}_{best} \leftarrow \arg \max_{\mathbf{x}} f(\mathbf{x})$ // 寻找对导弹 M1 遮蔽效果最佳的策略

// 2. 进化优化寻找最佳烟幕干扰弹投放时机和位置

3: **for** $g = 0 \rightarrow G_{max} - 1$ **do** // 迭代寻优过程, 逐步改善三机协同遮蔽效果

4: **for** $k = 1 \rightarrow K$ **do** // 对每个无人机编队方案进行优化

5: **if** $\text{rand}() < P_{DE}(g)$ **then** // 根据当前优化效果选择进化策略

6: $\text{Pop}_k^{(g+1)} \leftarrow$ 差分进化优化各机投放参数和起爆时机

7: **else**

8: $\text{Pop}_k^{(g+1)} \leftarrow$ 基于锦标赛选择优化无人机飞行轨迹和投弹策略

9: **end if**

10: 保留遮蔽效果最佳方案, 局部微调投放位置提升烟幕云团覆盖效果

11: **end for**

12: 子编队间交换优秀投放策略, 更新全局最优三机协同遮蔽方案

13: **end for**

14: **return** $\mathbf{G}_{best}$ // 输出最优的三架无人机烟幕干扰弹协同投放策略

基于上述算法求解得到 3 架无人机投放烟幕弹策略如下:

表 8-1 三无人机协同投放烟幕干扰弹最优策略

无人机	运动方向 / °	速度	投放点坐标 / m	起爆点坐标 / m	有效干扰时长 / s
FY1	9.05	70.87	(17800, 0, 1800)	(17874.41, 11.85, 1794.46)	4.5817
FY2	258.16	104.23	(11854.74, 706.9, 1800)	(11714.71, 258.16, 1189.64)	3.7094
FY3	99.17	126.37	(5604.32, 5499.83, 1800)	(548.76, 700, 567.98)	2.8429

结果显示, 三架无人机的运动方向分别为 9.05° , 258.16° , 99.17° , 运动速度分别为 70.87m/s , 104.23m/s , 126.37m/s , 总最大有效干扰时长为 **11.134009 秒**。

我们对上表方案中详细时空情况可视化，如下：

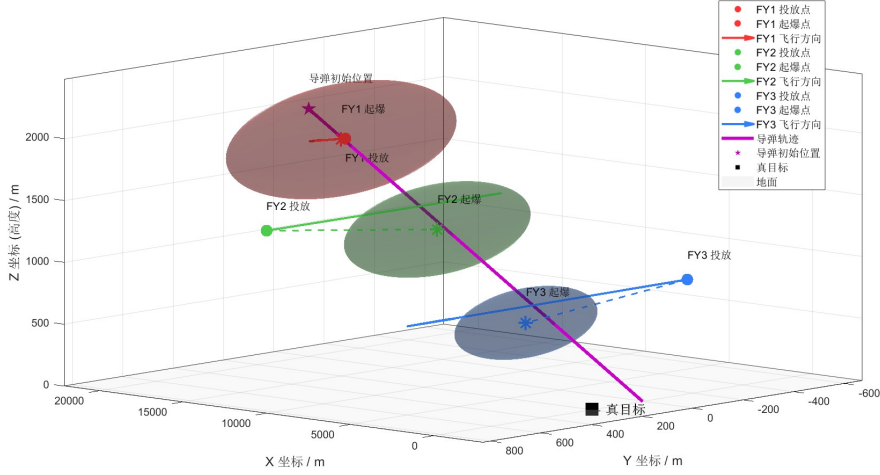


图 8-1 无人机协同干扰与导弹轨迹时空可视化

从图中可以看出，三架无人机接到指令时，其大致轨迹均朝着导弹轨迹靠近，即无人机提前到达与导弹可能相交的轨迹点，投放烟幕干扰弹，使得总遮蔽时长最大，符合实际场景，说明该结果具有一定合理性。

九、问题五的模型的建立和求解

问题五在前四小问的基础上，情景全面升级为 5 架无人机，各投放 3 枚烟幕干扰弹，来完成对 3 枚导弹的干扰遮蔽的复杂优化模型。

9.1 多机-多弹-多导场景下的动态遮蔽时长优化建模

9.1.1 决策变量约束

对编号进行整理，无人机编号为 j ($j = 1, 2, 3, 4, 5$)，其各自携带导弹编号为 i ($i = 1, 2, 3$)，遮蔽无人机编号 k , ($k = 1, 2, 3$)，共有 40 个决策变量。我们逐一归纳约束条件如下：

$$\begin{cases} \alpha_j \in [0^\circ, 360^\circ) \\ v_{D,j} \in [70, 140] \\ 0 \leq t'_{j,i} \leq t''_{j,i}, & j = 1, \dots, 5, i = 1, 2, 3. \\ t'_{j,i+1} - t'_{j,i} \geq 1 \text{ s}, & j = 1, \dots, 5, i = 1, 2. \end{cases} \quad (9-1)$$

9.1.2 导弹轨迹

记导弹 k ($k = 1, 2, 3$) 在 t 时刻的位置为 $P_{M,k}(t)$ ，将导弹轨迹模型拓展至一般情况，有：

$$\vec{e}_k = \frac{\overrightarrow{OP_{M,k}(0)}}{\left| \overrightarrow{OP_{M,k}(0)} \right|} \quad (9-2)$$

$$P_{M,k}(t) = P_{M,k}(0) - 300 \vec{e}_k t \quad (9-3)$$

9.1.3 烟幕弹系统轨迹

类比问题三，无人机 j ($j = 1, \dots, 5$) 抛下的烟幕弹 i ($i = 1, 2, 3$) 的轨迹如下所示：

$$\begin{aligned}
 P_{j,i}(t) &= \begin{cases} P_j(0) + v_D t \vec{e}(\alpha_j) & 0 \leq t \leq t'_{j,i} \\ P_j(t'_{j,i}) + v_D(t - t'_{j,i}) \vec{e}(\alpha_j) - (0, 0, \frac{1}{2}g(t - t'_{j,i})^2) & t'_{j,i} < t \leq t''_{j,i} \\ P_j(t''_{j,i}) - (0, 0, v_s(t - t''_{j,i})) & t''_{j,i} < t \leq t''_{j,i} + 20 \end{cases} \\
 &= \begin{cases} (P_j(0)_x + v_{D,j} \cos \alpha_j t, P_j(0)_y + v_{D,j} \sin \alpha_j t, P_j(0)_z) \\ (P_j(0)_x + v_{D,j} \cos \alpha_j t, P_j(0)_y + v_{D,j} \sin \alpha_j t, P_j(0)_z - \frac{1}{2}g(t - t'_{j,i})^2) \\ (P_j(0)_x + v_{D,j} \cos \alpha_j t'', P_j(0)_y + v_{D,j} \sin \alpha_j t'', P_j(0)_z - \frac{1}{2}g(t''_{j,i} - t'_{j,i})^2 - v_s(t - t''_{j,i})) \end{cases} \\
 &\quad (9-4)
 \end{aligned}$$

9.1.4 有效遮蔽分析

问题五中有多枚导弹，可能会出现多个云团对其中某些导弹形成有效遮蔽，而对另一些导弹不形成有效遮蔽的情况。根据题意，只有当三枚导弹看向目标圆柱的视线全部被遮挡时才能算有效遮蔽。由问题四，单枚导弹 k 有效遮蔽时长为 $m(\Omega_k)$ ，其中 $\Omega_k = \{t | \forall p \in D, Y_k^p = 1\}$ 。故，本问中有效遮蔽时长为三枚导弹各自遮蔽时间集合的交集的测度，记作：

$$T_{\text{总}} = m\left(\bigcap_{k=1}^3 \Omega_k\right) = m\left(\bigcap_{k=1}^3 \{t | \forall p \in D, Y_k^p = 1\}\right) \quad (9-5)$$

9.1.5 目标函数

本小问的目标为最大化真目标在未被任意一枚导弹观测到的总时间长度，即三枚导弹同时无法观测到目标的时间区间的测度：

$$\max T_{\text{总}} = m\left(\bigcap_{k=1}^3 \Omega_k\right) \quad (9-6)$$

9.1.6 最终模型汇总

$$\begin{aligned}
 \max \quad & T_{\text{总}} = m\left(\bigcap_{k=1}^3 \Omega_k\right) \\
 \text{s.t.} \quad & \left\{ \begin{array}{l}
 \text{决策变量约束:} \begin{cases} \alpha_j \in [0^\circ, 360^\circ] \\ v_{D,j} \in [70, 140] \\ 0 \leq t'_{j,i} \leq t''_{j,i}, \quad j = 1 \dots, 5, \quad i = 1, 2, 3 \\ t'_{j,i+1} - t'_{j,i} \geq 1 \text{ s} \quad j = 1, \dots, 5, \quad i = 1, 2 \end{cases} \\
 \text{有效遮蔽分析:} \begin{cases} d_{j,i,k}(t, P) = \frac{|\overrightarrow{P_{M,k}(t)\dot{P}} \times \overrightarrow{P_{M,k}(t)C_{S,j,i}(t)}}{|\overrightarrow{P_{M,k}(t)\dot{P}}|} \leq 10, \quad t \in [t''_{j,i}, t''_{j,i} + 20] \\ R_{j,i,k}(t) = |\overrightarrow{P_{M,k}(t)C_{S,j,i}(t)}}| \\ \varepsilon_{j,i,k}(t) = \frac{\overrightarrow{P_{M,k}(t)C_{S,j,i}(t)} \cdot \overrightarrow{P_{M,k}(t)\dot{P}}}{|\overrightarrow{P_{M,k}(t)\dot{P}}|^2} \\ Y_{j,i,k}^p(t) = \begin{cases} 1 & \text{遮挡} \\ 0 & \text{未遮挡} \end{cases} = g(R_{j,i,k}(t), \varepsilon_{j,i,k}(t), d_{j,i,k}(t, P)) \\ Y_k^p(t) = \begin{cases} 1 & \text{if } \exists(i, j), Y_{j,i,k}^p(t) = 1 \\ 0 & \text{else} \end{cases} \end{cases} \\
 \text{最终有效遮蔽时间:} \begin{cases} \Omega_k = \{t | \forall p \in D, Y_k^p = 1\} \\ T_{\text{总}} = m\left(\bigcap_{k=1}^3 \Omega_k\right) \end{cases}
 \end{array} \right.
 \end{aligned}$$

9.2 问题五的模型求解

问题五在问题四设计算法的基础上进一步调整超参数，使解的规模足够大。算法思路与问题四一致，不再赘述。

9.3 问题五的求解结果

本文求得总遮蔽时长为 20.31s，详细求解结果如下：

表 9-1 多机多导场景下烟幕投放起爆策略与有效遮蔽时长

无人机 编号	无人机 运动方向 (°)	无人机运动 速度 (m/s)	烟幕干扰弹 编号	烟幕干扰弹 投放点坐标 (m)	烟幕干扰弹 起爆点坐标 (m)	有效干扰 时长 (s)	干扰的 导弹编号
FY1	179.25	139.41	1	(17745.1, 1.6, 1798.0)	(17180.5, 3.1, 1717.9)	2.31	M1
FY1	179.25	139.41	2	(17326.5, 1.9, 1795.0)	(16584.2, 6.1, 1662.5)	2.50	M1
FY1	179.25	139.41	3	(17043.2, 3.4, 1797.0)	(16198.9, 8.8, 1618.5)	2.65	M1
FY2	242.5	131.5	1	(11777.0, 986.0, 1398.0)	(11470.0, 410.0, 1284.0)	1.61	M2
FY2	242.5	131.5	2	(11710.0, 865.0, 1396.0)	(11443.0, 377.0, 1305.0)	1.67	M2
FY2	242.5	131.5	3	(11530.0, 520.0, 1395.0)	(11043.0, -388.0, 1097.0)	2.08	M2
FY3	111.6	136.5	1	(5895.0, -2801.0, 697.0)	(5501.0, -2399.0, 501.0)	1.65	M3
FY3	111.6	136.5	2	(5701.0, -2600.0, 699.0)	(5299.0, -2201.0, 479.0)	1.48	M3
FY3	111.6	136.5	3	(5499.0, -2395.0, 698.0)	(5101.0, -1999.0, 461.0)	2.06	M3
FY4	253.9	138.1	1	(10746.0, 1138.0, 1798.0)	(10607.0, 678.0, 1737.0)	1.05	M2
FY4	253.9	138.1	2	(10673.0, 899.0, 1795.0)	(10251.0, 541.0, 1230.0)	1.30	M2
FY4	253.9	138.1	3	(10560.0, 507.0, 1799.0)	(10130.0, 450.0, 1203.0)	1.45	M2
FY5	104.1	135.80	1	(14835.5, -868.1, 1297.0)	(14616.9, -3.1, 1290.5)	1.65	M1
FY5	104.1	135.80	2	(14785.2, -660.5, 1298.0)	(14610.1, 8.9, 1295.2)	1.53	M1
FY5	104.1	135.80	3	(14739.1, -489.0, 1295.0)	(14566.8, 199.1, 1288.7)	1.32	M1

十、模型的评价与推广

10.1 模型的优点

- 1. 模型使用了大量数学物理知识，从质点运动轨迹到圆柱体几何投影，再到云团球体遮挡情况的建模，层层递进，严格推导，既保证物理可信性，又兼顾计算可行性。
- 2. 模型效率较高，针对低维混合整数连续变量空间，运用变步长二分法、向量化矩阵运算与改进粒子群相结合的多级算法，在 10^{-8} s 精度下仍保持分钟级求解速度。
- 3. 模型考虑范围广，思考情况较为完善，如单云遮挡和多云团遮挡的判定准则几乎囊括了所有可能情况。

10.2 模型的缺点

- 1. 模型假设无风场干扰，但实际大气存在湍流与涡动，这会导致有效遮蔽时长被高估，降低模型适用性。
- 2. 当无人机和烟幕弹数量较大时，决策变量维度较高，粒子群算法早熟收敛概率显著上升，可能陷入局部最优解。

10.3 模型的推广

- 1. 可以考虑更加复杂的战场情况，如增加风场、磁场等物理因素，使模型更符合真实场景。
- 2. 可以改进现有算法，使得在面对更高维数的变量时保持求解速度。

参考文献

- [1] 夏康杰. 曲线流在平面曲线的几个不等式中的应用 [D]. 温州大学, 2023.10.27781/d.cnki.gwzdx.2023.000168.
- [2] 豆包, 网页版, 字节跳动 (ByteDance) , 2025-09-06.
- [3] Kimi, K1.5, Moonshot AI, 2025-09-06.
- [4] DeepSeek, DeepSeek-V3.1, 深度求索 (DeepSeek) , 2025-09-07.

附录 A 文件列表

文件名	功能描述
AI 工具使用详情.pdf	数学建模 AI 使用情况
q1.py	问题一程序代码
q2.py	问题二程序代码
q3.py	问题三程序代码
q4.py	问题四程序代码
q5.py	问题五程序代码
result1.xlsx	问题三求解结果
result2.xlsx	问题四求解结果
result3.xlsx	问题五求解结果

附录 B 代码

q1.py

```
1 import numpy as np
2 import pandas as pd
3
4 # 物理参数
5 g = 9.80665
6 missile_speed = 300
7 drone_speed = 120
8 smoke_fall_speed = 3
9 effective_radius = 10
10 effective_duration = 20
11
12 # 时间参数
13 t_drop = 1.5
14 interval = 3.6
15 t_exp = t_drop + interval
16
17 # 初始位置
18 P_M1_0 = np.array([20000, 0, 2000])
19 P_FY1_0 = np.array([17800, 0, 1800])
20 P_target = np.array([0, 0, 0])
21
22 # 真目标圆柱体参数
23 C_base = np.array([0, 200, 0])
24 C_top = np.array([0, 200, 10])
25 R_cyl = 7
26 H_cyl = 10
```

```

27
28 # 导弹运动参数
29 d_M1 = P_target - P_M1_0
30 distance_M1 = np.linalg.norm(d_M1)
31 u_M1 = d_M1 / distance_M1
32 v_M1 = missile_speed * u_M1
33
34 print(f"导弹M1运动参数: ")
35 print(f"  初始位置: {P_M1_0}")
36 print(f"  飞行方向: {u_M1}")
37 print(f"  速度向量: {v_M1}")
38 print(f"  x方向速度分量: {v_M1[0]:.2f} m/s")
39 print(f"  z方向速度分量: {v_M1[2]:.2f} m/s")
40 print(f"  到假目标距离: {distance_M1:.2f} m")
41 print(f"  到达假目标时间: {distance_M1/missile_speed:.2f} s\n"
    )
42
43 # 无人机运动参数 - 朝原点方向水平飞行
44 direction_to_target_horizontal = np.array([P_target[0] -
    P_FY1_0[0], P_target[1] - P_FY1_0[1], 0])
45 u_FY1 = direction_to_target_horizontal / np.linalg.norm(
    direction_to_target_horizontal)
46 v_FY1 = drone_speed * u_FY1
47
48 print(f"无人机FY1运动参数: ")
49 print(f"  初始位置: {P_FY1_0}")
50 print(f"  飞行方向: {u_FY1}")
51 print(f"  速度向量: {v_FY1}")
52 print(f"  水平飞行速度: {drone_speed} m/s\n")
53
54 # 计算烟幕弹投放和起爆位置
55 P_drop = P_FY1_0 + v_FY1 * t_drop
56 print(f"烟幕弹投放参数: ")
57 print(f"  投放时间: {t_drop} s")
58 print(f"  投放位置: {P_drop}")
59
60 dt_fall = t_exp - t_drop
61 P_exp = P_drop + v_FY1 * dt_fall + np.array([0, 0, -0.5 * g *
    dt_fall**2])
62 print(f"  起爆时间: {t_exp} s")
63 print(f"  起爆位置: {P_exp}\n")
64
65 # 导弹在起爆时刻的位置
66 P_missile_at_exp = P_M1_0 + v_M1 * t_exp

```

```

67 print(f"=== 起爆时刻导弹位置 ===")
68 print(f"导弹在烟幕弹起爆时({t_exp}s)的坐标: {P_missile_at_exp}\n")
69
70 # 分析导弹与烟幕弹的相对位置
71 smoke_exp_x = P_exp[0]
72 smoke_exp_z = P_exp[2]
73
74 t_missile_at_smoke_x = (smoke_exp_x - P_M1_0[0]) / v_M1[0]
75 P_missile_at_smoke_x = P_M1_0 + v_M1 * t_missile_at_smoke_x
76 missile_height_at_smoke_x = P_missile_at_smoke_x[2]
77
78 print(f"=== 导弹与烟幕弹高度比较 ===")
79 print(f"烟幕弹起爆时的x坐标: {smoke_exp_x:.2f} m")
80 print(f"导弹运动到该x坐标的时间: {t_missile_at_smoke_x:.2f} s"
81 )
81 print(f"导弹在该x坐标时的高度: {missile_height_at_smoke_x:.2f} m")
82 print(f"烟幕弹起爆时的高度: {smoke_exp_z:.2f} m")
83
84 height_difference = smoke_exp_z - missile_height_at_smoke_x
85 if height_difference > 0:
86     print(f"烟幕弹在导弹轨迹上方 {height_difference:.2f} m")
87 elif height_difference < 0:
88     print(f"烟幕弹在导弹轨迹下方 {abs(height_difference):.2f} m")
89 else:
90     print(f"烟幕弹与导弹轨迹在同一高度")
91 print()
92
93 # 计算云团降落时间
94 target_height = 10
95 explosion_height = P_exp[2]
96
97 print(f"=== 云团降落分析 ===")
98 print(f"起爆位置高度: {explosion_height:.2f} m")
99
100 fall_distance = explosion_height - target_height
101
102 if fall_distance > 0:
103     fall_time = fall_distance / smoke_fall_speed
104     time_at_10m = t_exp + fall_time
105
106     print(f"需要下降距离: {fall_distance:.2f} m")

```

```

107     print(f"下降时间: {fall_time:.2f} s")
108     print(f"云团中心降落到离地面10m时的总用时: {time_at_10m:.2f} s\n")
109 else:
110     print(f"起爆位置已低于10m高度")
111     time_at_10m = t_exp
112     print(f"云团中心降落到离地面10m时的总用时: {time_at_10m:.2f} s\n")
113
114 def get_missile_position(t):
115     """计算指定时刻导弹位置"""
116     return P_M1_0 + v_M1 * t
117
118 def get_smoke_center(t):
119     """计算指定时刻烟幕中心位置"""
120     if t < t_drop:
121         return None
122     elif t < t_exp:
123         # 自由落体阶段
124         dt_current = t - t_drop
125         return P_drop + v_FY1 * dt_current + np.array([0, 0,
-0.5 * g * dt_current**2])
126     else:
127         # 云团下沉阶段
128         dt_fall_current = t_exp - t_drop
129         dt_sink = t - t_exp
130         P_exp_calc = P_drop + v_FY1 * dt_fall_current + np.
array([0, 0, -0.5 * g * dt_fall_current**2])
131         return P_exp_calc + np.array([0, 0, -smoke_fall_speed
* dt_sink])
132
133 def is_missile_in_smoke(t):
134     """判断指定时刻导弹是否在烟幕球体内"""
135     if t < t_exp:
136         return False
137
138     P_missile = get_missile_position(t)
139     C_smoke = get_smoke_center(t)
140
141     if C_smoke is None:
142         return False
143
144     distance = np.linalg.norm(P_missile - C_smoke)
145     return distance <= effective_radius

```

```

146
147 def is_fully_shielded(t):
148     """判断指定时刻圆柱体目标是否被完全遮挡"""
149     if t < t_exp:
150         return False
151
152     P_missile = get_missile_position(t)
153     C_smoke = get_smoke_center(t)
154
155     if C_smoke is None:
156         return False
157
158     # 对圆柱体进行采样
159     thetas = np.linspace(0.0, 2.0 * np.pi, 100, endpoint=False
160 )
161
162     x_circle = R_cyl * np.cos(thetas)
163     y_circle = C_base[1] + R_cyl * np.sin(thetas)
164
165     # 顶圆周
166     z_top = np.full_like(x_circle, C_top[2])
167     P_top = np.column_stack((x_circle, y_circle, z_top))
168
169     # 底圆周
170     z_bot = np.full_like(x_circle, C_base[2])
171     P_bot = np.column_stack((x_circle, y_circle, z_bot))
172
173     target_points = np.vstack((P_top, P_bot))
174     R_sq = effective_radius * effective_radius
175
176     # 检查所有采样点是否被遮挡
177     for T in target_points:
178         v = T - P_missile
179         v_len_sq = np.dot(v, v)
180
181         if v_len_sq < 1e-10: # 避免数值误差
182             # 导弹和目标重合的特殊情况
183             distance = np.linalg.norm(C_smoke - P_missile)
184         else:
185             # 计算投影参数
186             w = C_smoke - P_missile
187             t_proj = np.dot(w, v) / v_len_sq
188
189             # 计算烟幕中心到直线的最近点

```

```

189         closest_point = P_missile + t_proj * v
190
191         # 烟幕中心到直线的距离
192         distance = np.linalg.norm(C_smoke - closest_point)
193
194         # 烟幕中心到直线距离小于半径
195         if distance > effective_radius:
196             return False
197
198         # 方位关系判断（仅当导弹不在烟幕内时）
199         if v_len_sq >= 1e-10:
200             inside_smoke = np.linalg.norm(P_missile - C_smoke)
201             <= effective_radius
202
203             if not inside_smoke:
204                 if not (0.0 <= t_proj <= 1.0):
205                     return False
206
207         return True # 所有点都被遮挡
208
209 def binary_search_transition(t_start, t_end, condition_func,
210                             search_direction='first_true', tolerance=1e-8):
211     """通用二分法搜索函数，用于找到条件函数状态转换的精确时刻"""
212     left, right = t_start, t_end
213     while right - left > tolerance:
214         mid = (left + right) / 2
215         if condition_func(mid):
216             if search_direction == 'first_true':
217                 right = mid # 向左搜索第一个真值点
218             else:
219                 left = mid # 向右搜索最后一个真值点
220         else:
221             if search_direction == 'first_true':
222                 left = mid # 向右搜索第一个真值点
223             else:
224                 right = mid # 向左搜索最后一个真值点
225     return (left + right) / 2
226
227 # 遮挡判定分析
228 t_start = 5.1
229 effective_duration = 20.0
230 t_end = t_start + effective_duration

```

```

230 print("\n=== 开始进行遮挡判定分析===")
231 print(f"分析时间窗口：从 {t_start:.2f}s 到 {t_end:.2f}s")
232
233 # 粗略搜索找到所有关键时间段
234 dt_search = 0.1 # 粗略搜索步长
235 current_time = t_start
236
237 # 寻找所有关键时间段
238 shielding_start_rough = None
239 shielding_end_rough = None
240 missile_enter_rough = None
241 missile_exit_rough = None
242
243 last_shielded = False
244 last_in_smoke = False
245
246 print("\n--- 粗略搜索所有关键时间段 ---")
247
248 while current_time < t_end:
249     current_shielded = is_fully_shielded(current_time)
250     current_in_smoke = is_missile_in_smoke(current_time)
251
252     # 检测遮挡开始
253     if not last_shielded and current_shielded and
shielding_start_rough is None:
254         shielding_start_rough = (current_time - dt_search,
current_time)
255         print(f"检测到遮挡开始区间：[{shielding_start_rough
[0]:.2f}s, {shielding_start_rough[1]:.2f}s]")
256
257     # 检测遮挡结束
258     if last_shielded and not current_shielded and
shielding_end_rough is None:
259         shielding_end_rough = (current_time - dt_search,
current_time)
260         print(f"检测到遮挡结束区间：[{shielding_end_rough
[0]:.2f}s, {shielding_end_rough[1]:.2f}s]")
261
262     # 检测导弹穿入烟幕
263     if not last_in_smoke and current_in_smoke and
missile_enter_rough is None:
264         missile_enter_rough = (current_time - dt_search,
current_time)
265         print(f"检测到导弹穿入烟幕区间：[{missile_enter_rough

```

```

[0]:.2f}s, {missile_enter_rough[1]:.2f}s]")
266
267     # 检测导弹穿出烟幕
268     if last_in_smoke and not current_in_smoke and
missile_exit_rough is None:
269         missile_exit_rough = (current_time - dt_search,
current_time)
270         print(f"检测到导弹穿出烟幕区间: [{missile_exit_rough
[0]:.2f}s, {missile_exit_rough[1]:.2f}s]")
271
272         last_shielded = current_shielded
273         last_in_smoke = current_in_smoke
274         current_time += dt_search
275
276 print("\n--- 精确计算所有关键时刻 ---")
277
278 # 使用二分法精确计算遮挡关键时刻
279 first_shielding_time = None
280 shielding_end_time = None
281
282 if shielding_start_rough is not None:
283     first_shielding_time = binary_search_transition(
284         shielding_start_rough[0], shielding_start_rough[1],
285         is_fully_shielded, 'first_true'
286     )
287     print(f"首次完全遮挡时刻: {first_shielding_time:.8f}s")
288
289 if shielding_end_rough is not None:
290     shielding_end_time = binary_search_transition(
291         shielding_end_rough[0], shielding_end_rough[1],
292         is_fully_shielded, 'last_true'
293     )
294     print(f"遮挡结束时刻: {shielding_end_time:.8f}s")
295
296 # 使用二分法精确计算导弹穿越时刻
297 missile_enter_time = None
298 missile_exit_time = None
299
300 if missile_enter_rough is not None:
301     missile_enter_time = binary_search_transition(
302         missile_enter_rough[0], missile_enter_rough[1],
303         is_missile_in_smoke, 'first_true'
304     )
305     print(f"导弹穿入烟幕时刻: {missile_enter_time:.8f}s")

```



```

306
307 if missile_exit_rough is not None:
308     missile_exit_time = binary_search_transition(
309         missile_exit_rough[0], missile_exit_rough[1],
310         is_missile_in_smoke, 'last_true'
311     )
312     print(f"导弹穿出烟幕时刻: {missile_exit_time:.8f}s")
313
314 # 计算最终结果
315 print("\n=== 分析完成 ===")
316
317 if first_shielding_time is not None and shielding_end_time is
    not None:
318     total_shielded_duration = shielding_end_time -
        first_shielding_time
319     print(f"问题一: 烟幕干扰弹对M1的有效遮蔽时长为: {
        total_shielded_duration:.8f} 秒")
320     print(f"    - 首次完全遮挡时刻: {first_shielding_time:.8f}s"
        )
321     print(f"    - 遮挡结束时刻: {shielding_end_time:.8f}s")
322 else:
323     print("未检测到完全遮挡时间段")
324
325 # 导弹穿越分析
326 print("\n=== 导弹穿越烟幕分析 ===")
327 if missile_enter_time is not None and missile_exit_time is not
    None:
328     missile_smoke_duration = missile_exit_time -
        missile_enter_time
329     print(f"导弹在烟幕中的时长: {missile_smoke_duration:.8f}
        秒")
330     print(f"    - 穿入时刻: {missile_enter_time:.8f}s")
331     print(f"    - 穿出时刻: {missile_exit_time:.8f}s")
332 else:
333     print("导弹未穿越烟幕球体")
334
335 # 最终遮挡时间段计算
336 print("\n=== 最终遮挡时间段计算 ===")
337 if first_shielding_time is not None and missile_exit_time is
    not None:
338     final_shielding_duration = missile_exit_time -
        first_shielding_time
339     print(f"最终有效遮蔽时长 (穿出时刻 - 首次遮挡时刻): {
        final_shielding_duration:.8f} 秒")

```

```

340     print(f" - 首次完全遮挡时刻: {first_shielding_time:.8f}s"
341           )
342     print(f" - 导弹穿出烟幕时刻: {missile_exit_time:.8f}s")
343     print(f" - 时间差: {final_shielding_duration:.8f}s")
344 else:
    print("无法计算最终遮挡时长（缺少首次遮挡时刻或导弹穿出时
    刻）")

```

q2.py

```

1  import numpy as np
2  import pandas as pd
3  import random
4  import concurrent.futures
5  from functools import lru_cache
6
7  VERBOSE = True
8
9  class SmokeShieldingCalculator:
10     """物理模型和遮蔽时长计算"""
11
12     def __init__(self,
13                 missile_speed=300,
14                 smoke_fall_speed=3,
15                 effective_radius=10,
16                 effective_duration=20,
17                 gravity=9.80665,
18                 edge_sample_num=100):
19         # 物理参数
20         self.missile_speed = missile_speed
21         self.smoke_fall_speed = smoke_fall_speed
22         self.effective_radius = effective_radius
23         self.effective_duration = effective_duration
24         self.g = gravity
25         self.edge_sample_num = edge_sample_num
26
27         # 固定的场景参数
28         self.P_M1_0 = np.array([20000, 0, 2000])
29         self.P_target = np.array([0, 0, 0])
30         self.P_FY1_0 = np.array([17800, 0, 1800])
31
32         # 圆柱体目标参数
33         self.C_base = np.array([0, 200, 0])
34         self.C_top = np.array([0, 200, 10])
35         self.R_cyl = 7

```

```

36         self.H_cyl = 10
37
38         # 导弹运动参数
39         self._setup_missile_trajectory()
40
41         # 计算圆柱体边缘点
42         self._edge_points = self._get_cylinder_edge_points()
43
44     def _setup_missile_trajectory(self):
45         """设置导弹轨迹参数"""
46         d_M1 = self.P_target - self.P_M1_0
47         distance_M1 = np.linalg.norm(d_M1)
48         self.u_M1 = d_M1 / distance_M1
49         self.v_M1 = self.missile_speed * self.u_M1
50
51     def _get_unit_direction_vector(self, alpha):
52         """根据航向角计算单位方向向量"""
53         return np.array([np.cos(alpha), np.sin(alpha), 0])
54
55     def _get_missile_position(self, t):
56         """计算t时刻导弹位置"""
57         return self.P_M1_0 + self.v_M1 * t
58
59     def _get_drone_position(self, t, alpha, v_D, t_drop):
60         """计算t时刻无人机位置"""
61         e_alpha = self._get_unit_direction_vector(alpha)
62         return self.P_FY1_0 + v_D * t * e_alpha
63
64     def _get_smoke_bomb_position(self, t, alpha, v_D, t_drop,
65 t_explode):
66         """计算t时刻烟幕弹位置"""
67         if t < t_drop:
68             return None
69         elif t <= t_explode:
70             # 自由落体阶段
71             e_alpha = self._get_unit_direction_vector(alpha)
72             R = self.P_FY1_0 + v_D * t_drop * e_alpha
73             dt = t - t_drop
74             bomb_position = R + v_D * dt * e_alpha + np.array
75             ([0, 0, -0.5 * self.g * dt**2])
76
77             # 确保烟幕弹高度不低于地面
78             if bomb_position[2] < 0:
79                 bomb_position[2] = 0

```

```

78
79         return bomb_position
80     else:
81         return None
82
83     def _get_smoke_cloud_center(self, t, alpha, v_D, t_drop,
t_explode):
84         """计算t时刻烟幕云团中心位置"""
85         if t < t_explode:
86             return None
87         elif t <= t_explode + self.effective_duration:
88             # 计算起爆位置
89             e_alpha = self._get_unit_direction_vector(alpha)
90             R = self.P_FY1_0 + v_D * t_drop * e_alpha
91             dt_fall = t_explode - t_drop
92             B = R + v_D * dt_fall * e_alpha + np.array([0, 0,
-0.5 * self.g * dt_fall**2])
93
94             # 确保起爆位置不低于地面
95             if B[2] < 0:
96                 B[2] = 0
97
98             # 云团下沉
99             dt_sink = t - t_explode
100             smoke_center = B + np.array([0, 0, -self.
smoke_fall_speed * dt_sink])
101
102             # 确保云团高度不低于地面
103             if smoke_center[2] < 0:
104                 smoke_center[2] = 0
105
106             return smoke_center
107         else:
108             return None
109
110     def _get_cylinder_edge_points(self):
111         """获取圆柱体上下底面边缘采样点"""
112         thetas = np.linspace(0.0, 2.0 * np.pi, self.
edge_sample_num, endpoint=False)
113
114         x_circle = self.R_cyl * np.cos(thetas)
115         y_circle = self.C_base[1] + self.R_cyl * np.sin(thetas
)
116

```

```

117         # 顶圆周
118         z_top = np.full_like(x_circle, self.C_top[2])
119         P_top = np.column_stack((x_circle, y_circle, z_top))
120
121         # 底圆周
122         z_bot = np.full_like(x_circle, self.C_base[2])
123         P_bot = np.column_stack((x_circle, y_circle, z_bot))
124
125         return np.vstack((P_top, P_bot))
126
127     def _distance_point_to_line(self, P_missile, P_target,
128 C_smoke):
129         """计算烟幕中心到导弹-目标连线的距离"""
130         vec_missile_to_target = P_target - P_missile
131         vec_missile_to_smoke = C_smoke - P_missile
132
133         missile_target_distance = np.linalg.norm(
134 vec_missile_to_target)
135         if missile_target_distance < 1e-10:
136             return np.linalg.norm(vec_missile_to_smoke)
137
138         cross_product = np.cross(vec_missile_to_target,
139 vec_missile_to_smoke)
140         cross_product_norm = np.linalg.norm(cross_product)
141
142         return cross_product_norm / missile_target_distance
143
144     @lru_cache(maxsize=None)
145     def _has_missile_ever_penetrated(self, alpha, v_D, t_drop,
146 t_explode, t_now, dt=0.05):
147         """Y2: 从 t_explode 到 t_now 是否曾进入烟幕球体"""
148         t = t_explode
149         while t <= t_now:
150             C_smoke = self._get_smoke_cloud_center(t, alpha,
151 v_D, t_drop, t_explode)
152             if C_smoke is None:
153                 return False
154             dist = np.linalg.norm(self._get_missile_position(t
155 ) - C_smoke)
156             if dist <= self.effective_radius + 1e-9:
157                 return True
158             t += dt
159         return False

```

```

155     def _check_penetration_exit_shielding(self, P_missile,
156     C_smoke, edge_points):
157         """穿出时刻的向量夹角判定"""
158         V_mt = edge_points - P_missile
159         W = C_smoke - P_missile
160         dot_ws = np.dot(W, V_mt.T)
161         return not np.any(dot_ws <= 0)
162
163     def _check_general_shielding(self, P_missile, C_smoke,
164     edge_points):
165         """一般情况的遮挡判定"""
166         V_mt = edge_points - P_missile
167         V_length_sq = np.einsum('ij,ij->i', V_mt, V_mt)
168         W = C_smoke - P_missile
169         cross = np.cross(V_mt, W)
170         cross_norm = np.linalg.norm(cross, axis=1)
171
172         with np.errstate(divide='ignore', invalid='ignore'):
173             distances = np.where(V_length_sq > 1e-12,
174             cross_norm / np.sqrt(V_length_sq),
175             np.linalg.norm(C_smoke -
176             P_missile))
177
178         if np.any(distances > self.effective_radius):
179             return False
180
181         # 方位判定
182         dot_ws = np.dot(W, V_mt.T)
183         with np.errstate(divide='ignore', invalid='ignore'):
184             t_proj = np.where(V_length_sq > 1e-12, dot_ws /
185             V_length_sq, 0.0)
186
187         return not np.any((t_proj < 0.0) | (t_proj > 1.0))
188
189     def _is_target_shielded(self, t, alpha, v_D, t_drop,
190     t_explode):
191         """判断t时刻目标是否被完全遮蔽"""
192         # 只在烟幕云团存在期间进行判定
193         if t < t_explode or t > t_explode + self.
194         effective_duration:
195             return False
196
197         # 获取当前时刻的位置
198         P_missile = self._get_missile_position(t)

```

```

192         C_smoke = self._get_smoke_cloud_center(t, alpha, v_D,
193         t_drop, t_explode)
194
195         if C_smoke is None:
196             return False
197
198         # 判定是否曾经穿入云团
199         Y2 = self._has_missile_ever_penetrated(alpha, v_D,
200         t_drop, t_explode, t, dt=0.05)
201
202         # 判断导弹和云团的位置关系
203         missile_to_smoke_distance = np.linalg.norm(P_missile -
204         C_smoke)
205         tolerance = 1e-6
206
207         if missile_to_smoke_distance < self.effective_radius -
208         tolerance:
209             # R < 10: 导弹在云团内, 必然遮蔽
210             return True
211         elif abs(missile_to_smoke_distance - self.
212         effective_radius) <= tolerance:
213             # R = 10: 导弹在边界上
214             if Y2:
215                 # 曾穿入且在边界: 穿出时刻, 需要向量夹角判定
216                 return self._check_penetration_exit_shielding(
217                 P_missile, C_smoke, self._edge_points)
218             else:
219                 # 未穿入且在边界: 按一般判定
220                 return self._check_general_shielding(P_missile
221                 , C_smoke, self._edge_points)
222             else:
223                 # R > 10: 导弹在云团外
224                 if Y2:
225                     # 曾穿入但已在外部: 按一般判定
226                     return self._check_general_shielding(P_missile
227                     , C_smoke, self._edge_points)
228                 else:
229                     # 未穿入且在外部: 按一般判定
230                     return self._check_general_shielding(P_missile
231                     , C_smoke, self._edge_points)
232
233     def calculate_shielding_duration(self, alpha, v_D, t_drop,
234     t_explode, eps=1e-4):
235         """计算给定参数下的总遮蔽时长"""

```

```

226     # 约束检查
227     if t_drop > t_explode:
228         return 0.0
229
230     if not (0 <= alpha <= 2*np.pi and 70 <= v_D <= 140):
231         return 0.0
232
233     # 使用二分法精确计算遮蔽区间
234     def cond(t):
235         return self._is_target_shielded(t, alpha, v_D,
t_drop, t_explode)
236
237     # 粗扫
238     ts = np.arange(t_explode, t_explode + self.
effective_duration + 0.05, 0.05)
239     mask = np.array([cond(t) for t in ts])
240     if not mask.any():
241         return 0.0
242
243     idx0 = np.argmax(mask)
244     idx1 = len(mask) - 1 - np.argmax(mask[::-1])
245
246     def bisection(left, right, target_first_true):
247         """二分找第一次 True 或最后一次 True"""
248         for _ in range(40):
249             mid = 0.5 * (left + right)
250             if cond(mid):
251                 if target_first_true:
252                     right = mid
253                 else:
254                     left = mid
255             else:
256                 if target_first_true:
257                     left = mid
258                 else:
259                     right = mid
260             if right - left < eps:
261                 break
262         return 0.5 * (left + right)
263
264     # 第一个遮蔽精确时刻
265     t_left_lo = ts[max(idx0 - 1, 0)]
266     t_left_hi = ts[idx0]
267     T0 = bisection(t_left_lo, t_left_hi, True)

```



```

268
269     # 最后一个遮蔽精确时刻
270     t_right_lo = ts[idx1]
271     t_right_hi = ts[min(idx1 + 1, len(ts)-1)]
272     T1 = bisection(t_right_lo, t_right_hi, False)
273
274     return max(T1 - T0, 0.0)
275
276 def get_scenario_info(self):
277     """获取场景信息"""
278     return {
279         '导弹初始位置': self.P_M1_0.tolist(),
280         '目标位置': self.P_target.tolist(),
281         '无人机初始位置': self.P_FY1_0.tolist(),
282         '圆柱体底面中心': self.C_base.tolist(),
283         '圆柱体顶面中心': self.C_top.tolist(),
284         '圆柱体半径': self.R_cyl,
285         '圆柱体高度': self.H_cyl,
286         '导弹速度': self.missile_speed,
287         '烟幕下沉速度': self.smoke_fall_speed,
288         '烟幕有效半径': self.effective_radius,
289         '烟幕有效持续时间': self.effective_duration
290     }
291
292 class ParticleSwarmOptimizer:
293     """粒子群优化算法求解器"""
294
295     # 用于并行计算的计算器实例
296     _shared_calculator = None
297
298     def __init__(self, n_particles=50, n_iterations=100, w
299 =0.9, c1=2.0, c2=2.0):
300         """初始化粒子群优化器"""
301         self.n_particles = n_particles
302         self.n_iterations = n_iterations
303         self.w = w
304         self.c1 = c1
305         self.c2 = c2
306
307         # 创建计算器实例
308         self.calculator = SmokeShieldingCalculator()
309
310         # 设置共享计算器
311         if ParticleSwarmOptimizer._shared_calculator is None:

```

```

311         ParticleSwarmOptimizer._shared_calculator =
SmokeShieldingCalculator()
312
313     # 定义决策变量的边界
314     self.bounds = np.array([
315         [0, 2*np.pi],      # alpha
316         [70, 140],          # v_D
317         [0, 27.0],           # t_drop
318         [0, 46.0]            # t_explode
319     ])
320
321     self.dimension = len(self.bounds)
322
323     # 粒子群状态
324     self.particles = None
325     self.velocities = None
326     self.personal_best_positions = None
327     self.personal_best_scores = None
328     self.global_best_position = None
329     self.global_best_score = -np.inf
330
331     # 历史记录
332     self.history = {
333         'best_scores': [],
334         'mean_scores': [],
335         'best_positions': []
336     }
337
338     def gradient_objective_function(self, params):
339         """梯度化适应度函数：建立分层奖励机制"""
340         alpha, v_D, t_drop, t_explode = params
341
342         # 不可行解给予严重惩罚
343         if t_drop > t_explode:
344             return -100.0
345         if (t_explode - t_drop) > 19.1:
346             return -100.0
347         if not (0 <= alpha <= 2*np.pi and 70 <= v_D <= 140):
348             return -100.0
349
350         # 精确遮蔽奖励
351         duration = self.calculator.
calculate_shielding_duration(alpha, v_D, t_drop, t_explode)
352         if duration > 0:

```

```

353         return duration
354
355     # 梯度化奖励
356     proximity_score = self.
calculate_proximity_to_shielding(alpha, v_D, t_drop,
t_explode)
357     return -1.0 + 0.95 * proximity_score
358
359     def objective_function(self, params):
360         """目标函数：使用梯度化适应度函数"""
361         return self.gradient_objective_function(params)
362
363     @staticmethod
364     def static_objective_function(params):
365         """静态目标函数：用于并行计算"""
366         alpha, v_D, t_drop, t_explode = params
367
368         # 不可行解给予严重惩罚
369         if t_drop > t_explode:
370             return -100.0
371         if (t_explode - t_drop) > 19.1:
372             return -100.0
373         if not (0 <= alpha <= 2*np.pi and 70 <= v_D <= 140):
374             return -100.0
375
376         # 使用共享计算器
377         if ParticleSwarmOptimizer._shared_calculator is None:
378             ParticleSwarmOptimizer._shared_calculator =
SmokeShieldingCalculator()
379             calculator = ParticleSwarmOptimizer._shared_calculator
380
381         # 精确遮蔽奖励
382         duration = calculator.calculate_shielding_duration(
alpha, v_D, t_drop, t_explode)
383         if duration > 0:
384             return duration
385
386         # 梯度化奖励
387         proximity_score = ParticleSwarmOptimizer.
_static_calculate_proximity_to_shielding(alpha, v_D, t_drop,
t_explode)
388         return -1.0 + 0.95 * proximity_score
389
390     @staticmethod

```

```

391     def _static_calculate_proximity_to_shielding(alpha, v_D,
392 t_drop, t_explode):
393         """静态版本的接近度计算函数，用于并行计算"""
394         scores = []
395         # 确保共享计算器存在
396         if ParticleSwarmOptimizer._shared_calculator is None:
397             ParticleSwarmOptimizer._shared_calculator =
SmokeShieldingCalculator()
398             calculator = ParticleSwarmOptimizer._shared_calculator
399
400         # 检查几个关键时刻的接近程度
401         test_times = [t_explode + i for i in [1, 3, 5, 8, 10,
15]]
402         test_times = [t for t in test_times if t <= t_explode
+ calculator.effective_duration]
403
404         if not test_times:
405             return 0.0
406
407         for t in test_times:
408             if t > t_explode + calculator.effective_duration:
409                 continue
410
411             P_missile = calculator._get_missile_position(t)
412             C_smoke = calculator._get_smoke_cloud_center(t,
alpha, v_D, t_drop, t_explode)
413
414             if C_smoke is not None:
415                 # 导弹-烟幕距离接近度 (40%)
416                 distance = np.linalg.norm(P_missile - C_smoke)
417                 distance_score = max(0, 1 - distance / (3 *
calculator.effective_radius))
418
419                 # 导弹-目标连线与烟幕的几何关系 (40%)
420                 line_distance = calculator.
_distance_point_to_line(P_missile, calculator.P_target,
C_smoke)
421                 line_score = max(0, 1 - line_distance / (2 *
calculator.effective_radius))
422
423                 # 时间窗口合理性 (20%)
424                 time_score = 1.0 if 2 <= t_explode <= 12 else
0.3

```

```

425         combined_score = (0.4 * distance_score + 0.4 *
line_score + 0.2 * time_score)
426         scores.append(combined_score)
427
428         return max(scores) if scores else 0.0
429
430     def calculate_proximity_to_shielding(self, alpha, v_D,
t_drop, t_explode):
431         """计算接近遮蔽的程度，返回[0,1]"""
432         scores = []
433
434         # 检查几个关键时刻的接近程度
435         test_times = [t_explode + i for i in [1, 3, 5, 8, 10,
15]]
436         test_times = [t for t in test_times if t <= t_explode
+ self.calculator.effective_duration]
437
438         if not test_times:
439             return 0.0
440
441         for t in test_times:
442             if t > t_explode + self.calculator.
effective_duration:
443                 continue
444
445             P_missile = self.calculator._get_missile_position(
t)
446             C_smoke = self.calculator._get_smoke_cloud_center(
t, alpha, v_D, t_drop, t_explode)
447
448             if C_smoke is not None:
449                 # 导弹-烟幕距离接近度 (40%)
450                 distance = np.linalg.norm(P_missile - C_smoke)
451                 distance_score = max(0, 1 - distance / (3 *
self.calculator.effective_radius))
452
453                 # 导弹-目标连线与烟幕的几何关系 (40%)
454                 line_distance = self.calculator.
_distance_point_to_line(P_missile, self.calculator.P_target,
C_smoke)
455                 line_score = max(0, 1 - line_distance / (2 *
self.calculator.effective_radius))
456
457                 # 时间窗口合理性 (20%)

```

```

458         time_score = 1.0 if 2 <= t_explode <= 12 else
0.3
459
460         combined_score = (0.4 * distance_score + 0.4 *
line_score + 0.2 * time_score)
461         scores.append(combined_score)
462
463         return max(scores) if scores else 0.0
464
465     def initialize_particles(self):
466         """初始化粒子群"""
467         debug_print("初始化粒子群...")
468
469         self.particles = np.zeros((self.n_particles, self.
dimension))
470         self.velocities = np.zeros((self.n_particles, self.
dimension))
471         self.personal_best_positions = np.zeros((self.
n_particles, self.dimension))
472         self.personal_best_scores = np.full(self.n_particles,
-np.inf)
473
474         # 随机初始化粒子位置
475         for i in range(self.n_particles):
476             for j in range(self.dimension):
477                 min_val, max_val = self.bounds[j]
478                 self.particles[i, j] = random.uniform(min_val,
max_val)
479
480         # 确保时间约束
481         if self.particles[i, 2] > self.particles[i, 3]:
482             self.particles[i, 2], self.particles[i, 3] =
self.particles[i, 3], self.particles[i, 2]
483
484         # 初始化速度
485         for j in range(self.dimension):
486             range_val = self.bounds[j, 1] - self.bounds[j,
0]
487             self.velocities[i, j] = random.uniform(-0.1 *
range_val, 0.1 * range_val)
488
489         # 并行计算初始适应度
490         with concurrent.futures.ProcessPoolExecutor() as pool:
491             scores = list(pool.map(ParticleSwarmOptimizer.

```

```

static_objective_function, self.particles.tolist()))
492     for i, score in enumerate(scores):
493         self.personal_best_scores[i] = score
494         self.personal_best_positions[i] = self.particles[i
].copy()
495         if score > self.global_best_score:
496             self.global_best_score = score
497             self.global_best_position = self.particles[i].
copy()
498
499         debug_print(f"初始化完成, 最佳初始适应度: {self.
global_best_score:.6f}")
500
501     def update_particles(self):
502         """更新粒子位置和速度"""
503         for i in range(self.n_particles):
504             # 更新速度
505             r1, r2 = random.random(), random.random()
506
507             cognitive_component = self.c1 * r1 * (self.
personal_best_positions[i] - self.particles[i])
508             social_component = self.c2 * r2 * (self.
global_best_position - self.particles[i])
509
510             self.velocities[i] = (self.w * self.velocities[i]
+
511                                 cognitive_component +
social_component)
512
513             # 限制速度大小
514             for j in range(self.dimension):
515                 range_val = self.bounds[j, 1] - self.bounds[j,
0]
516                 max_velocity = 0.2 * range_val
517                 self.velocities[i, j] = np.clip(self.
velocities[i, j], -max_velocity, max_velocity)
518
519             # 更新位置
520             self.particles[i] += self.velocities[i]
521
522             # 边界约束
523             for j in range(self.dimension):
524                 self.particles[i, j] = np.clip(self.particles[
i, j],

```

```

525                                     self.bounds[j,
0], self.bounds[j, 1])
526         # 约束修复
527         t_drop = self.particles[i, 2]
528         t_explode = self.particles[i, 3]
529
530         # 修复时序约束
531         if t_drop > t_explode:
532             t_drop, t_explode = t_explode, t_drop
533
534         # 修复时间差约束
535         if (t_explode - t_drop) > 19.2:
536             t_explode = t_drop + 19.2
537             # 修复后可能超出上界，需要再次clip
538             t_explode = np.clip(t_explode, self.bounds[3,
0], self.bounds[3, 1])
539
540         # 更新粒子的时间参数
541         self.particles[i, 2] = t_drop
542         self.particles[i, 3] = t_explode
543
544     def evaluate_and_update_best(self):
545         """评估粒子适应度并更新最佳位置"""
546         with concurrent.futures.ProcessPoolExecutor() as pool:
547             scores = list(pool.map(ParticleSwarmOptimizer.
static_objective_function, self.particles.tolist()))
548
549         for i, score in enumerate(scores):
550             if score > self.personal_best_scores[i]:
551                 self.personal_best_scores[i] = score
552                 self.personal_best_positions[i] = self.
particles[i].copy()
553             if score > self.global_best_score:
554                 self.global_best_score = score
555                 self.global_best_position = self.particles[i].
copy()
556
557         # 记录历史
558         self.history['best_scores'].append(self.
global_best_score)
559         self.history['mean_scores'].append(np.mean(scores))
560         self.history['best_positions'].append(self.
global_best_position.copy())
561

```



```

562 def optimize(self):
563     """执行粒子群优化"""
564     debug_print("开始粒子群优化...")
565
566     # 初始化
567     self.initialize_particles()
568
569     # 迭代优化
570     for iteration in range(self.n_iterations):
571         # 更新粒子
572         self.update_particles()
573
574         # 评估适应度
575         self.evaluate_and_update_best()
576
577         # 动态调整惯性权重
578         self.w = 0.9 - 0.5 * iteration / self.n_iterations
579
580         # 输出进度
581         if (iteration + 1) % 5 == 0 or iteration == 0:
582             print(f"[Iter {iteration + 1:3d}]/{self.
n_iterations}] 最佳={self.global_best_score:.6f} 平均={
self.history['mean_scores'][-1]:.6f}")
583
584             print(f"优化结束：全局最佳适应度 = {self.
global_best_score:.6f}")
585
586             return self.get_optimization_result()
587
588     def get_optimization_result(self):
589         """获取优化结果"""
590         alpha_opt, v_D_opt, t_drop_opt, t_explode_opt = self.
global_best_position
591
592         result = {
593             'optimal_parameters': {
594                 'alpha_degrees': np.degrees(alpha_opt),
595                 'alpha_radians': alpha_opt,
596                 'v_D': v_D_opt,
597                 't_drop': t_drop_opt,
598                 't_explode': t_explode_opt
599             },
600             'optimal_shielding_duration': self.
global_best_score,

```

```

601         'optimization_history': self.history
602     }
603
604     return result
605
606 # 辅助函数
607 def save_pso_history(result_dict, filename="PSO_analysis.xlsx"
608 ):
609     """将 PSO 优化结果保存到 Excel 文件"""
610     optimal = result_dict["optimal_parameters"].copy()
611     optimal["shielding_duration"] = result_dict["
612     optimal_shielding_duration"]
613
614     df_summary = pd.DataFrame([optimal])
615
616     hist = result_dict["optimization_history"]
617     df_history = pd.DataFrame({
618         "iteration": range(1, len(hist["best_scores"]) + 1),
619         "best_score": hist["best_scores"],
620         "mean_score": hist["mean_scores"]
621     })
622
623     with pd.ExcelWriter(filename, engine="openpyxl") as writer
624     :
625         df_summary.to_excel(writer, sheet_name="summary",
626         index=False)
627         df_history.to_excel(writer, sheet_name="history",
628         index=False)
629
630     print(f"已保存优化结果到 {filename}")
631
632 def debug_print(*args, **kwargs):
633     """调试打印函数，根据VERBOSE控制是否输出"""
634     if VERBOSE:
635         print(*args, **kwargs)
636
637 # 主函数
638 def run_optimization_problem2():
639     """运行问题二的优化求解"""
640     print("=" * 60)
641     print("开始求解问题二：烟幕干扰弹投放策略优化")
642     print("=" * 60)
643
644     # 创建优化器

```

```

640     optimizer = ParticleSwarmOptimizer(
641         n_particles=200,          # 粒子数量
642         n_iterations=100,        # 迭代次数
643         w=0.9,                  # 惯性权重
644         c1=2.8,                 # 个体学习因子
645         c2=1.3                  # 群体学习因子
646     )
647     result = optimizer.optimize()
648
649     print("\n" + "=" * 60)
650     print("问题二优化结果")
651     print("=" * 60)
652
653     opt_params = result['optimal_parameters']
654     print(f"最优参数:")
655     print(f"  航向角  $\alpha$  = {opt_params['alpha_degrees']:.2f}° ({opt_params['alpha_radians']:.4f} 弧度)")
656     print(f"  无人机速度 v_D = {opt_params['v_D']:.2f} m/s")
657     print(f"  投放时刻 t' = {opt_params['t_drop']:.3f} s")
658     print(f"  起爆时刻 t'' = {opt_params['t_explode']:.3f} s")
659     print(f"\n最大遮蔽时长: {result['optimal_shielding_duration']:.6f} 秒")
660     save_pso_history(result, "PSO_analysis.xlsx")
661     return result
662
663 if __name__ == "__main__":
664     result = run_optimization_problem2()

```

q3.py

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import random
5  import concurrent.futures
6  from q2 import SmokeShieldingCalculator
7
8  VERBOSE = True
9
10 class MultiSmokeBombCalculator(SmokeShieldingCalculator):
11     """三枚烟幕弹协同遮蔽计算器"""
12
13     def __init__(self, **kwargs):
14         """初始化多烟幕弹计算器"""
15         super().__init__(**kwargs)

```

```

16
17     def _get_cylinder_sampling_points(self, theta_steps=24,
18 z_steps=10, r_steps=4):
19         """生成圆柱体表面采样点"""
20         points = []
21
22         # 侧面网格采样
23         theta_values = np.linspace(0, 2*np.pi, theta_steps,
24 endpoint=False)
25         z_values = np.linspace(self.C_base[2], self.C_top[2],
26 z_steps)
27         for theta in theta_values:
28             for z in z_values:
29                 x = self.R_cyl * np.cos(theta)
30                 y = self.C_base[1] + self.R_cyl * np.sin(theta
31 )
32                 points.append([x, y, z])
33
34         # 顶面极坐标采样
35         theta_values_top = np.linspace(0, 2*np.pi, theta_steps
36 , endpoint=False)
37         r_values = np.linspace(0, self.R_cyl, r_steps,
38 endpoint=False)
39         for r in r_values:
40             if r == 0:
41                 points.append([0, self.C_top[1], self.C_top
42 [2]])
43             else:
44                 for theta in theta_values_top:
45                     x = r * np.cos(theta)
46                     y = self.C_top[1] + r * np.sin(theta)
47                     points.append([x, y, self.C_top[2]])
48         return np.array(points)
49
50     def _get_missile_position_vec(self, ts):
51         """向量化计算导弹在多个时刻的位置"""
52         ts = np.asarray(ts)
53         positions = self.P_M1_0[np.newaxis, :] + self.v_M1[np.
54 newaxis, :] * ts[:, np.newaxis]
55         return positions
56
57     def _get_smoke_cloud_center_vec(self, ts, alpha, v_D,
58 t_drop, t_explode):
59         """向量化计算烟幕云团在多个时刻的中心位置"""

```

```

51         ts = np.asarray(ts)
52         N = len(ts)
53         centers = np.full((N, 3), np.nan)
54         valid_mask = (ts >= t_explode) & (ts <= t_explode +
self.effective_duration)
55         if not np.any(valid_mask):
56             return centers
57
58         # 只对有效时间进行计算
59         valid_ts = ts[valid_mask]
60         valid_count = len(valid_ts)
61
62         # 计算方向向量
63         e_alpha = self._get_unit_direction_vector(alpha)
64
65         # 计算起爆位置 B
66         dt_fall = t_explode - t_drop
67         R = self.P_FY1_0 + v_D * t_drop * e_alpha
68         B = R + v_D * dt_fall * e_alpha + np.array([0, 0, -0.5
* self.g * dt_fall**2])
69         if B[2] < 0:
70             B[2] = 0
71
72         # 计算云团下沉时间
73         dt_sink = valid_ts - t_explode
74
75         # 向量化计算云团位置
76         valid_centers = np.tile(B, (valid_count, 1))
77         valid_centers[:, 2] -= self.smoke_fall_speed * dt_sink
78
79         valid_centers[:, 2] = np.maximum(valid_centers[:, 2],
0)
80
81         centers[valid_mask] = valid_centers
82
83         return centers
84
85     def _distance_point_to_line_vec(self, P_missile_array,
P_target, C_smoke_array):
86         """向量化计算多个烟幕中心到导弹-目标连线的距离"""
87
88         # 导弹到目标
89         vec_missile_to_target = P_target[np.newaxis, :] -
P_missile_array

```

```

90
91     # 导弹到烟幕
92     vec_missile_to_smoke = C_smoke_array - P_missile_array
93
94     # 目标距离
95     missile_target_distances = np.linalg.norm(
vec_missile_to_target, axis=1)
96
97     # 处理距离过小的情况
98     small_distance_mask = missile_target_distances < 1e-10
99
100    # 计算叉积
101    cross_products = np.cross(vec_missile_to_target,
vec_missile_to_smoke)
102    cross_product_norms = np.linalg.norm(cross_products,
axis=1)
103
104    # 计算距离
105    distances = np.zeros(len(P_missile_array))
106
107    normal_mask = ~small_distance_mask
108    distances[normal_mask] = cross_product_norms[
normal_mask] / missile_target_distances[normal_mask]
109
110    # 导弹到目标距离很小时，直接返回导弹到烟幕的距离
111    distances[small_distance_mask] = np.linalg.norm(
vec_missile_to_smoke[small_distance_mask], axis=1)
112
113    return distances
114
115    def _is_target_shielded_by_any_bomb_vec(self, ts, alpha,
v_D, t_drop_list, t_explode_list):
116        """判断多个时刻目标圆柱体是否被多个烟幕弹完全遮蔽"""
117        ts = np.asarray(ts)
118        N = len(ts)
119
120        # 初始化结果数组
121        shielded_mask = np.zeros(N, dtype=bool)
122
123        # 对每个时刻进行判定
124        for i, t in enumerate(ts):
125            shielded_mask[i] = self.
_is_target_fully_shielded_multi_bomb(t, alpha, v_D,
t_drop_list, t_explode_list)

```

```

126
127     return shielded_mask
128
129     def calculate_multi_shielding_duration(self, alpha, v_D,
130     t_drop_list, t_explode_list, eps=0.05):
131         """计算三枚烟幕弹协同作用下的总遮蔽时长"""
132         if not self._check_constraints(alpha, v_D, t_drop_list
133     , t_explode_list):
134             return 0.0
135
136         earliest_start = min(t_explode_list)
137         latest_end = max(t_explode_list) + self.
138     effective_duration
139
140         # 向量化扫描找出所有遮蔽时刻
141         ts = np.arange(earliest_start, latest_end + eps, eps)
142         mask = self._is_target_shielded_by_any_bomb_vec(ts,
143     alpha, v_D, t_drop_list, t_explode_list)
144
145         if not mask.any():
146             return 0.0
147
148         # 为二分法定义条件函数
149         def cond(t):
150             """检查t时刻是否被任意烟幕弹遮蔽"""
151             return self._is_target_shielded_by_any_bomb(t,
152     alpha, v_D, t_drop_list, t_explode_list)
153
154         # 计算所有遮蔽区间的总时长
155         total_duration = 0.0
156
157         # 找到所有连续的True区间
158         in_shielded_region = False
159         region_start = 0
160
161         for i, is_shielded in enumerate(mask):
162             if is_shielded and not in_shielded_region:
163                 # 开始一个新的遮蔽区间
164                 region_start = i
165                 in_shielded_region = True
166             elif not is_shielded and in_shielded_region:
167                 # 结束当前遮蔽区间
168                 region_end = i - 1

```

```

165         # 二分法计算区间边界
166         t_start_precise = self.
_bisection_find_boundary(
167             ts[max(region_start - 1, 0)], ts[
region_start], cond, True, eps/10
168         )
169         t_end_precise = self._bisection_find_boundary(
170             ts[region_end], ts[min(region_end + 1, len
(ts)-1)], cond, False, eps/10
171         )
172
173         total_duration += max(t_end_precise -
t_start_precise, 0.0)
174         in_shielded_region = False
175
176         # 处理最后一个区间
177         if in_shielded_region:
178             t_start_precise = self._bisection_find_boundary(
179                 ts[max(region_start - 1, 0)], ts[region_start
], cond, True, eps/10
180             )
181             total_duration += max(ts[-1] - t_start_precise,
0.0)
182
183         return total_duration
184
185     def _bisection_find_boundary(self, left, right, cond_func,
find_first_true, eps):
186         """二分法找边界"""
187         for _ in range(40):
188             mid = 0.5 * (left + right)
189             if cond_func(mid):
190                 if find_first_true:
191                     right = mid
192                 else:
193                     left = mid
194             else:
195                 if find_first_true:
196                     left = mid
197                 else:
198                     right = mid
199             if right - left < eps:
200                 break
201         return 0.5 * (left + right)

```



```

202
203     def _check_constraints(self, alpha, v_D, t_drop_list,
204 t_explode_list):
205         """检查约束条件"""
206         # 基本范围约束
207         if not (0 <= alpha <= 2*np.pi and 70 <= v_D <= 140):
208             return False
209
210         # 时序约束
211         for i in range(3):
212             if t_drop_list[i] > t_explode_list[i]:
213                 return False
214
215         # 时间间隔约束
216         if t_drop_list[1] - t_drop_list[0] < 1.0:
217             return False
218         if t_drop_list[2] - t_drop_list[1] < 1.0:
219             return False
220
221         return True
222
223     def _is_target_shielded_by_any_bomb(self, t, alpha, v_D,
224 t_drop_list, t_explode_list):
225         """判断t时刻目标圆柱体是否被多个烟幕弹完全遮蔽"""
226         return self._is_target_fully_shielded_multi_bomb(t,
227 alpha, v_D, t_drop_list, t_explode_list)
228
229     def _is_target_fully_shielded_multi_bomb(self, t, alpha,
230 v_D, t_drop_list, t_explode_list):
231         """判断t时刻目标圆柱体是否被多个烟幕弹完全遮蔽"""
232         sampling_points = self._get_cylinder_sampling_points()
233         P_missile = self._get_missile_position(t)
234
235         # 获取当前时刻有效的云团
236         active_smoke_centers = []
237         for i in range(len(t_drop_list)):
238             if t_explode_list[i] <= t <= t_explode_list[i] +
239 self.effective_duration:
240                 C_smoke = self._get_smoke_cloud_center(t,
241 alpha, v_D, t_drop_list[i], t_explode_list[i])
242                 if C_smoke is not None:
243                     active_smoke_centers.append(C_smoke)
244         if not active_smoke_centers:
245             return False

```

```

240
241     # 检查每个采样点是否被遮蔽
242     for point in sampling_points:
243         point_shielded = False
244         for C_smoke in active_smoke_centers:
245             if self._is_point_shielded_by_smoke(P_missile,
point, C_smoke):
246                 point_shielded = True
247                 break
248
249         # 如果任意一个点未被遮蔽，整个目标就未被完全遮蔽
250         if not point_shielded:
251             return False
252
253     return True
254
255     def _is_point_shielded_by_smoke(self, P_missile,
target_point, C_smoke):
256         """判断圆柱体上的一个目标点是否被单个烟幕云团遮蔽"""
257         if C_smoke is None:
258             return False
259         missile_to_smoke_distance = np.linalg.norm(P_missile -
C_smoke)
260         tolerance = 1e-6
261
262         if missile_to_smoke_distance < self.effective_radius -
tolerance:
263             # R < 10: 导弹在云团内，必然遮蔽目标点
264             return True
265         else:
266             V_mt = target_point - P_missile
267             V_length = np.linalg.norm(V_mt)
268
269             if V_length < 1e-12:
270                 # 导弹和目标点重合，直接判断导弹是否在云团内
271                 return np.linalg.norm(P_missile - C_smoke) <=
self.effective_radius
272
273             # 计算云团中心到导弹-目标点连线的距离
274             W = C_smoke - P_missile
275             cross = np.cross(V_mt, W)
276             cross_norm = np.linalg.norm(cross)
277             distance_to_line = cross_norm / V_length
278

```

```

279         # 如果距离大于有效半径, 则不遮挡
280         if distance_to_line > self.effective_radius:
281             return False
282
283         # 检查云团中心在导弹-目标点连线上的投影位置
284         dot_product = np.dot(W, V_mt)
285         t_proj = dot_product / (V_length ** 2)
286
287         # 如果投影不在连线段上, 检查导弹是否在烟幕内
288         if not (0.0 <= t_proj <= 1.0):
289             # 检查导弹是否在烟幕内
290             missile_in_smoke = np.linalg.norm(W) <= self.
effective_radius
291             return missile_in_smoke
292
293         return True
294
295     def calculate_single_shielding_durations(self, alpha, v_D,
t_drop_list, t_explode_list,
296                                           individual_bombs=None,
eps=0.05):
297         """单个烟幕弹遮蔽时长计算函数"""
298         if not self._check_constraints(alpha, v_D, t_drop_list
, t_explode_list):
299             return {
300                 'individual_durations': [0.0] * len(
t_drop_list),
301                 'total_duration': 0.0
302             }
303
304         if individual_bombs is None:
305             individual_bombs = list(range(len(t_drop_list)))
306
307         individual_durations = []
308         detailed_time_segments = []
309
310         for i in individual_bombs:
311             time_start = t_explode_list[i]
312             time_end = t_explode_list[i] + self.
effective_duration
313
314             def cond_single(t, bomb_idx=i):
315                 if t_explode_list[bomb_idx] <= t <=
t_explode_list[bomb_idx] + self.effective_duration:

```

```

316         single_bomb_t_drop = [t_drop_list[bomb_idx
]]
317         single_bomb_t_explode = [t_explode_list[
bomb_idx]]
318         return self.
_is_target_shielded_by_any_bomb(t, alpha, v_D,
single_bomb_t_drop, single_bomb_t_explode)
319         return False
320
321         ts = np.arange(time_start, time_end + eps, eps)
322         single_bomb_t_drop = [t_drop_list[i]]
323         single_bomb_t_explode = [t_explode_list[i]]
324         mask = self._is_target_shielded_by_any_bomb_vec(ts
, alpha, v_D, single_bomb_t_drop, single_bomb_t_explode)
325
326         bomb_segments = []
327
328         if not mask.any():
329             duration_i = 0.0
330         else:
331             duration_i = 0.0
332
333             in_shielded_region = False
334             region_start = 0
335
336             for j, is_shielded in enumerate(mask):
337                 if is_shielded and not in_shielded_region:
338                     region_start = j
339                     in_shielded_region = True
340                 elif not is_shielded and
in_shielded_region:
341                     region_end = j - 1
342
343                     t_start_precise = self.
_bisection_find_boundary(
344                         ts[max(region_start - 1, 0)], ts[
region_start], cond_single, True, eps/10
345                     )
346                     t_end_precise = self.
_bisection_find_boundary(
347                         ts[region_end], ts[min(region_end
+ 1, len(ts)-1)], cond_single, False, eps/10
348                     )
349

```

```

350         segment_duration = max(t_end_precise -
t_start_precise, 0.0)
351         duration_i += segment_duration
352
353         bomb_segments.append({
354             'start_time': round(
t_start_precise, 4),
355             'end_time': round(t_end_precise,
4),
356             'duration': round(segment_duration
, 4)
357         })
358
359         in_shielded_region = False
360
361         # 处理最后一个区间
362         if in_shielded_region:
363             t_start_precise = self.
_bisection_find_boundary(
364                 ts[max(region_start - 1, 0)], ts[
region_start], cond_single, True, eps/10
365             )
366             segment_duration = max(ts[-1] -
t_start_precise, 0.0)
367             duration_i += segment_duration
368
369             bomb_segments.append({
370                 'start_time': round(t_start_precise,
4),
371                 'end_time': round(ts[-1], 4),
372                 'duration': round(segment_duration, 4)
373             })
374
375             individual_durations.append(duration_i)
376             detailed_time_segments.append(bomb_segments)
377
378         # 计算总遮蔽时长
379         total_duration = self.
calculate_multi_shielding_duration(
380             alpha, v_D, t_drop_list, t_explode_list, eps
381         )
382
383         return {
384             'individual_durations': individual_durations,

```

```

385         'total_duration': total_duration,
386         'detailed_time_segments': detailed_time_segments
387     }
388
389 class MultiSmokeBombOptimizer:
390     """三枚烟幕弹协同投放策略的粒子群优化算法"""
391
392     _shared_calculator = None
393
394     def __init__(self, n_particles=50, n_iterations=100, w
395 =0.9, c1=2.0, c2=2.0):
396         """初始化粒子群优化器"""
397         self.n_particles = n_particles
398         self.n_iterations = n_iterations
399         self.w = w
400         self.c1 = c1
401         self.c2 = c2
402
403         self.calculator = MultiSmokeBombCalculator()
404
405         if MultiSmokeBombOptimizer._shared_calculator is None:
406             MultiSmokeBombOptimizer._shared_calculator =
MultiSmokeBombCalculator()
407
408         # 定义8维决策变量的边界
409         self.bounds = np.array([
410             [0, 2*np.pi],      # alpha
411             [70, 140],          # v_D
412             [0, 25.0],          # t'□
413             [0, 46.0],          # t''□
414             [1.0, 26.0],        # t'□
415             [1.0, 46.0],        # t''□
416             [2.0, 27.0],        # t'□
417             [2.0, 46.0]         # t''□
418         ])
419
420         self.dimension = len(self.bounds)
421
422         # 粒子群状态
423         self.particles = None
424         self.velocities = None
425         self.personal_best_positions = None
426         self.personal_best_scores = None
427         self.global_best_position = None

```

```

427         self.global_best_score = -np.inf
428
429         # 历史记录
430         self.history = {
431             'best_scores': [],
432             'mean_scores': [],
433             'best_positions': []
434         }
435
436         def gradient_objective_function(self, params):
437             """梯度化适应度函数：建立分层奖励机制"""
438             alpha, v_D, t_drop1, t_explode1, t_drop2, t_explode2,
439             t_drop3, t_explode3 = params
440
441             t_drop_list = [t_drop1, t_drop2, t_drop3]
442             t_explode_list = [t_explode1, t_explode2, t_explode3]
443
444             # 不可行解给予严重惩罚
445             if not self.calculator._check_constraints(alpha, v_D,
446             t_drop_list, t_explode_list):
447                 return -100.0
448             for i in range(3):
449                 if (t_explode_list[i] - t_drop_list[i]) > 19.1:
450                     return -100.0
451             if not (0 <= alpha <= 2*np.pi and 70 <= v_D <= 140):
452                 return -100.0
453
454             # 精确遮蔽奖励
455             duration = self.calculator.
456             calculate_multi_shielding_duration(
457                 alpha, v_D, t_drop_list, t_explode_list
458             )
459             if duration > 0:
460                 return duration
461
462             # 梯度化奖励
463             proximity_score = self.
464             calculate_multi_proximity_to_shielding(alpha, v_D,
465             t_drop_list, t_explode_list)
466             return -1.0 + 0.95 * proximity_score
467
468         def objective_function(self, params):
469             """目标函数：使用梯度化适应度函数"""
470             return self.gradient_objective_function(params)

```

```

466
467     def calculate_multi_proximity_to_shielding(self, alpha,
468 v_D, t_drop_list, t_explode_list):
469         """计算三枚烟幕弹接近遮蔽的程度，返回[0,1]"""
470         scores = []
471
472         # 对每枚烟幕弹分别计算接近度，然后取最大值
473         for i in range(3):
474             single_scores = []
475
476             # 检查几个关键时刻的接近程度
477             test_times = [t_explode_list[i] + j for j in [1,
478 3, 5, 8, 10, 15]]
479             test_times = [t for t in test_times if t <=
480 t_explode_list[i] + self.calculator.effective_duration]
481
482             if not test_times:
483                 continue
484
485             # 完全向量化计算
486             test_times = np.array(test_times)
487             P_missiles = self.calculator.
488 _get_missile_position_vec(test_times)
489             C_smoke_centers = self.calculator.
490 _get_smoke_cloud_center_vec(test_times, alpha, v_D,
491 t_drop_list[i], t_explode_list[i])
492
493             # 找出有效的烟幕位置
494             valid_mask = (~np.isnan(C_smoke_centers).any(axis
495 =1)) & (test_times <= t_explode_list[i] + self.calculator.
496 effective_duration)
497
498             if np.any(valid_mask):
499                 # 获取有效位置
500                 valid_P_missiles = P_missiles[valid_mask]
501                 valid_C_smoke_centers = C_smoke_centers[
502 valid_mask]
503                 valid_times = test_times[valid_mask]
504
505                 # 向量化计算导弹-烟幕距离
506                 distances = np.linalg.norm(valid_P_missiles -
507 valid_C_smoke_centers, axis=1)
508                 distance_scores = np.maximum(0, 1 - distances
509 / (3 * self.calculator.effective_radius))

```



```

499         # 向量化计算导弹-目标连线距离
500         line_distances = self.calculator.
501         _distance_point_to_line_vec(valid_P_missiles, self.
calculator.P_target, valid_C_smoke_centers)
502         line_scores = np.maximum(0, 1 - line_distances
/ (2 * self.calculator.effective_radius))
503
504         # 向量化计算时间窗口合理性
505         time_scores = np.where((2 <= t_explode_list[i
]) & (t_explode_list[i] <= 12), 1.0, 0.3)
506         time_scores = np.full(len(valid_times),
time_scores) # 广播到每个时间点
507
508         # 向量化计算综合得分
509         combined_scores = 0.4 * distance_scores + 0.4
* line_scores + 0.2 * time_scores
510         single_scores.extend(combined_scores.tolist())
511
512         if single_scores:
513             scores.append(max(single_scores))
514
515         # 协同效应：取所有烟幕弹中的最高分，并给予小幅协同奖励
516         if not scores:
517             return 0.0
518
519         max_score = max(scores)
520         # 多个烟幕弹都有不错的分数，给予协同奖励
521         good_bombs = sum(1 for s in scores if s > 0.5)
522         collaboration_bonus = min(0.1, good_bombs * 0.03)
523
524         return min(1.0, max_score + collaboration_bonus)
525
526     @staticmethod
527     def static_objective_function(params):
528         """静态目标函数：用于并行计算"""
529         alpha, v_D, t_drop1, t_explode1, t_drop2, t_explode2,
t_drop3, t_explode3 = params
530
531         t_drop_list = [t_drop1, t_drop2, t_drop3]
532         t_explode_list = [t_explode1, t_explode2, t_explode3]
533
534         # 使用共享计算器
535         if MultiSmokeBombOptimizer._shared_calculator is None:

```

```

536         MultiSmokeBombOptimizer._shared_calculator =
MultiSmokeBombCalculator()
537         calculator = MultiSmokeBombOptimizer.
_shared_calculator
538
539         # 不可行解给予严重惩罚
540         if not calculator._check_constraints(alpha, v_D,
t_drop_list, t_explode_list):
541             return -100.0
542         for i in range(3):
543             if (t_explode_list[i] - t_drop_list[i]) > 19.1:
544                 return -100.0
545         if not (0 <= alpha <= 2*np.pi and 70 <= v_D <= 140):
546             return -100.0
547
548         # 精确遮蔽奖励
549         duration = calculator.
calculate_multi_shielding_duration(
550             alpha, v_D, t_drop_list, t_explode_list
551         )
552         if duration > 0:
553             return duration
554
555         # 梯度化奖励
556         proximity_score = MultiSmokeBombOptimizer.
_static_calculate_multi_proximity_to_shielding(
557             alpha, v_D, t_drop_list, t_explode_list
558         )
559         return -1.0 + 0.95 * proximity_score
560
561     @staticmethod
562     def _static_calculate_multi_proximity_to_shielding(alpha,
v_D, t_drop_list, t_explode_list):
563         """静态版本的多烟幕弹接近度计算函数，用于并行计算"""
564         scores = []
565         if MultiSmokeBombOptimizer._shared_calculator is None:
566             MultiSmokeBombOptimizer._shared_calculator =
MultiSmokeBombCalculator()
567             calculator = MultiSmokeBombOptimizer.
_shared_calculator
568
569         # 对每枚烟幕弹分别计算接近度
570         for i in range(3):
571             single_scores = []

```

```

572
573     # 检查几个关键时刻的接近程度
574     test_times = [t_explode_list[i] + j for j in [1,
3, 5, 8, 10, 15]]
575     test_times = [t for t in test_times if t <=
t_explode_list[i] + calculator.effective_duration]
576
577     if not test_times:
578         continue
579
580     # 完全向量化计算
581     test_times = np.array(test_times)
582     # 使用共享计算器的向量化函数
583     P_missiles = calculator._get_missile_position_vec(
test_times)
584     C_smoke_centers = calculator.
_get_smoke_cloud_center_vec(test_times, alpha, v_D,
t_drop_list[i], t_explode_list[i])
585
586     # 找出有效的烟幕位置
587     valid_mask = (~np.isnan(C_smoke_centers).any(axis
=1)) & (test_times <= t_explode_list[i] + calculator.
effective_duration)
588
589     if np.any(valid_mask):
590         # 获取有效位置
591         valid_P_missiles = P_missiles[valid_mask]
592         valid_C_smoke_centers = C_smoke_centers[
valid_mask]
593         valid_times = test_times[valid_mask]
594
595         # 向量化计算导弹-烟幕距离
596         distances = np.linalg.norm(valid_P_missiles -
valid_C_smoke_centers, axis=1)
597         distance_scores = np.maximum(0, 1 - distances
/ (3 * calculator.effective_radius))
598
599         # 向量化计算导弹-目标连线距离
600         line_distances = calculator.
_distance_point_to_line_vec(valid_P_missiles, calculator.
P_target, valid_C_smoke_centers)
601         line_scores = np.maximum(0, 1 - line_distances
/ (2 * calculator.effective_radius))
602

```

```

603         # 向量化计算时间窗口合理性
604         time_scores = np.where((2 <= t_explode_list[i
]) & (t_explode_list[i] <= 12), 1.0, 0.3)
605         time_scores = np.full(len(valid_times),
time_scores) # 广播到每个时间点
606
607         # 向量化计算综合得分
608         combined_scores = 0.4 * distance_scores + 0.4
* line_scores + 0.2 * time_scores
609         single_scores.extend(combined_scores.tolist())
610
611         if single_scores:
612             scores.append(max(single_scores))
613
614     # 协同奖励
615     if not scores:
616         return 0.0
617
618     max_score = max(scores)
619     good_bombs = sum(1 for s in scores if s > 0.5)
620     collaboration_bonus = min(0.1, good_bombs * 0.03)
621
622     return min(1.0, max_score + collaboration_bonus)
623
624     def _repair_particle(self, particle):
625         """约束修复"""
626         alpha, v_D, t_drop1, t_explode1, t_drop2, t_explode2,
t_drop3, t_explode3 = particle
627
628         # 修复时间间隔约束
629         if t_drop2 - t_drop1 < 1.0:
630             t_drop2 = t_drop1 + 1.0
631         if t_drop3 - t_drop2 < 1.0:
632             t_drop3 = t_drop2 + 1.0
633
634         # 修复时序约束
635         if t_drop1 > t_explode1:
636             t_explode1, t_drop1 = t_drop1, t_explode1
637         if t_drop2 > t_explode2:
638             t_explode2, t_drop2 = t_drop2, t_explode2
639         if t_drop3 > t_explode3:
640             t_explode3, t_drop3 = t_drop3, t_explode3
641
642         # 修复时间差约束

```

```

643         for i, (td, te) in enumerate([(t_drop1, t_explode1), (
t_drop2, t_explode2), (t_drop3, t_explode3)]):
644             if (te - td) > 19.2:
645                 if i == 0:
646                     t_explode1 = t_drop1 + 19.2
647                 elif i == 1:
648                     t_explode2 = t_drop2 + 19.2
649                 else:
650                     t_explode3 = t_drop3 + 19.2
651
652             # 修复边界约束
653             repaired = np.array([alpha, v_D, t_drop1, t_explode1,
t_drop2, t_explode2, t_drop3, t_explode3])
654             for j in range(self.dimension):
655                 repaired[j] = np.clip(repaired[j], self.bounds[j,
0], self.bounds[j, 1])
656
657             # 重新检查修复后是否仍满足约束
658             _, _, td1, te1, td2, te2, td3, te3 = repaired
659
660             # 如果边界截断破坏了约束，进行最终调整
661             if td2 - td1 < 1.0:
662                 td2 = min(td1 + 1.0, self.bounds[4, 1])
663             if td3 - td2 < 1.0:
664                 td3 = min(td2 + 1.0, self.bounds[6, 1])
665
666             repaired[2:] = [td1, te1, td2, te2, td3, te3]
667
668             return repaired
669
670     def initialize_particles(self):
671         """初始化粒子群"""
672         debug_print("初始化粒子群...")
673
674         self.particles = np.zeros((self.n_particles, self.
dimension))
675         self.velocities = np.zeros((self.n_particles, self.
dimension))
676         self.personal_best_positions = np.zeros((self.
n_particles, self.dimension))
677         self.personal_best_scores = np.full(self.n_particles,
-np.inf)
678
679         # 随机初始化粒子位置

```

```

680         for i in range(self.n_particles):
681             for j in range(self.dimension):
682                 min_val, max_val = self.bounds[j]
683                 self.particles[i, j] = random.uniform(min_val,
max_val)
684
685             # 修复约束
686             self.particles[i] = self._repair_particle(self.
particles[i])
687
688             # 根据粒子位置到边界的距离自适应调整初始速度
689             for j in range(self.dimension):
690                 range_val = self.bounds[j, 1] - self.bounds[j,
0]
691                 self.velocities[i, j] = random.uniform(-0.1 *
range_val, 0.1 * range_val)
692
693             # 并行计算初始适应度
694             with concurrent.futures.ProcessPoolExecutor() as pool:
695                 scores = list(pool.map(MultiSmokeBombOptimizer.
static_objective_function, self.particles.tolist()))
696             for i, score in enumerate(scores):
697                 self.personal_best_scores[i] = score
698                 self.personal_best_positions[i] = self.particles[i
].copy()
699                 if score > self.global_best_score:
700                     self.global_best_score = score
701                     self.global_best_position = self.particles[i].
copy()
702
703             debug_print(f"初始化完成，最佳初始适应度：{self.
global_best_score:.6f}")
704
705         def update_particles(self):
706             """更新粒子位置和速度"""
707             for i in range(self.n_particles):
708                 # 更新速度
709                 r1, r2 = random.random(), random.random()
710
711                 cognitive_component = self.c1 * r1 * (self.
personal_best_positions[i] - self.particles[i])
712                 social_component = self.c2 * r2 * (self.
global_best_position - self.particles[i])
713

```

```

714         self.velocities[i] = (self.w * self.velocities[i]
715 +
716                                cognitive_component +
717                                social_component)
718
719         # 限制速度大小
720         for j in range(self.dimension):
721             range_val = self.bounds[j, 1] - self.bounds[j,
722 0]
723             max_velocity = 0.2 * range_val
724             self.velocities[i, j] = np.clip(self.
725 velocities[i, j], -max_velocity, max_velocity)
726
727         # 更新位置
728         self.particles[i] += self.velocities[i]
729
730         # 边界约束
731         for j in range(self.dimension):
732             self.particles[i, j] = np.clip(self.particles[
733 i, j],
734                                           self.bounds[j,
735 0], self.bounds[j, 1])
736
737         # 修复约束
738         self.particles[i] = self._repair_particle(self.
739 particles[i])
740
741     def evaluate_and_update_best(self):
742         """评估粒子适应度并更新最佳位置"""
743         with concurrent.futures.ProcessPoolExecutor() as pool:
744             scores = list(pool.map(MultiSmokeBombOptimizer.
745 static_objective_function, self.particles.tolist()))
746
747         for i, score in enumerate(scores):
748             if score > self.personal_best_scores[i]:
749                 self.personal_best_scores[i] = score
750                 self.personal_best_positions[i] = self.
751 particles[i].copy()
752             if score > self.global_best_score:
753                 self.global_best_score = score
754                 self.global_best_position = self.particles[i].
755 copy()
756
757         # 记录历史

```

```

748         self.history['best_scores'].append(self.
global_best_score)
749         self.history['mean_scores'].append(np.mean(scores))
750         self.history['best_positions'].append(self.
global_best_position.copy())
751
752     def optimize(self):
753         """执行粒子群优化"""
754         debug_print("开始三枚烟幕弹协同优化...")
755
756         # 初始化
757         self.initialize_particles()
758
759         # 迭代优化
760         for iteration in range(self.n_iterations):
761             # 更新粒子
762             self.update_particles()
763
764             # 评估适应度
765             self.evaluate_and_update_best()
766
767             # 动态调整惯性权重
768             self.w = 0.9 - 0.5 * iteration / self.n_iterations
769
770             # 输出进度
771             if (iteration + 1) % 5 == 0 or iteration == 0:
772                 print(f"[Iter {iteration + 1:3d}]/{self.
n_iterations}] 最佳={self.global_best_score:.6f} 平均={
self.history['mean_scores'][-1]:.6f}")
773
774                 print(f"优化结束：全局最佳适应度 = {self.
global_best_score:.6f}")
775
776                 return self.get_optimization_result()
777
778     def get_optimization_result(self):
779         """获取优化结果"""
780         alpha_opt, v_D_opt, t_drop1, t_explode1, t_drop2,
t_explode2, t_drop3, t_explode3 = self.global_best_position
781
782         # 计算详细的遮蔽信息
783         t_drop_list = [t_drop1, t_drop2, t_drop3]
784         t_explode_list = [t_explode1, t_explode2, t_explode3]
785

```



```

786         detailed_info = self.calculator.
calculate_single_shielding_durations(
787             alpha_opt, v_D_opt, t_drop_list, t_explode_list
788         )
789
790         result = {
791             'optimal_parameters': {
792                 'alpha_degrees': np.degrees(alpha_opt),
793                 'alpha_radians': alpha_opt,
794                 'v_D': v_D_opt,
795                 'smoke_bomb_1': {'t_drop': t_drop1, 't_explode
': t_explode1},
796                 'smoke_bomb_2': {'t_drop': t_drop2, 't_explode
': t_explode2},
797                 'smoke_bomb_3': {'t_drop': t_drop3, 't_explode
': t_explode3}
798             },
799             'optimal_total_shielding_duration': self.
global_best_score,
800             'individual_shielding_durations': detailed_info['
individual_durations'],
801             'optimization_history': self.history
802         }
803
804         return result
805
806 # 辅助函数
807 def debug_print(*args, **kwargs):
808     """调试打印函数，根据VERBOSE控制是否输出"""
809     if VERBOSE:
810         print(*args, **kwargs)
811
812 def save_q3_results(result_dict, filename="q3_results.xlsx"):
813     """将问题三的优化结果保存到Excel文件"""
814     opt_params = result_dict["optimal_parameters"]
815     alpha = opt_params["alpha_radians"]
816     v_D = opt_params["v_D"]
817
818     calculator = MultiSmokeBombCalculator()
819
820     t_drop_list = [opt_params[f"smoke_bomb_{i+1}"]["t_drop"]
for i in range(3)]
821     t_explode_list = [opt_params[f"smoke_bomb_{i+1}"]["
t_explode"] for i in range(3)]

```

```

822
823     detailed_info = calculator.
calculate_single_shielding_durations(
824         alpha, v_D, t_drop_list, t_explode_list
825     )
826
827     # 创建问题三结果数据表
828     results_data = []
829
830     for i in range(3):
831         bomb_key = f"smoke_bomb_{i+1}"
832         t_drop = opt_params[bomb_key]["t_drop"]
833         t_explode = opt_params[bomb_key]["t_explode"]
834         drop_position = calculator._get_drone_position(t_drop,
alpha, v_D, t_drop)
835         explode_position = calculator._get_smoke_bomb_position
(t_explode, alpha, v_D, t_drop, t_explode)
836
837         # 获取每枚烟幕弹的单独遮蔽时长
838         individual_duration = detailed_info["
individual_durations"][i]
839
840         row_data = {
841             "无人机运动方向": f"{opt_params['alpha_degrees'
]:.2f}°",
842             "无人机运动速度 (m/s)": opt_params["v_D"],
843             "烟幕干扰弹编号": i + 1,
844             "烟幕干扰弹投放点的x坐标 (m)": round(drop_position
[0], 2),
845             "烟幕干扰弹投放点的y坐标 (m)": round(drop_position
[1], 2),
846             "烟幕干扰弹投放点的z坐标 (m)": round(drop_position
[2], 2),
847             "烟幕干扰弹起爆点的x坐标 (m)": round(
explode_position[0], 2),
848             "烟幕干扰弹起爆点的y坐标 (m)": round(
explode_position[1], 2),
849             "烟幕干扰弹起爆点的z坐标 (m)": round(
explode_position[2], 2),
850             "有效干扰时长 (s)": round(individual_duration, 4)
851         }
852         results_data.append(row_data)
853
854     df_results = pd.DataFrame(results_data)

```

```

855
856 # 创建详细时间段数据表
857 time_segments_data = []
858
859 for i in range(3):
860     bomb_segments = detailed_info["detailed_time_segments
861     ][i]
862
863     if bomb_segments: # 如果有遮蔽时间段
864         for segment_idx, segment in enumerate(
865             bomb_segments):
866             time_segments_data.append({
867                 "烟幕弹编号": i + 1,
868                 "时间段序号": segment_idx + 1,
869                 "遮蔽开始时间 (s)": segment["start_time"],
870                 "遮蔽结束时间 (s)": segment["end_time"],
871                 "时间段时长 (s)": segment["duration"]
872             })
873     else: # 如果没有遮蔽时间段
874         time_segments_data.append({
875             "烟幕弹编号": i + 1,
876             "时间段序号": 0,
877             "遮蔽开始时间 (s)": 0,
878             "遮蔽结束时间 (s)": 0,
879             "时间段时长 (s)": 0
880         })
881
882 df_time_segments = pd.DataFrame(time_segments_data)
883
884 # 创建汇总信息数据表
885 summary_data = [{
886     "无人机运动方向": f"{opt_params['alpha_degrees']:.2f}°",
887     "无人机运动速度 (m/s)": opt_params["v_D"],
888     "烟幕弹1单独遮蔽时长 (s)": round(detailed_info["
889     individual_durations"][0], 4),
890     "烟幕弹2单独遮蔽时长 (s)": round(detailed_info["
891     individual_durations"][1], 4),
892     "烟幕弹3单独遮蔽时长 (s)": round(detailed_info["
893     individual_durations"][2], 4),
894     "总有效干扰时长 (s)": round(detailed_info["
895     total_duration"], 4)
896 }]
```

```

892     df_summary = pd.DataFrame(summary_data)
893
894     # 创建优化历史数据表
895     hist = result_dict["optimization_history"]
896     df_history = pd.DataFrame({
897         "iteration": range(1, len(hist["best_scores"]) + 1),
898         "best_score": hist["best_scores"],
899         "mean_score": hist["mean_scores"]
900     })
901
902     with pd.ExcelWriter(filename, engine="openpyxl") as writer
903     :
904         df_results.to_excel(writer, sheet_name="问题三结果",
905         index=False)
906         df_time_segments.to_excel(writer, sheet_name="详细时间
907         段", index=False)
908         df_summary.to_excel(writer, sheet_name="遮蔽时长汇总",
909         index=False)
910         df_history.to_excel(writer, sheet_name="优化历史",
911         index=False)
912
913     print(f"已保存问题三优化结果到 {filename}")
914
915 # 主函数
916 def run_optimization_problem3():
917     """运行问题三的优化求解"""
918     print("=" * 60)
919     print("开始求解问题三：三枚烟幕干扰弹协同投放策略优化")
920     print("=" * 60)
921
922     optimizer = MultiSmokeBombOptimizer(
923         n_particles=200,
924         n_iterations=50,
925         w=0.9,
926         c1=2.8,
927         c2=1.3
928     )
929     result = optimizer.optimize()
930
931     print("\n" + "=" * 60)
932     print("问题三优化结果")
933     print("=" * 60)
934
935     opt_params = result['optimal_parameters']

```

```

931     print(f"最优参数:")
932     print(f"    航向角  $\alpha$  = {opt_params['alpha_degrees']:.2f}° ({opt_params['alpha_radians']:.4f} 弧度)")
933     print(f"    无人机速度  $v_D$  = {opt_params['v_D']:.2f} m/s")
934     print(f"")
935     print(f"    烟幕弹 1: 投放时刻  $t'_{\square}$  = {opt_params['smoke_bomb_1']['t_drop']:.3f} s, 起爆时刻  $t'_{\square}$  = {opt_params['smoke_bomb_1']['t_explode']:.3f} s")
936     print(f"    烟幕弹 2: 投放时刻  $t'_{\square}$  = {opt_params['smoke_bomb_2']['t_drop']:.3f} s, 起爆时刻  $t'_{\square}$  = {opt_params['smoke_bomb_2']['t_explode']:.3f} s")
937     print(f"    烟幕弹 3: 投放时刻  $t'_{\square}$  = {opt_params['smoke_bomb_3']['t_drop']:.3f} s, 起爆时刻  $t'_{\square}$  = {opt_params['smoke_bomb_3']['t_explode']:.3f} s")
938     print(f"")
939     print(f"    遮蔽时长详情:")
940     individual_durations = result['individual_shielding_durations']
941     print(f"    烟幕弹 1 单独遮蔽时长: {individual_durations[0]:.6f} 秒")
942     print(f"    烟幕弹 2 单独遮蔽时长: {individual_durations[1]:.6f} 秒")
943     print(f"    烟幕弹 3 单独遮蔽时长: {individual_durations[2]:.6f} 秒")
944     print(f"    总遮蔽时长: {result['optimal_total_shielding_duration']:.6f} 秒")
945
946     print(f"\n约束条件验证:")
947     t1_drop = opt_params['smoke_bomb_1']['t_drop']
948     t2_drop = opt_params['smoke_bomb_2']['t_drop']
949     t3_drop = opt_params['smoke_bomb_3']['t_drop']
950     print(f"    时间间隔  $t'_{\square} - t'_{\square}$  = {t2_drop - t1_drop:.3f} s ( $\geq 1.0$ )")
951     print(f"    时间间隔  $t'_{\square} - t'_{\square}$  = {t3_drop - t2_drop:.3f} s ( $\geq 1.0$ )")
952
953     save_q3_results(result, "q3_results.xlsx")
954     return result
955
956 if __name__ == "__main__":
957     result = run_optimization_problem3()

```

q4.py

```
1 import numpy as np
```

```

2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import random
5 import concurrent.futures
6 from q2 import SmokeShieldingCalculator
7
8 VERBOSE = True
9
10 class MultiDroneCalculator(SmokeShieldingCalculator):
11     """三架无人机协同单烟幕弹遮蔽计算器"""
12
13     # 为并行静态方法提供共享计算器实例
14     _shared_calculator = None
15
16     def __init__(self, **kwargs):
17         """初始化多无人机计算器"""
18         super().__init__(**kwargs)
19
20         # 三架无人机的初始位置
21         self.P_FY1_0 = np.array([17800, 0, 1800]) # FY1
22         self.P_FY2_0 = np.array([12000, 1400, 1400]) # FY2
23         self.P_FY3_0 = np.array([6000, -3000, 700]) # FY3
24
25         self.drone_initial_positions = [self.P_FY1_0, self.
P_FY2_0, self.P_FY3_0]
26
27     def _get_missile_position_vec(self, ts):
28         """向量化计算导弹在多个时刻的位置"""
29         ts = np.asarray(ts)
30         positions = self.P_M1_0[np.newaxis, :] + self.v_M1[np.
newaxis, :] * ts[:, np.newaxis]
31         return positions
32
33     def _get_drone_position_vec(self, ts, drone_idx, alpha,
v_D):
34         """向量化计算无人机在多个时刻的位置"""
35         ts = np.asarray(ts)
36         e_alpha = self._get_unit_direction_vector(alpha)
37         positions = self.drone_initial_positions[drone_idx][np.
newaxis, :] + v_D * ts[:, np.newaxis] * e_alpha[np.newaxis,
:]
38         return positions
39
40     def _get_smoke_cloud_center_vec(self, ts, drone_idx, alpha

```

```

, v_D, t_drop, t_explode):
41     """向量化计算烟幕云团在多个时刻的中心位置"""
42     ts = np.asarray(ts)
43     N = len(ts)
44     centers = np.full((N, 3), np.nan)
45     valid_mask = (ts >= t_explode) & (ts <= t_explode +
self.effective_duration)
46     if not np.any(valid_mask):
47         return centers
48
49     # 只对有效时间进行计算
50     valid_ts = ts[valid_mask]
51     valid_count = len(valid_ts)
52
53     # 计算方向向量
54     e_alpha = self._get_unit_direction_vector(alpha)
55
56     # 计算起爆位置 B
57     dt_fall = t_explode - t_drop
58     R = self.drone_initial_positions[drone_idx] + v_D *
t_drop * e_alpha
59     B = R + v_D * dt_fall * e_alpha + np.array([0, 0, -0.5
* self.g * dt_fall**2])
60     if B[2] < 0:
61         B[2] = 0
62
63     # 计算云团下沉时间
64     dt_sink = valid_ts - t_explode
65
66     # 向量化计算云团位置
67     valid_centers = np.tile(B, (valid_count, 1))
68     valid_centers[:, 2] -= self.smoke_fall_speed * dt_sink
69
70     valid_centers[:, 2] = np.maximum(valid_centers[:, 2],
0)
71
72     centers[valid_mask] = valid_centers
73
74     return centers
75
76     def _distance_point_to_line_vec(self, P_missile_array,
P_target, C_smoke_array):
77         """向量化计算多个烟幕中心到导弹-目标连线的距离"""
78

```

```

79         # 导弹到目标
80         vec_missile_to_target = P_target[np.newaxis, :] -
P_missile_array
81
82         # 导弹到烟幕
83         vec_missile_to_smoke = C_smoke_array - P_missile_array
84
85         # 目标距离
86         missile_target_distances = np.linalg.norm(
vec_missile_to_target, axis=1)
87
88         # 处理距离过小的情况
89         small_distance_mask = missile_target_distances < 1e-10
90
91         # 计算叉积
92         cross_products = np.cross(vec_missile_to_target,
vec_missile_to_smoke)
93         cross_product_norms = np.linalg.norm(cross_products,
axis=1)
94
95         # 计算距离
96         distances = np.zeros(len(P_missile_array))
97
98         normal_mask = ~small_distance_mask
99         distances[normal_mask] = cross_product_norms[
normal_mask] / missile_target_distances[normal_mask]
100
101         # 导弹到目标距离很小时，直接返回导弹到烟幕的距离
102         distances[small_distance_mask] = np.linalg.norm(
vec_missile_to_smoke[small_distance_mask], axis=1)
103
104         return distances
105
106     def _get_drone_position(self, t, drone_idx, alpha, v_D,
t_drop):
107         """计算t时刻第drone_idx架无人机位置"""
108         e_alpha = self._get_unit_direction_vector(alpha)
109         return self.drone_initial_positions[drone_idx] + v_D *
t * e_alpha
110
111     def _get_smoke_bomb_position(self, t, drone_idx, alpha,
v_D, t_drop, t_explode):
112         """计算t时刻第drone_idx架无人机投放的烟幕弹位置"""
113         if t < t_drop:

```



```

114         return None
115     elif t <= t_explode:
116         # 自由落体阶段
117         e_alpha = self._get_unit_direction_vector(alpha)
118         R = self.drone_initial_positions[drone_idx] + v_D
119         * t_drop * e_alpha
120         dt = t - t_drop
121         bomb_position = R + v_D * dt * e_alpha + np.array
122         ([0, 0, -0.5 * self.g * dt**2])
123
124         # 确保烟幕弹高度不低于地面
125         if bomb_position[2] < 0:
126             bomb_position[2] = 0
127
128         return bomb_position
129     else:
130         return None
131
132     def _get_smoke_cloud_center(self, t, drone_idx, alpha, v_D
133     , t_drop, t_explode):
134         """计算t时刻第drone_idx架无人机烟幕云团中心位置"""
135         if t < t_explode:
136             return None
137         elif t <= t_explode + self.effective_duration:
138             # 计算起爆位置
139             e_alpha = self._get_unit_direction_vector(alpha)
140             R = self.drone_initial_positions[drone_idx] + v_D
141             * t_drop * e_alpha
142             dt_fall = t_explode - t_drop
143             B = R + v_D * dt_fall * e_alpha + np.array([0, 0,
144             -0.5 * self.g * dt_fall**2])
145
146             # 确保起爆位置不低于地面
147             if B[2] < 0:
148                 B[2] = 0
149
150             # 云团下沉
151             dt_sink = t - t_explode
152             smoke_center = B + np.array([0, 0, -self.
153             smoke_fall_speed * dt_sink])
154
155             # 确保云团高度不低于地面
156             if smoke_center[2] < 0:
157                 smoke_center[2] = 0

```

```

152
153         return smoke_center
154     else:
155         return None
156
157     def _check_constraints_multi_drone(self, alpha_list,
v_D_list, t_drop_list, t_explode_list):
158         """检查多无人机约束条件"""
159         # 基本范围约束
160         for i in range(3):
161             if not (0 <= alpha_list[i] <= 2*np.pi and 70 <=
v_D_list[i] <= 140):
162                 return False
163
164             # 时序约束
165             if t_drop_list[i] > t_explode_list[i]:
166                 return False
167
168             # # 时间差约束（假设与单无人机类似）
169             # if (t_explode_list[i] - t_drop_list[i]) > 19.1:
170             #     return False
171
172         return True
173
174     def _get_cylinder_sampling_points(self, theta_steps=24,
z_steps=10, r_steps=4):
175         """生成圆柱体表面采样点"""
176         points = []
177
178         # 侧面网格采样
179         theta_values = np.linspace(0, 2*np.pi, theta_steps,
endpoint=False)
180         z_values = np.linspace(self.C_base[2], self.C_top[2],
z_steps)
181         for theta in theta_values:
182             for z in z_values:
183                 x = self.R_cyl * np.cos(theta)
184                 y = self.C_base[1] + self.R_cyl * np.sin(theta
)
185                 points.append([x, y, z])
186
187         # 顶面极坐标采样
188         theta_values_top = np.linspace(0, 2*np.pi, theta_steps
, endpoint=False)

```

```

189         r_values = np.linspace(0, self.R_cyl, r_steps,
endpoint=False)
190         for r in r_values:
191             if r == 0: # 中心点
192                 points.append([0, self.C_top[1], self.C_top
[2]])
193             else:
194                 for theta in theta_values_top:
195                     x = r * np.cos(theta)
196                     y = self.C_top[1] + r * np.sin(theta)
197                     points.append([x, y, self.C_top[2]])
198         return np.array(points)
199
200     def _is_target_fully_shielded_multi_drone(self, t,
alpha_list, v_D_list, t_drop_list, t_explode_list):
201         """判断t时刻目标圆柱体是否被多架无人机的烟幕弹完全遮蔽
"""
202         sampling_points = self._get_cylinder_sampling_points()
203         P_missile = self._get_missile_position(t)
204
205         # 获取当前时刻有效的云团
206         active_smoke_centers = []
207         for drone_idx in range(3):
208             if t_explode_list[drone_idx] <= t <=
t_explode_list[drone_idx] + self.effective_duration:
209                 C_smoke = self._get_smoke_cloud_center(t,
drone_idx, alpha_list[drone_idx],
210                                                         v_D_list[
drone_idx], t_drop_list[drone_idx],
211 t_explode_list[drone_idx])
212                 if C_smoke is not None:
213                     active_smoke_centers.append(C_smoke)
214
215         if not active_smoke_centers:
216             return False
217
218         # 检查每个采样点是否被遮蔽
219         for point in sampling_points:
220             point_shielded = False
221             for C_smoke in active_smoke_centers:
222                 if self._is_point_shielded_by_smoke(P_missile,
point, C_smoke):
223                     point_shielded = True

```

```

224         break
225
226     # 如果任意一个点未被遮蔽，整个目标就未被完全遮蔽
227     if not point_shielded:
228         return False
229
230     return True
231
232     def _is_point_shielded_by_smoke(self, P_missile,
target_point, C_smoke):
233         """判断圆柱体上的一个目标点是否被单个烟幕云团遮蔽"""
234         if C_smoke is None:
235             return False
236         missile_to_smoke_distance = np.linalg.norm(P_missile -
C_smoke)
237         tolerance = 1e-6
238
239         if missile_to_smoke_distance < self.effective_radius -
tolerance:
240             # R < 10: 导弹在云团内，必然遮蔽目标点
241             return True
242         else:
243             V_mt = target_point - P_missile
244             V_length = np.linalg.norm(V_mt)
245
246             if V_length < 1e-12:
247                 # 导弹和目标点重合，直接判断导弹是否在云团内
248                 return np.linalg.norm(P_missile - C_smoke) <=
self.effective_radius
249
250             # 计算云团中心到导弹-目标点连线的距离
251             W = C_smoke - P_missile
252             cross = np.cross(V_mt, W)
253             cross_norm = np.linalg.norm(cross)
254             distance_to_line = cross_norm / V_length
255
256             # 如果距离大于有效半径，则不遮挡
257             if distance_to_line > self.effective_radius:
258                 return False
259
260             # 检查云团中心在导弹-目标点连线上的投影位置
261             dot_product = np.dot(W, V_mt)
262             t_proj = dot_product / (V_length ** 2)
263

```

```

264         # 如果投影不在连线段上, 检查导弹是否在烟幕内
265         if not (0.0 <= t_proj <= 1.0):
266             # 检查导弹是否在烟幕内
267             missile_in_smoke = np.linalg.norm(W) <= self.
effective_radius
268             return missile_in_smoke
269
270         return True
271
272     def get_drone_info(self):
273         """获取多无人机场景信息"""
274         info = super().get_scenario_info()
275         info.update({
276             'FY1初始位置': self.P_FY1_0.tolist(),
277             'FY2初始位置': self.P_FY2_0.tolist(),
278             'FY3初始位置': self.P_FY3_0.tolist(),
279             '无人机数量': 3,
280             '决策变量维度': 12
281         })
282         return info
283
284     def _is_target_shielded_by_any_drone_vec(self, ts,
alpha_list, v_D_list, t_drop_list, t_explode_list):
285         """向量化判断多个时刻目标圆柱体是否被多架无人机的烟幕
弹完全遮蔽"""
286         ts = np.asarray(ts)
287         N = len(ts)
288
289         # 初始化结果数组
290         shielded_mask = np.zeros(N, dtype=bool)
291
292         # 对每个时刻进行判定
293         for i, t in enumerate(ts):
294             shielded_mask[i] = self.
_is_target_fully_shielded_multi_drone(t, alpha_list,
v_D_list, t_drop_list, t_explode_list)
295
296         return shielded_mask
297
298     def _is_target_shielded_by_any_drone(self, t, alpha_list,
v_D_list, t_drop_list, t_explode_list):
299         """判断t时刻目标圆柱体是否被多架无人机的烟幕弹遮蔽"""
300         return self._is_target_fully_shielded_multi_drone(t,
alpha_list, v_D_list, t_drop_list, t_explode_list)

```

```

301
302     def calculate_multi_drone_shielding_duration(self,
alpha_list, v_D_list, t_drop_list, t_explode_list, eps=0.05)
:
303         """计算三架无人机协同作用下的总遮蔽时长"""
304         if not self._check_constraints_multi_drone(alpha_list,
v_D_list, t_drop_list, t_explode_list):
305             return 0.0
306
307         earliest_start = min(t_explode_list)
308         latest_end = max(t_explode_list) + self.
effective_duration
309
310         # 向量化扫描找出所有遮蔽时刻
311         ts = np.arange(earliest_start, latest_end + eps, eps)
312         mask = self._is_target_shielded_by_any_drone_vec(ts,
alpha_list, v_D_list, t_drop_list, t_explode_list)
313
314         if not mask.any():
315             return 0.0
316
317         # 为二分法定义条件函数
318         def cond(t):
319             """检查t时刻是否被任意烟幕弹遮蔽"""
320             return self._is_target_shielded_by_any_drone(t,
alpha_list, v_D_list, t_drop_list, t_explode_list)
321
322         # 计算所有遮蔽区间的总时长
323         total_duration = 0.0
324
325         # 找到所有连续的True区间
326         in_shielded_region = False
327         region_start = 0
328
329         for i, is_shielded in enumerate(mask):
330             if is_shielded and not in_shielded_region:
331                 # 开始一个新的遮蔽区间
332                 region_start = i
333                 in_shielded_region = True
334             elif not is_shielded and in_shielded_region:
335                 # 结束当前遮蔽区间
336                 region_end = i - 1
337
338                 # 二分法计算区间边界

```

```

339         t_start_precise = self.
_bisection_find_boundary(
340             ts[max(region_start - 1, 0)], ts[
region_start], cond, True, eps/10
341         )
342         t_end_precise = self._bisection_find_boundary(
343             ts[region_end], ts[min(region_end + 1, len
(ts)-1)], cond, False, eps/10
344         )
345
346         total_duration += max(t_end_precise -
t_start_precise, 0.0)
347         in_shielded_region = False
348
349         # 处理最后一个区间
350         if in_shielded_region:
351             t_start_precise = self._bisection_find_boundary(
352                 ts[max(region_start - 1, 0)], ts[region_start
], cond, True, eps/10
353             )
354             total_duration += max(ts[-1] - t_start_precise,
0.0)
355
356         return total_duration
357
358     def _bisection_find_boundary(self, left, right, cond_func,
find_first_true, eps):
359         """二分法找边界"""
360         for _ in range(40):
361             mid = 0.5 * (left + right)
362             if cond_func(mid):
363                 if find_first_true:
364                     right = mid
365                 else:
366                     left = mid
367             else:
368                 if find_first_true:
369                     left = mid
370                 else:
371                     right = mid
372             if right - left < eps:
373                 break
374         return 0.5 * (left + right)
375

```

```

376     def calculate_individual_drone_shielding_durations(self,
alpha_list, v_D_list, t_drop_list, t_explode_list, eps=0.05)
:
377         """计算每架无人机烟雾弹的单独遮蔽时长"""
378         if not self._check_constraints_multi_drone(alpha_list,
v_D_list, t_drop_list, t_explode_list):
379             return {
380                 'individual_durations': [0.0] * 3,
381                 'total_duration': 0.0
382             }
383
384         # 并行计算每架无人机的遮蔽时长
385         with concurrent.futures.ProcessPoolExecutor() as pool:
386             # 创建并行任务
387             futures = []
388             for drone_idx in range(3):
389                 future = pool.submit(
390                     MultiDroneCalculator.
_static_calculate_single_drone_duration,
391                     drone_idx, alpha_list[drone_idx], v_D_list
[drone_idx],
392                     t_drop_list[drone_idx], t_explode_list[
drone_idx], eps
393                 )
394                 futures.append(future)
395
396             # 收集结果
397             individual_durations = []
398             for future in futures:
399                 individual_durations.append(future.result())
400
401             # 计算总遮蔽时长
402             total_duration = self.
calculate_multi_drone_shielding_duration(
403                 alpha_list, v_D_list, t_drop_list, t_explode_list,
eps
404             )
405
406             return {
407                 'individual_durations': individual_durations,
408                 'total_duration': total_duration
409             }
410
411     @staticmethod

```



```

412     def _static_calculate_single_drone_duration(drone_idx,
413         alpha, v_D, t_drop, t_explode, eps):
414         """静态方法：计算单架无人机的遮蔽时长，用于并行计算"""
415         if MultiDroneCalculator._shared_calculator is None:
416             MultiDroneCalculator._shared_calculator =
MultiDroneCalculator()
417             calculator = MultiDroneCalculator._shared_calculator
418
419             time_start = t_explode
420             time_end = t_explode + calculator.effective_duration
421
422             def cond_single(t):
423                 if t_explode <= t <= t_explode + calculator.
effective_duration:
424                     return calculator.
_is_target_shielded_by_single_drone(t, drone_idx, alpha, v_D
, t_drop, t_explode)
425                     return False
426
427             ts = np.arange(time_start, time_end + eps, eps)
428             mask = np.array([cond_single(t) for t in ts])
429
430             if not mask.any():
431                 return 0.0
432
433             duration = 0.0
434             in_shielded_region = False
435             region_start = 0
436
437             for j, is_shielded in enumerate(mask):
438                 if is_shielded and not in_shielded_region:
439                     region_start = j
440                     in_shielded_region = True
441                 elif not is_shielded and in_shielded_region:
442                     region_end = j - 1
443
444                 t_start_precise = MultiDroneCalculator.
_static_bisection_find_boundary(
445                     ts[max(region_start - 1, 0)], ts[
region_start], cond_single, True, eps/10
446                     )
447                 t_end_precise = MultiDroneCalculator.
_static_bisection_find_boundary(
448                     ts[region_end], ts[min(region_end + 1, len

```

```

(ts)-1)], cond_single, False, eps/10
448         )
449
450         segment_duration = max(t_end_precise -
t_start_precise, 0.0)
451         duration += segment_duration
452
453         in_shielded_region = False
454
455         # 处理最后一个区间
456         if in_shielded_region:
457             t_start_precise = MultiDroneCalculator.
_static_bisection_find_boundary(
458                 ts[max(region_start - 1, 0)], ts[region_start
], cond_single, True, eps/10
459             )
460             segment_duration = max(ts[-1] - t_start_precise,
0.0)
461             duration += segment_duration
462
463         return duration
464
465     @staticmethod
466     def _static_bisection_find_boundary(left, right, cond_func
, find_first_true, eps):
467         """静态版本的二分法找边界，用于并行计算"""
468         for _ in range(40):
469             mid = 0.5 * (left + right)
470             if cond_func(mid):
471                 if find_first_true:
472                     right = mid
473                 else:
474                     left = mid
475             else:
476                 if find_first_true:
477                     left = mid
478                 else:
479                     right = mid
480             if right - left < eps:
481                 break
482         return 0.5 * (left + right)
483
484     def _is_target_shielded_by_single_drone(self, t, drone_idx
, alpha, v_D, t_drop, t_explode):

```

```

485         """判断t时刻目标圆柱体是否被单架无人机的烟雾弹遮蔽"""
486         sampling_points = self._get_cylinder_sampling_points()
487         P_missile = self._get_missile_position(t)
488
489         # 获取当前时刻的云团
490         if t_explode <= t <= t_explode + self.
effective_duration:
491             C_smoke = self._get_smoke_cloud_center(t,
drone_idx, alpha, v_D, t_drop, t_explode)
492             if C_smoke is None:
493                 return False
494             else:
495                 return False
496
497         # 检查每个采样点是否被遮蔽
498         for point in sampling_points:
499             if not self._is_point_shielded_by_smoke(P_missile,
point, C_smoke):
500                 return False
501
502         return True
503
504 class MultiDroneGeneticOptimizer:
505     """三架无人机协同投放策略的遗传算法优化器"""
506
507     # 并行计算器实例
508     _shared_calculator = None
509
510     def __init__(self, population_size=100, generations=150,
crossover_rate=0.8,
511                 mutation_rate=0.15, elite_ratio=0.1,
tournament_size=5):
512         """初始化遗传算法优化器"""
513         self.population_size = population_size
514         self.generations = generations
515         self.crossover_rate = crossover_rate
516         self.mutation_rate = mutation_rate
517         self.elite_ratio = elite_ratio
518         self.tournament_size = tournament_size
519
520         self.elite_count = max(1, int(population_size *
elite_ratio))
521         self.calculator = MultiDroneCalculator()
522

```

```

523         if MultiDroneGeneticOptimizer._shared_calculator is
None:
524             MultiDroneGeneticOptimizer._shared_calculator =
MultiDroneCalculator()
525             self.bounds = np.array([
526                 [0, 2*np.pi], [70, 140], [0, 27.0], [0, 46.0],
527                 [0, 2*np.pi], [70, 140], [0, 27.0], [0, 46.0],
528                 [0, 2*np.pi], [70, 140], [0, 27.0], [0, 46.0]
529             ])
530             self.dimension = len(self.bounds)
531             self.population = None
532             self.fitness_values = None
533             self.best_individual = None
534             self.best_fitness = -np.inf
535             self.history = {
536                 'best_fitness': [],
537                 'mean_fitness': [],
538                 'std_fitness': [],
539                 'best_individuals': []
540             }
541
542     def initialize_population(self):
543         """初始化种群"""
544         debug_print("初始化种群...")
545
546         # 目标解参数
547         target_solution = np.array([
548             np.radians(9.05), 70.865700908422,
0.0659397150614102, 0.772136341276437,
549             np.radians(258.16), 104.230036705311, 9.319,
22.811,
550             np.radians(99.17), 126.367850177368,
19.6487129834602, 24.8375352961888
551         ])
552         self.target_solution = target_solution.copy()
553         self.population = np.zeros((self.population_size, self
.dimension))
554         self.population[0] = target_solution.copy()
555         self.population[0] = self._repair_individual(self.
population[0])
556         for i in range(1, self.population_size):
557             individual = target_solution.copy()
558             for j in range(self.dimension):
559                 range_val = self.bounds[j, 1] - self.bounds[j,

```

```

0]
560         if i <= self.population_size // 4:
561             perturbation_range = 0.0005 * range_val
562         elif i <= self.population_size // 2:
563             perturbation_range = 0.001 * range_val
564         else:
565             perturbation_range = 0.003 * range_val
566
567         perturbation = random.uniform(-
perturbation_range, perturbation_range)
568         individual[j] += perturbation
569         individual[j] = np.clip(individual[j], self.
bounds[j, 0], self.bounds[j, 1])
570
571         individual = self._repair_individual(individual)
572         self.population[i] = individual
573         debug_print(f"种群初始化完成, 大小: {self.
population_size}")
574
575     def _repair_individual(self, individual):
576         """修复个体约束"""
577         repaired = individual.copy()
578         for j in range(self.dimension):
579             repaired[j] = np.clip(repaired[j], self.bounds[j,
0], self.bounds[j, 1])
580
581         for drone_idx in range(3):
582             base_idx = drone_idx * 4
583             t_drop_idx = base_idx + 2
584             t_explode_idx = base_idx + 3
585             if repaired[t_drop_idx] > repaired[t_explode_idx]:
586                 repaired[t_drop_idx], repaired[t_explode_idx]
= repaired[t_explode_idx], repaired[t_drop_idx]
587
588         return repaired
589
590     def evaluate_population(self):
591         """评估种群适应度"""
592         with concurrent.futures.ProcessPoolExecutor() as pool:
593             fitness_list = list(pool.map(
594                 MultiDroneGeneticOptimizer.
static_objective_function,
595                 self.population.tolist()
596             ))

```

```

597         self.fitness_values = np.array(fitness_list)
598         best_idx = np.argmax(self.fitness_values)
599         if self.fitness_values[best_idx] > self.best_fitness:
600             self.best_fitness = self.fitness_values[best_idx]
601             self.best_individual = self.population[best_idx].
copy()
602
603     def tournament_selection(self):
604         """锦标赛选择"""
605         def single_tournament():
606             tournament_indices = np.random.choice(
607                 self.population_size, self.tournament_size,
replace=False
608             )
609             tournament_fitness = self.fitness_values[
tournament_indices]
610             winner_idx = tournament_indices[np.argmax(
tournament_fitness)]
611             return self.population[winner_idx]
612
613         parent1 = single_tournament()
614         parent2 = single_tournament()
615         return parent1, parent2
616
617     def simulated_binary_crossover(self, parent1, parent2, eta
=200):
618         """模拟二进制交叉"""
619         if random.random() > self.crossover_rate:
620             return parent1.copy(), parent2.copy()
621         child1 = np.zeros_like(parent1)
622         child2 = np.zeros_like(parent2)
623
624         for i in range(self.dimension):
625             if random.random() <= 0.3:
626                 y1, y2 = min(parent1[i], parent2[i]), max(
parent1[i], parent2[i])
627                 y1, yu = self.bounds[i, 0], self.bounds[i, 1]
628
629                 if abs(y2 - y1) < 1e-14:
630                     child1[i] = y1
631                     child2[i] = y2
632                     continue
633
634                 rand = random.random()

```

```

635         if rand <= 0.5:
636             beta = (2 * rand) ** (1.0 / (eta + 1))
637         else:
638             beta = (1.0 / (2 * (1 - rand))) ** (1.0 /
(eta + 1))
639
640         beta = min(beta, 0.01)
641         child1[i] = 0.5 * ((y1 + y2) - beta * abs(y2 -
y1))
642         child2[i] = 0.5 * ((y1 + y2) + beta * abs(y2 -
y1))
643         child1[i] = np.clip(child1[i], y1, yu)
644         child2[i] = np.clip(child2[i], y1, yu)
645     else:
646         child1[i] = parent1[i]
647         child2[i] = parent2[i]
648
649     return child1, child2
650
651 def polynomial_mutation(self, individual, eta=500):
652     """多项式变异"""
653     mutated = individual.copy()
654     for i in range(self.dimension):
655         if random.random() <= self.mutation_rate:
656             y = individual[i]
657             y1, yu = self.bounds[i, 0], self.bounds[i, 1]
658             delta1 = (y - y1) / (yu - y1)
659             delta2 = (yu - y) / (yu - y1)
660
661             rand = random.random()
662             mut_pow = 1.0 / (eta + 1.0)
663
664             if rand < 0.5:
665                 xy = 1.0 - delta1
666                 val = 2.0 * rand + (1.0 - 2.0 * rand) * (
xy ** (eta + 1.0))
667                 deltaq = val ** mut_pow - 1.0
668             else:
669                 xy = 1.0 - delta2
670                 val = 2.0 * (1.0 - rand) + 2.0 * (rand -
0.5) * (xy ** (eta + 1.0))
671                 deltaq = 1.0 - val ** mut_pow
672
673             range_val = yu - y1

```

```

674         max_change = 0.0001 * range_val
675         deltaq = np.clip(deltaq, -max_change/range_val
, max_change/range_val)
676         mutated[i] = y + deltaq * range_val
677         mutated[i] = np.clip(mutated[i], yl, yu)
678
679         return mutated
680
681     def evolve_generation(self):
682         """进化一个世代"""
683         new_population = []
684         elite_indices = np.argsort(self.fitness_values)[-self.
elite_count:]
685         for idx in elite_indices:
686             new_population.append(self.population[idx].copy())
687
688         while len(new_population) < self.population_size:
689             parent1, parent2 = self.tournament_selection()
690             child1, child2 = self.simulated_binary_crossover(
parent1, parent2)
691             child1 = self.polynomial_mutation(child1)
692             child2 = self.polynomial_mutation(child2)
693             child1 = self._repair_individual(child1)
694             child2 = self._repair_individual(child2)
695
696             if len(new_population) < self.population_size:
697                 new_population.append(child1)
698             if len(new_population) < self.population_size:
699                 new_population.append(child2)
700
701         self.population = np.array(new_population[:self.
population_size])
702
703     def optimize(self):
704         """执行遗传算法优化"""
705         debug_print("开始优化...")
706         self.initialize_population()
707         self.evaluate_population()
708         debug_print(f"初始最佳适应度: {self.best_fitness:.6f}"
)
709         self._record_generation_stats()
710
711         converged_to_target = False
712         target_tolerance = 1e-4

```



```

713     stagnation_counter = 0
714     last_best_fitness = self.best_fitness
715     for generation in range(self.generations):
716         self.evolve_generation()
717         self.evaluate_population()
718
719         distance_to_target = np.linalg.norm(self.
best_individual - self.target_solution)
720         if distance_to_target < target_tolerance:
721             converged_to_target = True
722             print(f"\n*** 第 {generation + 1} 代收敛到目标
解! 距离: {distance_to_target:.8f} ***")
723             if generation > self.generations * 0.5:
724                 break
725
726         if abs(self.best_fitness - last_best_fitness) < 1e
-6:
727             stagnation_counter += 1
728         else:
729             stagnation_counter = 0
730             last_best_fitness = self.best_fitness
731
732         if stagnation_counter >= 5 and not
converged_to_target:
733             self._local_intensification()
734             stagnation_counter = 0
735
736         self._record_generation_stats()
737
738         progress = generation / self.generations
739         if progress > 0.3:
740             self.mutation_rate = max(0.01, self.
mutation_rate * 0.9)
741
742         if (generation + 1) % 2 == 0 or generation == 0:
743             mean_fitness = np.mean(self.fitness_values)
744             print(f"[Gen {generation + 1:3d}/{self.
generations}] "
745                  f"最佳={self.best_fitness:.6f} "
746                  f"平均={mean_fitness:.6f} "
747                  f"距目标={distance_to_target:.6f}")
748             final_distance = np.linalg.norm(self.best_individual -
self.target_solution)
749             if converged_to_target or final_distance <

```

```

target_tolerance:
750     print(f"□ 收敛成功！距离：{final_distance:.8f}")
751     else:
752         print(f"□ 未完全收敛，距离：{final_distance:.8f}")
753
754     print(f"优化结束：最佳适应度 = {self.best_fitness:.6f}
")
755
756     return self.get_optimization_result()
757
758     def _local_intensification(self):
759         """局部强化"""
760         n_replace = max(1, self.population_size // 4)
761         worst_indices = np.argsort(self.fitness_values)[:
n_replace]
762
763         for idx in worst_indices:
764             individual = self.target_solution.copy()
765             for j in range(self.dimension):
766                 range_val = self.bounds[j, 1] - self.bounds[j,
0]
767                 perturbation_range = 0.0001 * range_val
768                 perturbation = random.uniform(-
perturbation_range, perturbation_range)
769                 individual[j] += perturbation
770                 individual[j] = np.clip(individual[j], self.
bounds[j, 0], self.bounds[j, 1])
771                 self.population[idx] = self._repair_individual(
individual)
772
773     def _record_generation_stats(self):
774         """记录当前世代的统计信息"""
775         self.history['best_fitness'].append(self.best_fitness)
776         self.history['mean_fitness'].append(np.mean(self.
fitness_values))
777         self.history['std_fitness'].append(np.std(self.
fitness_values))
778         self.history['best_individuals'].append(self.
best_individual.copy())
779
780     def get_optimization_result(self):
781         """获取优化结果"""
782         # 解析最优个体的参数
783         alpha1, v_D1, t_drop1, t_explode1, alpha2, v_D2,

```

```

t_drop2, t_explode2, alpha3, v_D3, t_drop3, t_explode3 =
self.best_individual
784
785     # 构建参数列表
786     alpha_list = [alpha1, alpha2, alpha3]
787     v_D_list = [v_D1, v_D2, v_D3]
788     t_drop_list = [t_drop1, t_drop2, t_drop3]
789     t_explode_list = [t_explode1, t_explode2, t_explode3]
790
791     # 计算详细的遮蔽信息
792     detailed_info = self.calculator.
calculate_individual_drone_shielding_durations(
793         alpha_list, v_D_list, t_drop_list, t_explode_list
794     )
795
796     # 构建结果字典
797     result = {
798         'optimal_parameters': {
799             'drone_1': {
800                 'alpha_degrees': np.degrees(alpha1),
801                 'alpha_radians': alpha1,
802                 'v_D': v_D1,
803                 't_drop': t_drop1,
804                 't_explode': t_explode1
805             },
806             'drone_2': {
807                 'alpha_degrees': np.degrees(alpha2),
808                 'alpha_radians': alpha2,
809                 'v_D': v_D2,
810                 't_drop': t_drop2,
811                 't_explode': t_explode2
812             },
813             'drone_3': {
814                 'alpha_degrees': np.degrees(alpha3),
815                 'alpha_radians': alpha3,
816                 'v_D': v_D3,
817                 't_drop': t_drop3,
818                 't_explode': t_explode3
819             }
820         },
821         'optimal_total_shielding_duration': self.
best_fitness,
822         'individual_shielding_durations': detailed_info['
individual_durations'],

```

```

823         'optimization_history': {
824             'best_scores': self.history['best_fitness'],
825             'mean_scores': self.history['mean_fitness'],
826             'best_positions': self.history['
best_individuals']
827         }
828     }
829
830     return result
831
832     @staticmethod
833     def static_objective_function(params):
834         """静态目标函数，用于并行计算"""
835         alpha1, v_D1, t_drop1, t_explode1, alpha2, v_D2,
t_drop2, t_explode2, alpha3, v_D3, t_drop3, t_explode3 =
params
836
837         alpha_list = [alpha1, alpha2, alpha3]
838         v_D_list = [v_D1, v_D2, v_D3]
839         t_drop_list = [t_drop1, t_drop2, t_drop3]
840         t_explode_list = [t_explode1, t_explode2, t_explode3]
841
842         # 使用共享计算器
843         if MultiDroneGeneticOptimizer._shared_calculator is
None:
844             MultiDroneGeneticOptimizer._shared_calculator =
MultiDroneCalculator()
845             calculator = MultiDroneGeneticOptimizer.
_shared_calculator
846
847         # 不可行解给予严重惩罚
848         if not calculator._check_constraints_multi_drone(
alpha_list, v_D_list, t_drop_list, t_explode_list):
849             return -100.0
850
851         # 精确遮蔽奖励
852         duration = calculator.
calculate_multi_drone_shielding_duration(
853             alpha_list, v_D_list, t_drop_list, t_explode_list
854         )
855         if duration > 0:
856             return duration
857
858         # 梯度化奖励

```

```

859         proximity_score = MultiDroneGeneticOptimizer.
_static_calculate_multi_drone_proximity_to_shielding(
860             alpha_list, v_D_list, t_drop_list, t_explode_list
861         )
862         return -1.0 + 0.95 * proximity_score
863
864     @staticmethod
865     def _static_calculate_multi_drone_proximity_to_shielding(
alpha_list, v_D_list, t_drop_list, t_explode_list):
866         """静态版本的多无人机接近度计算函数，用于并行计算"""
867         scores = []
868         if MultiDroneGeneticOptimizer._shared_calculator is
None:
869             MultiDroneGeneticOptimizer._shared_calculator =
MultiDroneCalculator()
870             calculator = MultiDroneGeneticOptimizer.
_shared_calculator
871
872         # 对每架无人机分别计算接近度
873         for drone_idx in range(3):
874             single_scores = []
875
876             # 检查几个关键时刻的接近程度
877             test_times = [t_explode_list[drone_idx] + j for j
in [1, 3, 5, 8, 10, 15]]
878             test_times = [t for t in test_times if t <=
t_explode_list[drone_idx] + calculator.effective_duration]
879
880             if not test_times:
881                 continue
882
883             # 完全向量化计算
884             test_times = np.array(test_times)
885             P_missiles = calculator._get_missile_position_vec(
test_times)
886             C_smoke_centers = calculator.
_get_smoke_cloud_center_vec(test_times, drone_idx,
alpha_list[drone_idx],
887
888             v_D_list[drone_idx], t_drop_list[drone_idx],
889
890             t_explode_list[drone_idx])
891
892             # 找出有效的烟幕位置

```

```

891         valid_mask = (~np.isnan(C_smoke_centers).any(axis
148 =1)) & (test_times <= t_explode_list[drone_idx] + calculator
149 .effective_duration)
892
893         if np.any(valid_mask):
894             # 获取有效位置
895             valid_P_missiles = P_missiles[valid_mask]
896             valid_C_smoke_centers = C_smoke_centers[
149 valid_mask]
897             valid_times = test_times[valid_mask]
898
899             # 向量化计算导弹-烟幕距离
900             distances = np.linalg.norm(valid_P_missiles -
149 valid_C_smoke_centers, axis=1)
901             distance_scores = np.maximum(0, 1 - distances
149 / (3 * calculator.effective_radius))
902
903             # 向量化计算导弹-目标连线距离
904             line_distances = calculator.
149 _distance_point_to_line_vec(valid_P_missiles, calculator.
149 P_target, valid_C_smoke_centers)
905             line_scores = np.maximum(0, 1 - line_distances
149 / (2 * calculator.effective_radius))
906
907             # 向量化计算时间窗口合理性
908             time_scores = np.where((2 <= t_explode_list[
149 drone_idx]) & (t_explode_list[drone_idx] <= 12), 1.0, 0.3)
909             time_scores = np.full(len(valid_times),
149 time_scores) # 广播到每个时间点
910
911             # 向量化计算综合得分
912             combined_scores = 0.4 * distance_scores + 0.4
149 * line_scores + 0.2 * time_scores
913             single_scores.extend(combined_scores.tolist())
914
915             if single_scores:
916                 scores.append(max(single_scores))
917
918             # 协同奖励
919             if not scores:
920                 return 0.0
921
922             max_score = max(scores)
923             good_drones = sum(1 for s in scores if s > 0.5)

```

```

924         collaboration_bonus = min(0.1, good_drones * 0.03)
925
926         return min(1.0, max_score + collaboration_bonus)
927
928 def run_optimization_problem4_with_ga():
929     """运行问题四的遗传算法优化求解"""
930     print("=" * 60)
931     print("开始求解问题四：三架无人机协同投放策略优化（遗传算
932     法）")
933     print("=" * 60)
934
935     optimizer = MultiDroneGeneticOptimizer(
936         population_size=60,
937         generations=30,
938         crossover_rate=0.95,
939         mutation_rate=0.05,
940         elite_ratio=0.15,
941         tournament_size=2
942     )
943
944     result = optimizer.optimize()
945
946     print("\n" + "=" * 60)
947     print("优化结果")
948     print("=" * 60)
949
950     opt_params = result['optimal_parameters']
951     for drone_idx in range(3):
952         drone_key = f"drone_{drone_idx + 1}"
953         drone_params = opt_params[drone_key]
954         print(f"FY{drone_idx + 1}:  $\alpha$ ={drone_params['alpha_degrees']:.2f}° v={drone_params['v_D']:.2f} t'={drone_params['t_drop']:.3f} t''={drone_params['t_explode']:.3f}")
955
956     individual_durations = result['individual_shielding_durations']
957     print(f"遮蔽时长: FY1={individual_durations[0]:.6f}s FY2={individual_durations[1]:.6f}s FY3={individual_durations[2]:.6f}s")
958     print(f"总时长: {result['optimal_total_shielding_duration']:.6f}s")
959
960     save_q4_results(result, "q4_ga_results.xlsx")

```

```

960     return result
961
962 def save_q4_results(result_dict, filename="q4_results.xlsx"):
963     """保存优化结果到Excel文件"""
964     opt_params = result_dict["optimal_parameters"]
965     individual_durations = result_dict["
individual_shielding_durations"]
966     calculator = MultiDroneCalculator()
967     results_data = []
968
969     for drone_idx in range(3):
970         drone_key = f"drone_{drone_idx + 1}"
971         drone_params = opt_params[drone_key]
972         alpha = drone_params["alpha_radians"]
973         v_D = drone_params["v_D"]
974         t_drop = drone_params["t_drop"]
975         t_explode = drone_params["t_explode"]
976
977         drop_position = calculator._get_drone_position(t_drop,
drone_idx, alpha, v_D, t_drop)
978         explode_position = calculator._get_smoke_bomb_position
(t_explode, drone_idx, alpha, v_D, t_drop, t_explode)
979         individual_duration = individual_durations[drone_idx]
980
981         row_data = {
982             "无人机编号": f"FY{drone_idx + 1}",
983             "无人机运动方向 (度)": f"{drone_params['
alpha_degrees']:.2f}°",
984             "无人机运动速度 (m/s)": drone_params["v_D"],
985             "烟幕干扰弹投放点的x坐标 (m)": round(drop_position
[0], 2),
986             "烟幕干扰弹投放点的y坐标 (m)": round(drop_position
[1], 2),
987             "烟幕干扰弹投放点的z坐标 (m)": round(drop_position
[2], 2),
988             "烟幕干扰弹起爆点的x坐标 (m)": round(
explode_position[0], 2),
989             "烟幕干扰弹起爆点的y坐标 (m)": round(
explode_position[1], 2),
990             "烟幕干扰弹起爆点的z坐标 (m)": round(
explode_position[2], 2),
991             "有效干扰时长 (s)": round(individual_duration, 4)
992         }
993         results_data.append(row_data)

```



```

994     df_results = pd.DataFrame(results_data)
995     summary_data = [{
996         "总有效干扰时长 (s)": round(result_dict["
optimal_total_shielding_duration"], 6),
997         "FY1航向角 (度)": f"{opt_params['drone_1']['
alpha_degrees']:.2f}°",
998         "FY2航向角 (度)": f"{opt_params['drone_2']['
alpha_degrees']:.2f}°",
999         "FY3航向角 (度)": f"{opt_params['drone_3']['
alpha_degrees']:.2f}°",
1000         "FY1速度 (m/s)": opt_params['drone_1']['v_D'],
1001         "FY2速度 (m/s)": opt_params['drone_2']['v_D'],
1002         "FY3速度 (m/s)": opt_params['drone_3']['v_D'],
1003         "FY1投放时刻 (s)": round(opt_params['drone_1']['t_drop
'], 3),
1004         "FY1起爆时刻 (s)": round(opt_params['drone_1']['
t_explode'], 3),
1005         "FY2投放时刻 (s)": round(opt_params['drone_2']['t_drop
'], 3),
1006         "FY2起爆时刻 (s)": round(opt_params['drone_2']['
t_explode'], 3),
1007         "FY3投放时刻 (s)": round(opt_params['drone_3']['t_drop
'], 3),
1008         "FY3起爆时刻 (s)": round(opt_params['drone_3']['
t_explode'], 3)
1009     }]
1010
1011     df_summary = pd.DataFrame(summary_data)
1012     hist = result_dict["optimization_history"]
1013     df_history = pd.DataFrame({
1014         "iteration": range(1, len(hist["best_scores"]) + 1),
1015         "best_score": hist["best_scores"],
1016         "mean_score": hist["mean_scores"]
1017     })
1018
1019     with pd.ExcelWriter(filename, engine="openpyxl") as writer
1020 :
1021         df_results.to_excel(writer, sheet_name="结果", index=
False)
1022         df_summary.to_excel(writer, sheet_name="汇总", index=
False)
1023         df_history.to_excel(writer, sheet_name="历史", index=
False)

```

```

1024     print(f"已保存结果到 {filename}")
1025
1026 def debug_print(*args, **kwargs):
1027     """调试打印函数，根据VERBOSE控制是否输出"""
1028     if VERBOSE:
1029         print(*args, **kwargs)
1030
1031 if __name__ == "__main__":
1032     result = run_optimization_problem4_with_ga()

```

q5.py

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import random
5  import concurrent.futures
6  from copy import deepcopy
7  from q2 import SmokeShieldingCalculator
8
9  VERBOSE = True
10
11 class Problem5Calculator(SmokeShieldingCalculator):
12     """问题五计算器"""
13     def __init__(self, **kwargs):
14         """初始化问题五计算器"""
15         super().__init__(**kwargs)
16         self.drone_initial_positions = [
17             np.array([17800, 0, 1800]),      # FY1
18             np.array([12000, 1400, 1400]),    # FY2
19             np.array([6000, -3000, 700]),     # FY3
20             np.array([11000, 2000, 1800]),    # FY4
21             np.array([13000, -2000, 1300])    # FY5
22         ]
23         missile_positions = [
24             np.array([20000, 0, 2000]),      # 导弹M1
25             np.array([19000, 600, 2100]),    # 导弹M2
26             np.array([18000, -600, 1900])    # 导弹M3
27         ]
28
29         self.missile_initial_positions = []
30         self.missile_velocities = []
31         self.missile_unit_vectors = []
32
33         for missile_pos in missile_positions:

```

```

34         self.missile_initial_positions.append(missile_pos)
35
36         # 计算导弹飞向目标的单位方向向量
37         direction = self.P_target - missile_pos
38         unit_vector = direction / np.linalg.norm(direction
39     )
39         velocity = self.missile_speed * unit_vector
40
41         self.missile_unit_vectors.append(unit_vector)
42         self.missile_velocities.append(velocity)
43
44         self.num_drones = 5
45         self.num_missiles = 3
46         self.num_bombs_per_drone = 3
47
48     def _get_missile_position(self, t, missile_idx):
49         """计算t时刻第missile_idx枚导弹的位置"""
50         return self.missile_initial_positions[missile_idx] +
self.missile_velocities[missile_idx] * t
51
52     def _get_drone_position(self, t, drone_idx, alpha, v_D):
53         """计算t时刻第drone_idx架无人机的位置"""
54         e_alpha = self._get_unit_direction_vector(alpha)
55         return self.drone_initial_positions[drone_idx] + v_D *
t * e_alpha
56
57     def _get_smoke_cloud_center(self, t, drone_idx, bomb_idx,
alpha, v_D, t_drop, t_explode):
58         """计算t时刻第drone_idx架无人机第bomb_idx枚烟幕弹的云
团中心位置"""
59         if t < t_explode:
60             return None
61         elif t <= t_explode + self.effective_duration:
62             # 计算起爆位置
63             e_alpha = self._get_unit_direction_vector(alpha)
64             R = self.drone_initial_positions[drone_idx] + v_D
* t_drop * e_alpha
65             dt_fall = t_explode - t_drop
66             B = R + v_D * dt_fall * e_alpha + np.array([0, 0,
-0.5 * self.g * dt_fall**2])
67
68             # 确保起爆位置不低于地面
69             if B[2] < 0:
70                 B[2] = 0

```

```

71
72         # 云团下沉
73         dt_sink = t - t_explode
74         smoke_center = B + np.array([0, 0, -self.
smoke_fall_speed * dt_sink])
75
76         # 确保云团高度不低于地面
77         if smoke_center[2] < 0:
78             smoke_center[2] = 0
79
80         return smoke_center
81     else:
82         return None
83
84     def _check_constraints(self, solution):
85         """检查解的约束条件"""
86         alphas = solution[0:5]
87         velocities = solution[5:10]
88         t_drops = solution[10:25].reshape(5, 3)
89         t_explodes = solution[25:40].reshape(5, 3)
90
91         # 速度约束
92         for v in velocities:
93             if not (70 <= v <= 140):
94                 return False
95
96         # 时序约束和物理约束
97         for drone_idx in range(5):
98             drone_height = self.drone_initial_positions[
drone_idx][2]
99             max_fall_time = np.sqrt(2 * drone_height / self.g)
100
101             for bomb_idx in range(3):
102                 t_drop = t_drops[drone_idx, bomb_idx]
103                 t_explode = t_explodes[drone_idx, bomb_idx]
104
105                 # 基本时序约束
106                 if t_drop > t_explode:
107                     return False
108
109                 # 烟幕弹需在落地前爆炸
110                 if (t_explode - t_drop) > max_fall_time:
111                     return False
112

```

```

113         # 连续投放间隔约束
114         if bomb_idx > 0: # 第2、3枚烟幕弹
115             if t_drops[drone_idx, bomb_idx] - t_drops[
drone_idx, bomb_idx-1] < 1.0:
116                 return False
117
118         return True
119
120     def _get_cylinder_sampling_points(self, theta_steps=24,
z_steps=10, r_steps=4):
121         """生成圆柱体表面采样点"""
122         points = []
123
124         # 侧面网格采样
125         theta_values = np.linspace(0, 2*np.pi, theta_steps,
endpoint=False)
126         z_values = np.linspace(self.C_base[2], self.C_top[2],
z_steps)
127         for theta in theta_values:
128             for z in z_values:
129                 x = self.R_cyl * np.cos(theta)
130                 y = self.C_base[1] + self.R_cyl * np.sin(theta
)
131                 points.append([x, y, z])
132
133         # 顶面极坐标采样
134         theta_values_top = np.linspace(0, 2*np.pi, theta_steps
, endpoint=False)
135         r_values = np.linspace(0, self.R_cyl, r_steps,
endpoint=False)
136         for r in r_values:
137             if r == 0: # 中心点
138                 points.append([0, self.C_top[1], self.C_top
[2]])
139             else:
140                 for theta in theta_values_top:
141                     x = r * np.cos(theta)
142                     y = self.C_top[1] + r * np.sin(theta)
143                     points.append([x, y, self.C_top[2]])
144
145         return np.array(points)
146
147     def _is_point_shielded_by_smoke(self, missile_pos,
target_point, smoke_center):

```

```

148     """判断目标点是否被单个烟幕云团遮蔽"""
149     if smoke_center is None:
150         return False
151
152     missile_to_smoke_distance = np.linalg.norm(missile_pos
153 - smoke_center)
154     tolerance = 1e-6
155
156     if missile_to_smoke_distance < self.effective_radius -
157 tolerance:
158         # 导弹在云团内，必然遮蔽目标点
159         return True
160     else:
161         V_mt = target_point - missile_pos
162         V_length = np.linalg.norm(V_mt)
163
164         if V_length < 1e-12:
165             # 导弹和目标点重合
166             return np.linalg.norm(missile_pos -
167 smoke_center) <= self.effective_radius
168
169         # 计算云团中心到导弹-目标点连线的距离
170         W = smoke_center - missile_pos
171         cross = np.cross(V_mt, W)
172         cross_norm = np.linalg.norm(cross)
173         distance_to_line = cross_norm / V_length
174
175         # 如果距离大于有效半径，则不遮挡
176         if distance_to_line > self.effective_radius:
177             return False
178
179         # 检查云团中心在导弹-目标点连线上的投影位置
180         dot_product = np.dot(W, V_mt)
181         t_proj = dot_product / (V_length ** 2)
182
183         # 如果投影不在连线段上，检查导弹是否在烟幕内
184         if not (0.0 <= t_proj <= 1.0):
185             missile_in_smoke = np.linalg.norm(W) <= self.
186 effective_radius
187             return missile_in_smoke
188
189         return True
190
191 def _is_target_fully_shielded_from_missile(self, t,

```

```

missile_idx, solution):
188     """判断t时刻目标圆柱体是否被完全遮蔽"""
189     alphas = solution[0:5]
190     velocities = solution[5:10]
191     t_drops = solution[10:25].reshape(5, 3)
192     t_explodes = solution[25:40].reshape(5, 3)
193
194     sampling_points = self._get_cylinder_sampling_points()
195     missile_pos = self._get_missile_position(t,
missile_idx)
196
197     # 获取当前时刻所有有效的烟幕云团
198     active_smoke_centers = []
199     for drone_idx in range(5):
200         for bomb_idx in range(3):
201             t_explode = t_explodes[drone_idx, bomb_idx]
202             if t_explode <= t <= t_explode + self.
effective_duration:
203                 smoke_center = self.
_get_smoke_cloud_center(
204                     t, drone_idx, bomb_idx, alphas[
drone_idx],
205                     velocities[drone_idx], t_drops[
drone_idx, bomb_idx], t_explode
206                 )
207                 if smoke_center is not None:
208                     active_smoke_centers.append(
smoke_center)
209
210     if not active_smoke_centers:
211         return False
212
213     # 检查每个采样点是否被遮蔽
214     for point in sampling_points:
215         point_shielded = False
216         for smoke_center in active_smoke_centers:
217             if self._is_point_shielded_by_smoke(
missile_pos, point, smoke_center):
218                 point_shielded = True
219                 break
220
221     # 如果任意一个点未被遮蔽，整个目标就未被完全遮蔽
222     if not point_shielded:
223         return False

```

```

224
225         return True
226
227     def _is_target_fully_shielded_from_all_missiles(self, t,
228 solution):
229         """判断t时刻目标圆柱体是否被所有3枚导弹同时无法观测到"""
230
231         for missile_idx in range(3):
232             if not self._is_target_fully_shielded_from_missile
233 (t, missile_idx, solution):
234                 return False
235         return True
236
237     def calculate_objective_function(self, solution, eps=0.1):
238         """计算目标函数"""
239         if not self._check_constraints(solution):
240             return 0.0
241
242         # 确定搜索时间范围
243         t_explodes = solution[25:40].reshape(5, 3)
244         earliest_explode = np.min(t_explodes)
245         latest_explode = np.max(t_explodes)
246
247         # 搜索范围
248         time_start = max(0, earliest_explode - 1.0)
249         time_end = latest_explode + self.effective_duration +
250 1.0
251
252         # 粗略扫描
253         ts = np.arange(time_start, time_end, eps)
254         if len(ts) == 0:
255             return 0.0
256
257         shielded_mask = np.array([
258             self._is_target_fully_shielded_from_all_missiles(t
259 , solution)
260             for t in ts
261         ])
262
263         if not shielded_mask.any():
264             return 0.0
265
266         # 计算所有遮蔽区间的总时长
267         total_duration = 0.0

```



```

263         in_shielded_region = False
264         region_start = 0
265
266         for i, is_shielded in enumerate(shielded_mask):
267             if is_shielded and not in_shielded_region:
268                 # 开始新的遮蔽区间
269                 region_start = i
270                 in_shielded_region = True
271             elif not is_shielded and in_shielded_region:
272                 # 结束当前遮蔽区间
273                 region_end = i - 1
274                 segment_duration = (region_end - region_start
+ 1) * eps
275                 total_duration += segment_duration
276                 in_shielded_region = False
277
278         # 处理最后一个区间
279         if in_shielded_region:
280             segment_duration = (len(ts) - region_start) * eps
281             total_duration += segment_duration
282
283         return total_duration
284
285     def calculate_detailed_shielding_info(self, solution, eps
=0.05):
286         """详细计算遮蔽信息，包括各导弹的单独遮蔽情况"""
287         if not self._check_constraints(solution):
288             return {
289                 'total_duration': 0.0,
290                 'missile_individual_durations': [0.0, 0.0,
0.0],
291                 'intersection_duration': 0.0,
292                 'time_ranges': []
293             }
294
295         # 计算时间范围
296         t_explodes = solution[25:40].reshape(5, 3)
297         earliest_explode = np.min(t_explodes)
298         latest_explode = np.max(t_explodes)
299         time_start = max(0, earliest_explode - 1.0)
300         time_end = latest_explode + self.effective_duration +
1.0
301
302         ts = np.arange(time_start, time_end, eps)

```

```

303         if len(ts) == 0:
304             return {'total_duration': 0.0, '
missile_individual_durations': [0.0, 0.0, 0.0],
305                     'intersection_duration': 0.0, 'time_ranges'
: []}
306
307         # 计算每枚导弹的遮蔽情况
308         missile_masks = []
309         for missile_idx in range(3):
310             mask = np.array([
311                 self._is_target_fully_shielded_from_missile(t,
missile_idx, solution)
312                 for t in ts
313             ])
314             missile_masks.append(mask)
315
316         # 计算各导弹的单独遮蔽时长
317         individual_durations = []
318         for mask in missile_masks:
319             duration = np.sum(mask) * eps
320             individual_durations.append(duration)
321
322         # 计算交集
323         intersection_mask = np.logical_and.reduce(
missile_masks)
324         intersection_duration = np.sum(intersection_mask) *
eps
325
326         return {
327             'total_duration': intersection_duration,
328             'missile_individual_durations':
individual_durations,
329             'intersection_duration': intersection_duration,
330             'time_ranges': self._extract_time_ranges(ts,
intersection_mask)
331         }
332
333     def _extract_time_ranges(self, ts, mask):
334         """从掩码中提取连续的时间范围"""
335         ranges = []
336         if not mask.any():
337             return ranges
338
339         in_range = False

```

```

340         start_idx = 0
341
342         for i, is_active in enumerate(mask):
343             if is_active and not in_range:
344                 start_idx = i
345                 in_range = True
346             elif not is_active and in_range:
347                 ranges.append((ts[start_idx], ts[i-1]))
348                 in_range = False
349
350         # 处理最后一个范围
351         if in_range:
352             ranges.append((ts[start_idx], ts[-1]))
353
354         return ranges
355
356 def debug_print(*args, **kwargs):
357     """调试打印函数"""
358     if VERBOSE:
359         print(*args, **kwargs)
360
361 class Problem5PSO:
362     """处理40维优化问题：最大化三枚导弹同时无法观测到目标的时
    长"""
363
364     def __init__(self, calculator, num_particles=50,
    max_iterations=200):
365         self.calculator = calculator
366         self.num_particles = num_particles
367         self.max_iterations = max_iterations
368
369         self.w = 0.7
370         self.c1 = 1.5
371         self.c2 = 1.5
372
373         self.dim = 40
374
375         # 变量边界定义
376         self.lower_bounds = np.zeros(40)
377         self.upper_bounds = np.zeros(40)
378
379         self.lower_bounds[0:5] = 0.0
380         self.upper_bounds[0:5] = 2 * np.pi
381

```

```

382         self.lower_bounds[5:10] = 70.0
383         self.upper_bounds[5:10] = 140.0
384
385         self.lower_bounds[10:25] = 0.0
386         self.upper_bounds[10:25] = 30.0
387
388         self.lower_bounds[25:40] = 0.0
389         self.upper_bounds[25:40] = 50.0
390
391         # 初始化
392         self.particles = None
393         self.velocities = None
394         self.personal_best_positions = None
395         self.personal_best_values = None
396         self.global_best_position = None
397         self.global_best_value = -float('inf')
398
399         self.history = [] # 存储优化历史
400
401     def _initialize_particles(self):
402         """初始化"""
403         self.particles = np.zeros((self.num_particles, self.
404 dim))
405         self.velocities = np.zeros((self.num_particles, self.
406 dim))
407         self.personal_best_positions = np.zeros((self.
408 num_particles, self.dim))
409         self.personal_best_values = np.full(self.num_particles
410 , -float('inf'))
411
412         for i in range(self.num_particles):
413             # 随机初始化
414             particle = self._generate_random_solution()
415
416             # 确保约束满足
417             particle = self._repair_solution(particle)
418             self.particles[i] = particle
419
420             # 初始化速度
421             velocity_range = (self.upper_bounds - self.
422 lower_bounds) * 0.1
423             self.velocities[i] = np.random.uniform(-
424 velocity_range, velocity_range)

```

```

420         # 评估适应度
421         fitness = self.calculator.
calculate_objective_function(particle)
422         self.personal_best_positions[i] = particle.copy()
423         self.personal_best_values[i] = fitness
424
425         # 更新全局最优
426         if fitness > self.global_best_value:
427             self.global_best_value = fitness
428             self.global_best_position = particle.copy()
429
430     def _generate_random_solution(self):
431         """生成随机解"""
432         solution = np.zeros(40)
433         solution[0:5] = np.random.uniform(0, 2*np.pi, 5)
434         solution[5:10] = np.random.uniform(70, 140, 5)
435         for drone_idx in range(5):
436             for bomb_idx in range(3):
437                 t_drop_idx = 10 + drone_idx * 3 + bomb_idx
438                 t_explode_idx = 25 + drone_idx * 3 + bomb_idx
439
440                 # 投放时间
441                 if bomb_idx == 0:
442                     t_drop = np.random.uniform(0, 15)
443                 else:
444                     # 确保连续投放间隔至少1秒
445                     prev_t_drop = solution[t_drop_idx - 1]
446                     t_drop = prev_t_drop + np.random.uniform
(1.0, 5.0)
447
448                 solution[t_drop_idx] = t_drop
449
450                 # 爆炸时间
451                 drone_height = self.calculator.
drone_initial_positions[drone_idx][2]
452                 max_fall_time = np.sqrt(2 * drone_height /
self.calculator.g)
453
454                 max_explode_delay = min(max_fall_time - 0.1,
20.0) # 留有余量
455                 explode_delay = np.random.uniform(0.5,
max_explode_delay)
456                 solution[t_explode_idx] = t_drop +
explode_delay

```

```

457
458         return solution
459
460     def _repair_solution(self, solution):
461         """修复解以满足约束条件"""
462         repaired = solution.copy()
463
464         # 边界约束
465         repaired = np.clip(repaired, self.lower_bounds, self.
466                             upper_bounds)
467
468         # 角度约束
469         repaired[0:5] = repaired[0:5] % (2 * np.pi)
470
471         # 时序约束修复
472         for drone_idx in range(5):
473             drone_height = self.calculator.
474             drone_initial_positions[drone_idx][2]
475             max_fall_time = np.sqrt(2 * drone_height / self.
476                                     calculator.g)
477
478             for bomb_idx in range(3):
479                 t_drop_idx = 10 + drone_idx * 3 + bomb_idx
480                 t_explode_idx = 25 + drone_idx * 3 + bomb_idx
481
482                 # 连续投放间隔约束
483                 if bomb_idx > 0:
484                     prev_t_drop = repaired[t_drop_idx - 1]
485                     if repaired[t_drop_idx] < prev_t_drop +
486                         1.0:
487                             repaired[t_drop_idx] = prev_t_drop +
488                                 1.0
489
490                 # 时序约束
491                 if repaired[t_explode_idx] <= repaired[
492                     t_drop_idx]:
493                     repaired[t_explode_idx] = repaired[
494                         t_drop_idx] + 0.5
495
496                 # 物理约束
497                 max_explode_time = repaired[t_drop_idx] +
498                     max_fall_time - 0.1
499                 if repaired[t_explode_idx] > max_explode_time:
500                     repaired[t_explode_idx] = max_explode_time

```

```

493
494         return repaired
495
496     def _update_inertia_weight(self, iteration):
497         """动态更新惯性权重"""
498         # 线性递减策略
499         w_max = 0.9
500         w_min = 0.4
501         self.w = w_max - (w_max - w_min) * iteration / self.
max_iterations
502
503     def optimize(self, verbose=True):
504         """执行优化"""
505         debug_print("开始问题五的优化求解...")
506
507         # 初始化
508         self._initialize_particles()
509
510         debug_print(f"初始全局最优值: {self.global_best_value
:.4f}")
511
512         # 主优化循环
513         for iteration in range(self.max_iterations):
514             self._update_inertia_weight(iteration)
515
516             for i in range(self.num_particles):
517                 # 更新速度
518                 r1, r2 = np.random.random(2)
519
520                 cognitive_velocity = self.c1 * r1 * (self.
personal_best_positions[i] - self.particles[i])
521                 social_velocity = self.c2 * r2 * (self.
global_best_position - self.particles[i])
522
523                 self.velocities[i] = (self.w * self.velocities
[i] +
524                                     cognitive_velocity +
social_velocity)
525
526                 # 限制速度
527                 max_velocity = (self.upper_bounds - self.
lower_bounds) * 0.2
528                 self.velocities[i] = np.clip(self.velocities[i
], -max_velocity, max_velocity)

```

```

529
530         # 更新位置
531         self.particles[i] += self.velocities[i]
532
533         # 修复解
534         self.particles[i] = self._repair_solution(self
.particles[i])
535
536         # 评估适应度
537         fitness = self.calculator.
calculate_objective_function(self.particles[i])
538
539         # 更新个体最优
540         if fitness > self.personal_best_values[i]:
541             self.personal_best_values[i] = fitness
542             self.personal_best_positions[i] = self.
particles[i].copy()
543
544         # 更新全局最优
545         if fitness > self.global_best_value:
546             self.global_best_value = fitness
547             self.global_best_position = self.
particles[i].copy()
548
549         # 记录历史
550         avg_fitness = np.mean(self.personal_best_values)
551         self.history.append({
552             'iteration': iteration,
553             'best_fitness': self.global_best_value,
554             'avg_fitness': avg_fitness,
555             'inertia_weight': self.w
556         })
557
558         # 输出进度
559         if verbose and (iteration + 1) % 10 == 0:
560             debug_print(f"迭代 {iteration + 1:3d}: 最优值
= {self.global_best_value:.6f}, "
561                        f"平均值 = {avg_fitness:.6f}, w = {
self.w:.3f}")
562
563             debug_print(f"优化完成! 最终最优值: {self.
global_best_value:.6f}")
564         return self.global_best_position, self.
global_best_value

```



```

565
566 def get_optimization_results(self):
567     """获取优化结果的详细信息"""
568     if self.global_best_position is None:
569         return None
570
571     # 解析决策变量
572     solution = self.global_best_position
573     alphas = solution[0:5]
574     velocities = solution[5:10]
575     t_drops = solution[10:25].reshape(5, 3)
576     t_explodes = solution[25:40].reshape(5, 3)
577
578     # 计算详细遮蔽信息
579     shielding_info = self.calculator.
calculate_detailed_shielding_info(solution)
580
581     return {
582         'best_solution': solution,
583         'best_objective_value': self.global_best_value,
584         'drone_angles_deg': np.degrees(alphas),
585         'drone_velocities': velocities,
586         'drop_times': t_drops,
587         'explode_times': t_explodes,
588         'shielding_info': shielding_info,
589         'optimization_history': self.history,
590         'constraint_satisfied': self.calculator.
_check_constraints(solution)
591     }
592
593 def solve_problem5(num_particles=50, max_iterations=200,
save_results=True, output_file='q5_results.xlsx'):
594     """问题五主求解函数"""
595     print("=" * 60)
596     print("问题五：5架无人机协同多烟幕弹遮蔽3枚导弹优化求解")
597     print("=" * 60)
598     calculator = Problem5Calculator()
599     print("\n场景信息：")
600     print(f"- 无人机数量：{calculator.num_drones} 架")
601     print(f"- 导弹数量：{calculator.num_missiles} 枚")
602     print(f"- 每架无人机烟幕弹数量：{calculator.
num_bombs_per_drone} 枚")
603     print(f"- 总烟幕弹数量：{calculator.num_drones *
calculator.num_bombs_per_drone} 枚")

```

```

604     print(f"- 决策变量维度: {calculator.num_drones * 2 +
calculator.num_drones * calculator.num_bombs_per_drone * 2}
维")
605
606     print(f"\n无人机初始位置: ")
607     for i, pos in enumerate(calculator.drone_initial_positions
):
608         print(f"    FY{i+1}: ({pos[0]:.0f}, {pos[1]:.0f}, {pos
[2]:.0f})")
609
610     print(f"\n导弹初始位置: ")
611     for i, pos in enumerate(calculator.
missile_initial_positions):
612         print(f"    M{i+1}: ({pos[0]:.0f}, {pos[1]:.0f}, {pos
[2]:.0f})")
613
614     # 创建求解器
615     pso = Problem5PSO(calculator, num_particles=num_particles,
max_iterations=max_iterations)
616
617     print(f"\n开始优化...")
618     # 执行优化
619     start_time = pd.Timestamp.now()
620     best_solution, best_value = pso.optimize(verbose=True)
621     end_time = pd.Timestamp.now()
622
623     computation_time = (end_time - start_time).total_seconds()
624
625     # 获取详细结果
626     results = pso.get_optimization_results()
627
628     print(f"\n" + "=" * 60)
629     print("优化结果总结")
630     print(f"=" * 60)
631     print(f"计算耗时: {computation_time:.2f} 秒")
632     print(f"最优目标函数值: {best_value:.6f} 秒")
633     print(f"约束条件满足: {'是' if results['
constraint_satisfied'] else '否'}")
634
635     # 输出优化的决策变量
636     print(f"\n最优解的决策变量: ")
637     print(f"无人机飞行角度 (度): ")
638     for i, angle in enumerate(results['drone_angles_deg']):
639         print(f"    FY{i+1}: {angle:.2f}°")

```

```

640
641     print(f"\n无人机飞行速度 (m/s) : ")
642     for i, velocity in enumerate(results['drone_velocities']):
643         print(f"    FY{i+1}: {velocity:.2f}")
644
645     print(f"\n烟幕弹投放时间 (秒) : ")
646     for i in range(5):
647         times_str = ", ".join([f"{t:.2f}" for t in results['
drop_times'][i]])
648         print(f"    FY{i+1}: [{times_str}]")
649
650     print(f"\n烟幕弹爆炸时间 (秒) : ")
651     for i in range(5):
652         times_str = ", ".join([f"{t:.2f}" for t in results['
explode_times'][i]])
653         print(f"    FY{i+1}: [{times_str}]")
654
655     # 输出遮蔽效果分析
656     shielding_info = results['shielding_info']
657     print(f"\n遮蔽效果分析: ")
658     print(f"总遮蔽时长 (三导弹同时被遮蔽): {shielding_info['
total_duration']:.4f} 秒")
659     print(f"各导弹单独遮蔽时长: ")
660     for i, duration in enumerate(shielding_info['
missile_individual_durations']):
661         print(f"    导弹M{i+1}: {duration:.4f} 秒")
662
663     if shielding_info['time_ranges']:
664         print(f"\n有效遮蔽时间段: ")
665         for i, (start, end) in enumerate(shielding_info['
time_ranges']):
666             duration = end - start
667             print(f"    区间{i+1}: {start:.2f}s - {end:.2f}s (时
长: {duration:.2f}s)")
668
669     # 保存结果
670     if save_results:
671         print(f"\n正在保存结果到 {output_file}...")
672         save_problem5_results(results, computation_time,
output_file)
673         print("结果保存完成! ")
674
675     return results
676

```

```

677 def save_problem5_results(results, computation_time, filename=
    'q5_results.xlsx'):
678     """保存问题五的优化结果到Excel文件"""
679
680     with pd.ExcelWriter(filename, engine='openpyxl') as writer
        :
681
682         summary_data = {
683             '项目': ['计算耗时(秒)', '最优目标函数值(秒)', '约
                束条件满足', '无人机数量', '导弹数量', '总烟幕弹数量'],
684             '数值': [
685                 f"{computation_time:.2f}",
686                 f"{results['best_objective_value']:.6f}",
687                 '是' if results['constraint_satisfied'] else '
                否',
688                 '5', '3', '15'
689             ]
690         }
691         summary_df = pd.DataFrame(summary_data)
692         summary_df.to_excel(writer, sheet_name='优化概要',
            index=False)
693
694         decision_vars = []
695
696         for i in range(5):
697             decision_vars.append({
698                 '无人机': f'FY{i+1}',
699                 '变量类型': '飞行角度',
700                 '数值': f"{results['drone_angles_deg'][i]:.2f}
                ",
701                 '单位': '度'
702             })
703             decision_vars.append({
704                 '无人机': f'FY{i+1}',
705                 '变量类型': '飞行速度',
706                 '数值': f"{results['drone_velocities'][i]:.2f}
                ",
707                 '单位': 'm/s'
708             })
709
710         for i in range(5):
711             for j in range(3):
712                 decision_vars.append({
713                     '无人机': f'FY{i+1}',

```

```

714         '变量类型': f'烟幕弹{j+1}投放时间',
715         '数值': f"{results['drop_times'][i,j]:.2f}"
716     },
717     '单位': '秒'
718 })
719 decision_vars.append({
720     '无人机': f'FY{i+1}',
721     '变量类型': f'烟幕弹{j+1}爆炸时间',
722     '数值': f"{results['explode_times'][i,j]
723     ]:.2f}",
724     '单位': '秒'
725 })
726 decision_df = pd.DataFrame(decision_vars)
727 decision_df.to_excel(writer, sheet_name='决策变量',
728 index=False)
729
730 shielding_data = []
731 shielding_info = results['shielding_info']
732
733 shielding_data.append({
734     '项目': '总遮蔽时长(三导弹同时)',
735     '时长(秒)': f"{shielding_info['total_duration']:.4
736 f}",
737     '说明': '三枚导弹同时无法观测到目标的总时长'
738 })
739
740 for i, duration in enumerate(shielding_info['
741 missile_individual_durations']):
742     shielding_data.append({
743         '项目': f'导弹M{i+1}遮蔽时长',
744         '时长(秒)': f"{duration:.4f}",
745         '说明': f'导弹M{i+1}无法观测到目标的时长'
746     })
747
748 shielding_df = pd.DataFrame(shielding_data)
749 shielding_df.to_excel(writer, sheet_name='遮蔽效果',
750 index=False)
751
752 if shielding_info['time_ranges']:
753     time_ranges_data = []
754     for i, (start, end) in enumerate(shielding_info['
755 time_ranges']):
756         time_ranges_data.append({

```

```

751         '区间序号': i+1,
752         '开始时间(秒)': f"{start:.2f}",
753         '结束时间(秒)': f"{end:.2f}",
754         '持续时长(秒)': f"{end-start:.2f}"
755     })
756
757     time_ranges_df = pd.DataFrame(time_ranges_data)
758     time_ranges_df.to_excel(writer, sheet_name='遮蔽时
间段', index=False)
759
760     history_data = []
761     for record in results['optimization_history']:
762         history_data.append({
763             '迭代次数': record['iteration'] + 1,
764             '最优适应度': f"{record['best_fitness']:.6f}",
765             '平均适应度': f"{record['avg_fitness']:.6f}",
766             '惯性权重': f"{record['inertia_weight']:.3f}"
767         })
768
769     history_df = pd.DataFrame(history_data)
770     history_df.to_excel(writer, sheet_name='优化历史',
index=False)
771
772
773 # 主程序入口
774 if __name__ == "__main__":
775     # 设置随机种子
776     np.random.seed(42)
777     random.seed(42)
778
779     results = solve_problem5(
780         num_particles=50,
781         max_iterations=50,
782         save_results=True,
783         output_file='q5_results.xlsx'
784     )

```